
The SPC-700

for SNES Musicians

*Computer architecture, audio synthesis, and the small chip
inside a 1990 video game console — taught from first
principles, in the service of making music.*

Contents

Preface	1
How to Use This Book	3
I The Idea of a Computer	4
1 State, Bytes, and Memory	6
1.1 Bits and bytes	6
1.2 Hexadecimal	8
1.3 Multi-byte values	9
1.4 Memory	10
1.5 Memory layout: regions and conventions	11
1.6 Bytes that mean something	12
1.7 What you should remember	13
1.8 Exercises	13
2 The CPU and Execution	14
2.1 The CPU has its own state	14
2.2 The principal registers	15
2.3 The program counter	16
2.4 What an instruction looks like	17
2.5 Mnemonics and assembly	18
2.6 Control flow: jumps and branches	19
2.7 The stack	20
2.8 Putting it together: the running computer	21
2.9 What you should remember	22
2.10 Exercises	23
3 Instructions	24
3.1 Instructions as state transitions	24

3.2	Operands and addressing modes	25
3.3	A vocabulary check	27
3.4	Status flags	28
3.5	A useful mental model: the instruction as a function	29
3.6	A worked example	30
3.7	What you should remember	31
3.8	Exercises	32
4	Where the SPC-700 Lives	33
4.1	Two CPUs, one console	33
4.2	Why two CPUs?	35
4.3	What “having” 64 KiB of RAM means	36
4.4	The S-DSP, in brief	36
4.5	The mailbox to the main CPU	37
4.6	The clock asymmetry	38
4.7	What you’ll need to do	39
4.8	What you should remember	39
4.9	Exercises	40
II	Programming the SPC-700	41
5	Interlude — Setting Up	43
5.1	What you’ll install	43
5.2	What the stub ROM is, and why you don’t have to write 65816	44
5.3	Your first SPC-700 program	45
5.4	Building it	46
5.5	Running it	46
5.6	What just happened	48
6	The Programmer’s Model	49
6.1	Registers	49
6.2	The PSW flags	50
6.3	The memory map	52
6.4	The I/O registers	53
6.5	Direct page, properly	54
6.6	Putting it together	55
6.7	What you should remember	56
6.8	Exercises	56
6.9	Try this	56

7	Boot and Code Loading	58
7.1	The IPL ROM	58
7.2	The upload protocol	59
7.3	How long does upload take?	60
7.4	After the upload: hiding the IPL ROM	61
7.5	Where your code lives	61
7.6	Resident drivers vs. one-shot uploads	62
7.7	What you should remember	62
7.8	Exercises	63
8	Moving Data	64
8.1	MOV is the core verb	64
8.2	The full move family	65
8.3	The addressing modes, named	67
8.4	Moving 16-bit values: MOVW	68
8.5	The stack: PUSH and POP	68
8.6	XCN: a small but useful instruction	69
8.7	Worked example: copying a block of memory	69
8.8	What you should remember	70
8.9	Exercises	70
9	Arithmetic	72
9.1	Add and subtract: ADC, SBC, and the carry flag	72
9.2	Multi-byte arithmetic	73
9.3	16-bit math the easy way: ADDW, SUBW, CMPW	73
9.4	Compare: CMP	74
9.5	Increment and decrement	75
9.6	Multiply: MUL YA	76
9.7	Divide: DIV YA, X	76
9.8	Decimal adjust: DAA, DAS	77
9.9	Two-address arithmetic in memory	77
9.10	What you should remember	78
9.11	Exercises	78
10	Logic, Shifts, and Bits	80
10.1	Byte-wise logic: AND, OR, EOR	80
10.2	Shifts: ASL and LSR	81
10.3	Rotates: ROL and ROR	81
10.4	XCN, revisited	82
10.5	Single-bit operations on direct page	82

10.6	Test-and-modify on absolute bytes	82
10.7	Branching on a single bit	83
10.8	Bit operations on the carry flag	83
10.9	Worked example: building a voice mask	84
10.10	What you should remember	85
10.11	Exercises	86
11	Control Flow	87
11.1	Conditional branches	87
11.2	The unconditional branch: BRA	88
11.3	Compare-and-branch in one instruction	88
11.4	Jumps: JMP	89
11.5	Subroutine calls: CALL and RET	90
11.6	Software interrupt: BRK	92
11.7	Returning from interrupt: RETI	92
11.8	CPU control instructions	92
11.9	A simple main loop	93
11.10	What you should remember	94
11.11	Exercises	94
III	The S-DSP	96
12	The DSP Register Window	98
12.1	The window	98
12.2	The MOVW idiom	99
12.3	When the DSP actually reads its registers	99
12.4	The DSP register map at a glance	100
12.5	A first interaction: silencing the DSP	103
12.6	Don't write what you don't have to	103
12.7	What you should remember	104
12.8	Exercises	105
13	Voices, Pitch, and Envelopes	106
13.1	A voice, conceptually	106
13.2	SRCN: picking the sample	107
13.3	Pitch: PITCHL and PITCHH	108
13.4	ADSR mode	109
13.5	GAIN mode	110
13.6	VOLL and VOLR	111

13.7 KON: starting a note	112
13.8 KOFF: ending a note	112
13.9 ENDX: knowing when a sample finishes	113
13.10 What you should remember	114
13.11 Exercises	114
14 BRR Samples	115
14.1 The block layout	115
14.2 The header byte	115
14.3 The four predictor filters	116
14.4 The sample directory	117
14.5 What a sample looks like in memory	118
14.6 Encoding samples	118
14.7 Pitfalls	119
14.8 A small but realistic plan	120
14.9 What you should remember	121
14.10 Exercises	121
15 Echo, Noise, and Pitch Modulation	122
15.1 Echo: the famous SNES reverb	122
15.2 A safe echo setup	124
15.3 EDL = 0: the special case	126
15.4 The FIR filter	126
15.5 Echo buffer placement	127
15.6 Noise	128
15.7 Pitch modulation	129
15.8 What you should remember	129
15.9 Exercises	130
IV Music	131
16 Inter-CPU Communication	133
16.1 The mapping	133
16.2 The IPL upload protocol, in detail	134
16.3 Runtime communication	135
16.4 The collision problem	136
16.5 A polling skeleton	136
16.6 Latency	138
16.7 What you should remember	138

16.8 Exercises	139
17 Anatomy of a Sound Driver	140
17.1 What a driver does	140
17.2 The driver's main loop	141
17.3 Tick timing	142
17.4 Per-channel state	144
17.5 A single channel's tick	145
17.6 <code>service_dsp_writes</code>	146
17.7 Voice allocation and SFX priority	147
17.8 A worst-case ARAM budget	148
17.9 What you should remember	148
17.10 Exercises	149
18 A Composer's Workflow	150
18.1 The four stages	150
18.2 Choosing a tool	151
18.3 Where the existing N-SPC drivers fit	152
18.4 The composing experience	152
18.5 Sample preparation, in detail	153
18.6 A first project plan	154
18.7 What you should remember	155
18.8 Exercises	155
19 Tooling, Testing, and SPC Files	156
19.1 Assemblers	156
19.2 Emulators with SPC debuggers	157
19.3 What to debug for	158
19.4 The SPC file format	158
19.5 Creating SPC files	160
19.6 ID666 metadata	160
19.7 Running on real hardware	161
19.8 A small testing checklist	161
19.9 What you should remember	162
19.10 Exercises	162
V Appendices	163
A Full Instruction Reference	164

A.1	Move and stack	164
A.2	Arithmetic	166
A.3	Logic	168
A.4	Shifts and rotates	169
A.5	Bits	169
A.6	Branches and jumps	169
A.7	Flag and CPU control	170
B	DSP Register Map	172
B.1	Per-voice registers (for voice V , base = $V \times 16$)	172
B.2	Global registers	173
B.3	FLG (\$6C) decoded	175
B.4	Echo configuration constants	175
C	Common Pitfalls and Errata	176
C.1	Instruction-level pitfalls	176
C.2	DSP-level pitfalls	177
C.3	BRR pitfalls	178
C.4	IPL and port pitfalls	179
C.5	Undocumented opcodes	179
D	Suggested Learning Path	180
E	A One-Octave Pitch Table	182
E.1	The math	182
E.2	The table	183
E.3	Extending to multiple octaves	184
E.4	Looking up a note	184
E.5	A note on tuning	186
F	Embedding the SPC in a Host Application	188

Preface

This book teaches you how to write music for the Super Nintendo. Not in the sense of “compose a tune in a tracker and click export,” though you will eventually do that. In the sense of *understanding what is happening* when the SNES makes sound, well enough that you could, in principle, write your own audio code from scratch.

That ambition has a real cost: most of this book is not about music. It is about a small chip called the SPC-700, which runs at one megahertz and has sixty-four kilobytes of memory, and which the SNES uses to do nothing but produce sound. To write music for it, you have to understand what kind of thing it is, what it can be told to do, and what it cannot. That requires understanding what a chip *is*, what its memory looks like, and what “telling it to do” something means. Most of this book is about that.

I assume mathematical comfort, not programming experience. You should be willing to read notation like “ $M(a) = v$ ” and equations like “frequency = pitch $\times$ 32000 / 0x1000” without treating them as decorative. But I do not assume that you know assembly language, systems programming, or even the basic vocabulary of CPUs. If you have programmed before, that will help in places; if you have not, the book will build the needed concepts from the ground up.

I am specifically not going to assume:

- you know what a register is
- you know what a CPU does, mechanically, when it runs
- you know what “binary” means in any non-trivial sense
- you know what a stack is, in the computing sense

- you know anything about old chips, including the 6502 family that the SPC-700 is descended from. (If you do, fine; if you don't, this book will not need it.)

What you will know by the end:

- how a small computer represents and manipulates state, mathematically
- what the SPC-700's instruction set is and what each instruction does
- what the S-DSP — the digital signal processor next to the SPC-700 — is, and how the SPC tells it to make sound
- how compressed audio samples (BRR format) work
- what a sound driver is, and how the modern composing tools fit into the picture

This book also has a companion repository:

<https://github.com/spencer2718/spc700-book>

Starting in Chapter 5, some chapters include hands-on exercises that use the repository's stub ROM, SPC-700 source files, BRR samples, and verification notes. The printed text explains the concepts; the repository gives you the small programs to assemble, run, break, and inspect. You do not need the repository for the first four chapters. You will need it once the book turns from explanation to observation.

Read in order. The book builds, and beginning in Chapter 5 the exercises build with it.

A note on sources. The SNES audio system was reverse-engineered by a community of enthusiasts over decades. The two foundational references are the SNESdev wiki and Martin Korth's *fullsnes* document, and where this book describes precise hardware behavior, it is summarizing those. You should read them yourself once this book has prepared you to.

How to Use This Book

The book has four parts and four appendices.

Part I — The idea of a computer. Four chapters that build up, from the ground, what a CPU is and how it works. We will not yet be talking about the SPC-700 specifically; we will be building the vocabulary needed to talk about any small CPU.

Part II — Programming the SPC-700. Seven chapters that take the abstract picture of Part I and apply it to the specific chip: its registers, its instructions, how to write code that runs on it.

Part III — The S-DSP. Four chapters about the audio chip the SPC-700 controls. This is where the actual sound happens. Voices, samples, envelopes, echo.

Part IV — Music. Four chapters on putting it together: how the main game CPU and the audio CPU communicate, what a sound driver is, and what tools and workflows you'll actually use.

The appendices are reference material — the full instruction set, the DSP register map, a list of common pitfalls, a suggested progression for self-study, and a worked pitch table.

Chapter 5 walks you through the required tooling: Mesen2, Asar, and the companion repository. You do not need those tools for Part I, but from Chapter 5 onward the book increasingly asks you to observe real SPC-700 behavior rather than only read about it.

Part I

The Idea of a Computer

Before we can talk about the SPC-700, we need a picture of what kind of object it is. This part builds that picture from first principles, without reference to any particular chip. We start with the static picture — what *state* a computer holds — and move to the dynamic picture — how that state changes over time.

If you find these chapters easy, that is good and expected; they are deliberately careful, and the speed will pick up in Part II. If you find them not-easy, slow down. The rest of the book depends on them.

Chapter 1

State, Bytes, and Memory

A running computer, abstractly, is a very large tuple of bits. Some of those bits are in memory. Some are in special places we will call *registers*. Some encode the instructions the computer is in the middle of executing. All of them, together, are *the state of the machine*.

That is the picture we are going to spend this chapter making concrete. We start with a single bit and build up to addressable memory.

1.1 Bits and bytes

A **bit** is the smallest unit of information: either 0 or 1. We will not need to think about bits as physical voltages or magnetic domains; for our purposes a bit is just an element of the set $\{0, 1\}$.

A **byte** is an ordered collection of 8 bits. We write it as a string of 0s and 1s, with the leftmost bit conventionally called the *most significant* and the rightmost the *least significant*:

1 0 1 1 0 0 1 0

↑

bit 7

(most sig.)

↑

bit 0

(least sig.)

There are $2^8 = 256$ distinct bytes. We can think of a byte as an element of the set $\{0, 1\}^8$ — an 8-tuple of bits.

But it is more useful, almost always, to think of a byte as a *number*. Two interpretations dominate:

Unsigned: read the byte as a base-2 numeral. The byte $b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0$ represents the integer

$$b_7 \cdot 2^7 + b_6 \cdot 2^6 + \cdots + b_1 \cdot 2 + b_0$$

This gives values from 0 to 255 inclusive. The byte 10110010 represents $128 + 32 + 16 + 2 = 178$.

Signed (two's complement): the same bits, but bit 7 is interpreted as a coefficient of -2^7 instead of $+2^7$:

$$-b_7 \cdot 2^7 + b_6 \cdot 2^6 + \cdots + b_1 \cdot 2 + b_0$$

This gives values from -128 to $+127$. The byte 10110010 now represents $-128 + 32 + 16 + 2 = -78$.

Both interpretations exist simultaneously in the bits — the *bits* don't know whether they're signed or unsigned. The interpretation is something the program, or the programmer, imposes from outside. A given byte has both readings available; which one is "right" depends on what the byte is being used for.

Two's complement is not arbitrary. It has a useful algebraic property: addition and subtraction work the same way for signed and unsigned numbers, modulo 256. If you ignore the question of whether a byte is signed, an adder circuit just computes the result modulo 256, and it happens to come out right under both interpretations. We will lean on this when we get to arithmetic.

1.2 Hexadecimal

Writing bytes as 8-bit binary strings is verbose. Writing them in decimal is cryptic — the byte structure is invisible. So we use **hexadecimal**, base 16, where each hex digit corresponds to exactly four bits:

Bits	Hex	Bits	Hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

A byte is two hex digits, since $8 = 4 + 4$. The byte 10110010 becomes B2 in hex; the byte 00000000 is 00; the byte 11111111 is FF.

We will write hex values with a leading dollar sign: \$B2, \$00, \$FF. This is the convention used by SPC-700 assemblers and by the SNESdev wiki. (Some other contexts use 0x instead, like 0xB2. The two notations mean exactly the same thing.)

In this notation:

- The unsigned range of a byte is \$00 to \$FF, i.e. 0 to 255.
- The signed range is \$80 (= −128) to \$7F (= +127).
- The byte \$80 is the most negative byte; \$7F is the most positive; \$FF is either 255 (unsigned) or −1 (signed).

If you can convert fluently between binary, decimal, and hex for small values, the rest of this book will be easier. If not, work through a few conversions by hand. Here is a small drill:

What is \$3C as an unsigned decimal? As an 8-bit binary string? What is \$F0 as a signed decimal? As a binary string? What is the byte whose unsigned value is 100? Express it in hex.

(Answers: 60, 00111100; −16, 11110000; \$64.)

1.3 Multi-byte values

Many quantities don't fit in a byte. The SPC-700 we'll meet later has 64 KiB of memory — that's 65,536 distinct memory locations, which requires 16 bits to address ($2^{16} = 65,536$). Audio sample values are 16-bit. Pitch values are 16-bit. We need a way to represent values larger than a byte.

The standard trick: store a 16-bit value as two bytes — a *low byte* holding bits 0–7, and a *high byte* holding bits 8–15.

For example, the 16-bit value \$1A2B (= 6,699 decimal) is composed of:

- high byte: \$1A
- low byte: \$2B

When we store a 16-bit value in memory, we put both bytes somewhere — but in what order? There are two conventions:

- **Little-endian:** the low byte is at the lower address, the high byte at the higher.
- **Big-endian:** the high byte is at the lower address, the low byte at the higher.

The SPC-700 (and most chips you will encounter today) is little-endian. So the value \$1A2B stored at address \$2000 looks like:

address	\$2000	\$2001
contents	\$2B	\$1A
	↑	↑
	low	high

This is mildly confusing the first time you see it — it looks “backwards” relative to how we write the number — but it is consistent and you will internalize it quickly.

For 32-bit values (4 bytes), the same idea extends. The lowest-order byte sits at the lowest address. We will rarely need 32-bit values in this book.

1.4 Memory

Memory is a finite array of bytes addressable by a number. Mathematically, a memory of size N is a function

$$M : \{0, 1, \dots, N - 1\} \rightarrow \{0, 1, \dots, 255\}$$

where the input is an *address* and the output is the byte stored there. N is the size of memory in bytes.

For the SPC-700, $N = 65,536$, so addresses run from \$0000 to \$FFFF. We say the SPC-700 has a “16-bit address space,” meaning addresses are 16-bit numbers.

Two operations on memory:

- **Read:** given an address a , observe $M(a)$. The contents of memory don’t change.
- **Write:** given an address a and a value v , replace the function M with the new function M' such that $M'(a) = v$ and $M'(x) = M(x)$ for all $x \neq a$.

You can think of memory as a giant array indexed from 0 to $N - 1$, and reads and writes as the array operations you’d expect. The “function” formalism is just a clean way to talk about it.

1.5 Memory layout: regions and conventions

Although memory is, mathematically, just a flat array of N bytes, in practice different *regions* of memory are used for different things by convention. A typical layout for a small computer might look like:

address	contents
\$0000-\$00FF	← small, fast-access "scratch" region for working variables
\$0100-\$01FF	← stack (we'll define this in the next chapter)
\$0200-\$FFFF	← general-purpose RAM: program code, data, samples
\$F0-\$FF	← special I/O addresses (more on this in a moment)
\$FFC0-\$FFFF	← boot ROM

This is approximately what the SPC-700's memory looks like, in fact. We'll see it in detail later.

Two things about this picture to internalize now.

Different regions have different semantics. Most of memory is just “bytes you can read and write.” But some addresses are special. Reading or writing certain addresses might trigger hardware events: starting a sound, sending a value to another chip, controlling a timer. In our memory-as-function metaphor, these addresses are like a function that has *side effects* — calling $M(a)$ or assigning to $M(a)$ does more than just read or write a byte.

Specifically: when we get to Part II, we will see that on the SPC-700, writing to address \$F2 selects which register on the audio chip to talk to, and writing to address \$F3 writes a value to that register. These are not bytes of memory in the normal sense. They are *gateways* — addresses that, when accessed, cause the rest of the world to do something.

This pattern — using normal memory addresses for hardware control — is called **memory-mapped I/O**, and it is universal in small computer architectures. It is why we treat memory uniformly: a “memory write” is the only verb the CPU has, and the hardware decides what each address means.

Some of memory is read-only. The boot ROM at the top of the address space is, on this kind of machine, literally read-only — there are no bits to

flip there. Writes to those addresses are silently ignored, and reads return values that are physically baked into the chip. We treat this as a quirk of the M function: for some addresses, M is fixed by the manufacturer.

1.6 Bytes that mean something

We've described bytes as either unsigned or signed integers. They can also represent other things:

- **Booleans:** by convention, \$00 means false and any nonzero value means true. Or, sometimes, individual bits within a byte each encode a separate boolean — eight booleans packed into one byte.
- **Characters:** in ASCII, the byte \$41 represents 'A', \$42 represents 'B', and so on. (We will not use much ASCII in this book.)
- **Code:** a byte can be the encoding of a CPU instruction. We'll see how in the next chapter.
- **Sample data:** a byte (or pair of bytes) might represent the amplitude of an audio waveform at one moment in time.
- **An address:** a 16-bit value (two bytes) might be a pointer to somewhere else in memory.

A byte by itself does not know which of these it is. The interpretation is supplied by *what the program does with it*. If the program adds two bytes together, they were numbers. If the program looks up an entry in a table indexed by the byte, it was an index. If the program jumps to the address encoded by two bytes, those bytes were a pointer.

This is a deeper point than it might sound. A computer's state is just bytes. The *meaning* of those bytes lives in the program — in the choices about which byte goes where and what is done with it. Computers are very, very good at not caring what their bytes mean.

1.7 What you should remember

- A byte is 8 bits, viewed as a number from 0 to 255 (unsigned) or -128 to $+127$ (signed, two's-complement).
- We write bytes in hexadecimal: $\$00$ to $\$FF$.
- Memory is a finite array of bytes addressable by a number, formalized as a function M : addresses $\rightarrow$ bytes.
- Multi-byte values are stored low byte first (little-endian).
- Some memory addresses are special: reading or writing them triggers hardware behavior (memory-mapped I/O).
- Bytes don't know what they mean. Meaning is supplied by the program.

1.8 Exercises

1. Convert these bytes between hex, unsigned decimal, signed decimal, and binary: $\$2A$, $\$80$, $\$FF$, $\$01$.
 2. The 16-bit value $\$ABCD$ is stored at addresses $\$1000$ - $\$1001$ in little-endian form. What byte is at $\$1000$? What byte is at $\$1001$?
 3. A memory write to $\$F3$ causes a hardware chip to do something (in our example, to write to an audio register). Is there any way the CPU can tell, after the write, what the chip actually did? (Hint: this is a question about the limits of "memory" as a metaphor.)
 4. Design a small region of memory to hold three 16-bit values named x , y , z , starting at address $\$10$. Which addresses contain which bytes?
 5. A byte at address $\$30$ holds the value $\$80$. Without knowing what the byte is for, can you say whether it is "negative"? Why or why not?
-

Chapter 2

The CPU and Execution

In Chapter 1 we built up the *static* picture: a computer’s state is a tuple of bytes, most of them in memory. This chapter introduces the *dynamic* picture — what makes the bytes change.

The thing that makes the bytes change is the **CPU** (Central Processing Unit). A CPU is a small, deterministic machine that, on each “step,” reads a byte (or a few bytes) from memory, interprets them as an instruction, and updates the state of the system accordingly. Then it does it again. And again. Forever, until power is cut.

This chapter answers: where does the CPU get its inputs from? What state does it have of its own? How does it know what to do next? And what does “doing” something even mean, mechanically?

2.1 The CPU has its own state

The CPU is *not* just a function applied to memory. It has internal state of its own. This internal state is small — typically a handful of bytes — but it is critically important because everything the CPU does involves it.

The pieces of internal state are called **registers**. A register is, mathematically, just a named byte (or named multi-byte cell) that lives inside the CPU rather than in memory. It is exactly like a memory location with two differences:

1. It has a *name* rather than an address. Registers are referred to by name in the instructions, not by address.
2. It is *fast*. The CPU can read and write its own registers in a fraction of the time it takes to read or write memory.

That second property is the only reason registers exist. If memory were as fast as the CPU's internal state, there would be no need for separate registers — the CPU could just operate on memory directly. But memory is slower, often a lot slower. So CPUs have a small number of fast internal slots — registers — and most arithmetic, comparison, and logic is done by moving values from memory into registers, doing the work there, and moving the results back to memory.

The number of registers is a design choice. Modern desktop CPUs have dozens. Small old CPUs like the SPC-700 have a handful — fewer than ten registers, total, holding state that fits in well under twenty bytes. This is part of why programming a small CPU feels like a constraint puzzle: at any moment, almost everything you care about is in memory, and you have only a few slots to bring values into for work.

2.2 The principal registers

Different CPUs have different register sets, but a few register *roles* show up across almost all of them. The SPC-700 has the same roles as most small CPUs of its era; we'll meet its specific registers in Part II. For now, here are the roles in the abstract.

The accumulator (A). A general-purpose register where most arithmetic happens. The accumulator is the “main slot” — when you want to add two values, you typically load one of them into A and add the other to it. The result ends up in A.

Index registers (X, Y, possibly more). Auxiliary registers, often used for “where in a table am I” purposes. Sometimes used for arithmetic too, but their canonical role is to hold offsets or counters.

The program counter (PC). A 16-bit register holding the address of the next instruction to execute. This one is special and we'll spend the next section on it.

The stack pointer (SP). Holds the address of the top of a stack used for temporary storage (we'll define "stack" formally below). Like PC, it's automatically managed by certain instructions.

The flag register (also called PSW, "Processor Status Word"). A single byte where each individual bit is a boolean flag reporting on something. We'll cover this in detail in Part II; for now, know that after every arithmetic operation, certain bits in this register are set or cleared depending on the result.

These registers, plus a few others depending on the CPU, are the entirety of the CPU's internal state. The SPC-700 has exactly the registers above and nothing more. Its complete internal state — what makes one moment in time of the running CPU different from another — is captured by maybe ten bytes.

The state of the system as a whole is therefore:

$$\text{state} = (\text{registers}, \text{memory})$$

— a small tuple of register bytes plus a 64-KiB memory function. That's it. That's the whole computer.

2.3 The program counter

Of all the registers, the **program counter** is the one that makes the CPU "go."

The program counter, **PC**, is a register holding the address (in memory) of the next instruction to execute. The CPU's basic operation is:

1. Read the bytes at memory address PC.
2. Interpret those bytes as an instruction.

3. Execute the instruction (which usually means: change some bytes in memory and/or registers).
4. Update PC so it points to the byte after the instruction we just executed.
5. Go to step 1.

This is called the **fetch–decode–execute cycle**, and every CPU you’ve ever interacted with does some version of it. The SPC-700 does it about 1,024,000 times per second.

Two consequences of this picture deserve emphasis.

Programs and data live in the same memory. There is no separate “code memory” and “data memory” — the bytes that the CPU executes as instructions are bytes in the same M function as the bytes that store your variables. The CPU executes whatever bytes happen to be at PC; if PC ends up pointing into a region containing data, the CPU will dutifully try to interpret that data as instructions, with whatever results that produces (usually nonsense). It is the programmer’s job to keep PC pointing at things that were *intended* to be instructions.

The CPU does not “see” the program globally. It sees one instruction at a time. It does not know what the next instruction will be until it has fetched it. It does not know what came before. Its entire view of the program is “what’s at this address?”, advanced byte by byte. A “program” is a *static* arrangement of bytes in memory; the *running* of the program is a trajectory of PC through that arrangement.

2.4 What an instruction looks like

We’ve said the CPU interprets bytes at PC as an “instruction.” What does that mean concretely?

An **instruction** is a sequence of one or more bytes encoding (a) what operation to perform and (b) any data the operation needs. The first byte of an instruction is the **opcode** — a number that tells the CPU which of its

operations is being requested. The bytes after the opcode, if any, are the **operands** — additional data the instruction needs.

For example, on the SPC-700, the byte \$E8 is the opcode for “load the accumulator with an immediate value.” This instruction is two bytes long: the opcode \$E8, followed by one operand byte that specifies what value to load. So the two-byte sequence

\$E8 \$42

means “load A with the value \$42.” After executing this instruction, the register A contains \$42, and PC has advanced by two (since the instruction was two bytes long), so it now points at the next instruction.

A different opcode, \$E5, means “load A with the byte at a 16-bit absolute address.” This is a three-byte instruction: opcode plus two address bytes. So

\$E5 \$30 \$20

means “load A with the byte at memory address \$2030” — the address being formed from the operand bytes in little-endian order. After execution, A holds whatever byte is at \$2030, and PC has advanced by three.

Different opcodes have different lengths. Some are one byte (no operand), some two, some three. The CPU knows the length of each opcode by hardware design — when it fetches a \$E8 it knows to fetch one more byte; when it fetches a \$E5 it knows to fetch two more.

The complete map from opcodes to operations is the **instruction set** of the CPU. The SPC-700’s instruction set has 256 distinct opcodes — every possible byte value \$00 through \$FF is a valid opcode, and each one means something specific.

2.5 Mnemonics and assembly

If you and I had to communicate by reading and writing strings of opcodes — \$E8 \$42 \$E5 \$30 \$20 . . . — we would lose our minds. So we don’t. We

use a notation called **assembly language**, where each instruction is written with a short text **mnemonic** that names the operation, followed by the operands in a human-readable form.

The two examples from the previous section, in SPC-700 assembly:

```
mov a, #$42      ; equivalent to bytes $E8 $42
mov a, !$2030    ; equivalent to bytes $E5 $30 $20
```

`mov` is the mnemonic for “load.” The `#` prefix on `$42` means “this is a literal value, not an address.” The `!` prefix on `$2030` means “this is an absolute 16-bit address.” `A ;` introduces a comment.

A program written in assembly is converted to actual bytes by a tool called an **assembler**. The assembler is a function that takes a text file of assembly and produces a sequence of bytes — the encoded instructions, ready to be placed in memory and executed.

This is a one-to-one translation: each line of assembly becomes a specific sequence of bytes. There is no “compilation” in the modern sense, no optimization, no abstraction — just a direct encoding from human-readable mnemonics to machine-readable opcodes. If you could memorize the opcode for every mnemonic plus all the addressing-mode conventions (don’t try), you could assemble programs by hand.

We will use SPC-700 assembly notation throughout this book. You can read it as “what the CPU does,” and the assembler does the bookkeeping of turning it into bytes.

2.6 Control flow: jumps and branches

So far the CPU has executed instructions in sequence, advancing PC by the length of each instruction. What if we want to do something other than execute the next instruction?

We change PC. Some instructions, called **jumps** or **branches**, exist specifically to write a new value to PC instead of letting it advance by one

instruction-length. After such an instruction executes, the CPU's next fetch happens from the new PC, and execution continues from there.

A jump is unconditional — it always writes the new PC. A **conditional branch** writes the new PC only if some condition is true (usually, only if a particular flag in PSW is set or clear). A conditional branch is therefore the way the CPU implements “if statements,” “loops,” and any decision-making at all.

We'll cover jumps and branches in detail in Chapter 11. For now, the picture is: PC moves linearly forward unless an instruction explicitly tells it to move somewhere else.

2.7 The stack

One more piece of state to introduce. Many CPU operations need a small amount of “scratch” storage that doesn't have to live anywhere in particular — a place to push something, do something, and pop it back. The classic example: when one routine calls another, we need to remember where to return to; when the called routine finishes, we need to come back. These return addresses can pile up, especially if routines call other routines, which call still others.

The **stack** is a region of memory used as a last-in-first-out (LIFO) data structure for exactly this kind of scratch storage. The CPU maintains a register called the **stack pointer (SP)**, which holds the address of the current top of the stack.

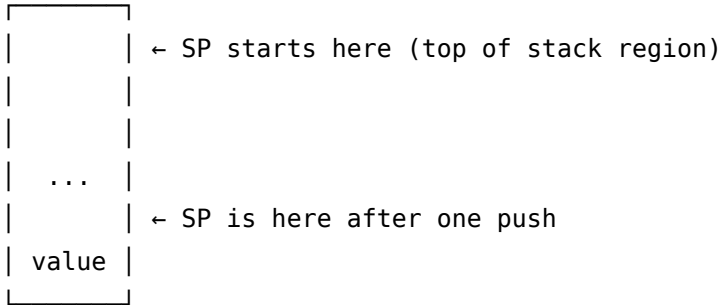
Two operations:

- **PUSH** a value: write the value to memory at SP, then decrement SP.
- **POP** a value: increment SP, then read the value from memory at SP.

Why does SP decrement on push and increment on pop? Because the stack conventionally grows *downward* in memory — the “top” of the stack is at a low address, and as we push, the top moves to even lower addresses. This is purely a convention but it is universal.

A useful mental model: imagine a region of memory dedicated to the stack, with SP starting at the high end. Each push writes a byte at SP and moves SP one byte lower. Each pop moves SP one byte higher and reads what's there. As long as you push and pop in matched pairs, the stack acts like a tower of plates.

Memory



The stack is used by the CPU automatically for some operations (like calling and returning from subroutines), and it is available to the programmer for any temporary storage they want. We'll see it in action throughout Part II.

2.8 Putting it together: the running computer

We can now describe what a running computer is, mathematically. The state of the computer at any moment is the tuple

$$S = (A, X, Y, SP, PSW, PC, M)$$

where the first six are register values (most one byte, PC and SP being special) and M is the memory function. Execution is a sequence of states $S_0, S_1, S_2, \dots$, where S_{t+1} is computed from S_t by:

1. Reading bytes from M starting at PC to fetch the next instruction.
2. Decoding the opcode.
3. Applying the corresponding state transition — updating A, X, Y, PSW, M , and PC according to what the instruction does.

Each instruction is, formally, a function from the set of CPU states to itself. The CPU's instruction set is a finite collection of such functions, indexed by opcode. The "execution" of a program is just iterated function application: take the current state, look at the opcode at PC, apply that function, get the next state, repeat.

This formalism is not a metaphor. It is what is happening, mathematically, inside the chip. Engineers build the chip so that it implements these functions in physical circuits, and the act of programming the chip is the act of arranging bytes in memory so that the resulting trajectory through state space accomplishes something we want — like making the right voltage changes at the audio output jack to play a tune.

2.9 What you should remember

- A CPU has a small set of internal registers — fast-access named slots — separate from memory.
- The most important register is the **program counter (PC)**, holding the address of the next instruction.
- The CPU executes by fetching bytes at PC, decoding them as an instruction, executing it, and advancing PC. This is the **fetch–decode–execute cycle**.
- An **instruction** is one or more bytes: an opcode plus optional operands.
- We write programs in **assembly language** with mnemonics; an **assembler** translates them to bytes.
- **Jumps** and **branches** modify PC explicitly to change the order of execution.
- The **stack** is a LIFO scratch region of memory, indexed by the stack pointer SP.
- The state of the running computer is a tuple of registers and memory; execution is iterated function application.

2.10 Exercises

1. The CPU has, abstractly, a function `execute_one_instruction: State → State`. Suppose the SPC-700 runs at 1.024 MHz and the average instruction takes 4 cycles. How many times per second does this function get applied?
 2. PC currently holds `$0200`. The byte at `$0200` is the opcode for a 3-byte instruction. After the instruction executes, what does PC hold (assuming it's not a jump)?
 3. Why are registers faster than memory? (You can answer this in physical terms or just take it as a stipulation.)
 4. After pushing the bytes `$11`, `$22`, `$33` onto the stack in that order, then popping one byte, what is the popped value? What's still on the stack?
 5. A “self-modifying program” writes new bytes into the region of memory that contains its own code. Does the CPU notice, in any special way, that these bytes are “code”? Or do they just become whatever the CPU executes when PC reaches them?
-

Chapter 3

Instructions

We have, in the last chapter, sketched what a CPU does: fetch bytes, decode them, transform state. We have not yet looked at what the *menu* of available transformations looks like. This chapter does that, in the abstract — what kinds of things instructions do, how they specify their operands, and how they communicate side information through flags.

We are still not yet talking about the SPC-700 specifically. We are building vocabulary. Almost everything in this chapter applies to any small CPU, and the SPC-700 will turn out to be a particularly tidy instance.

3.1 Instructions as state transitions

Every instruction is a function from the machine state to itself. Different instructions modify different parts of the state.

Some examples of what an instruction might do:

- **Move data.** “Set register A equal to the byte at memory address \$30.” This changes A; it leaves memory and other registers alone.
- **Arithmetic.** “Add the byte at memory address \$30 to register A, replacing A with the sum.” This changes A and may change some flag bits (e.g. whether the result was zero).
- **Logical operation.** “Set A equal to A bitwise-AND the byte at \$30.” Changes A and flags.

- **Jump.** “Set PC equal to the value \$0200.” Changes PC and nothing else.
- **Conditional branch.** “If the zero flag is set, set PC equal to PC + signed offset.” May or may not change PC.
- **Stack operation.** “Push A onto the stack.” Changes SP and writes one byte of memory.
- **No-op.** “Do nothing for one instruction time.” Changes only PC (which advances normally).

The total catalog of operations available is the **instruction set**. The SPC-700’s instruction set has on the order of 70 distinct *operations*, each available in one or more variants. With variants for different operand types (described below), the total opcode count fills all 256 possible byte values.

We’ll lay out the SPC-700’s instruction set systematically in Part II. The categories we’ll use are:

1. **Data movement.** MOV-style instructions that copy bytes between registers and memory.
2. **Arithmetic.** Add, subtract, compare, increment, decrement, multiply, divide.
3. **Logic.** AND, OR, XOR, shifts, rotates.
4. **Bit manipulation.** Set, clear, test individual bits.
5. **Control flow.** Jumps, branches, subroutine calls and returns.
6. **CPU control.** Set or clear flags, no-op, etc.

Almost every CPU has these same six categories, with minor variations.

3.2 Operands and addressing modes

Most instructions need to be told what to operate on. The details of *how* the operands are specified — where the data comes from, where it goes — are called **addressing modes**, and they’re one of the most important things to get straight about an instruction set.

Consider a simple operation: “load A with a byte.” Where does the byte come from? Several possibilities:

Immediate. The byte is encoded directly in the instruction itself. The operand byte that follows the opcode is the value to load. We’ve seen this:

```
mov a, #$42
```

The # says “this is a literal.” After this instruction, $A = \$42$. Cost: 2 bytes (opcode + value); fast.

Direct (or “absolute,” or “memory”) addressing. The instruction encodes a memory address; the byte at that address is the value:

```
mov a, !$2030
```

The ! denotes an absolute 16-bit address. After this instruction, $A = M(\$2030)$. Cost: 3 bytes (opcode + two address bytes); a memory read.

Short-direct (“zero-page” or “direct-page”) addressing. Like direct, but the address is only one byte instead of two — restricting which addresses can be reached but making the instruction smaller and faster:

```
mov a, $30
```

This loads A from a memory address constructed by combining a single byte (\$30) with an implicit high byte (often \$00, but it can vary on some CPUs). On the SPC-700, the high byte is configurable — this is a feature called “direct page” that we’ll cover in Part II. Cost: 2 bytes; faster than absolute.

Indexed addressing. Take a base address and add the value of an index register to it before reading:

```
mov a, !$2000+x
```

This loads A from $M(\$2000 + X)$. If X holds 5, A becomes $M(\$2005)$; if X holds 100, A becomes $M(\$2064)$. Indexed addressing is how you walk through arrays — set X to the index, and one instruction reaches the right element.

Indirect addressing. Treat memory as containing a *pointer* — an address itself — and load through that pointer. There are several variants. One important one: “treat the two bytes at direct-page address `dp` as a 16-bit pointer; add `Y`; load from that address”:

```
mov  a, [$30]+y
```

If the bytes at `$30` and `$31` form the pointer `$2000`, and `Y` holds 5, this loads `A` from `M($2005)`. This is how you implement “follow this pointer with an offset” — fundamental for traversing data structures.

Register-to-register. Just copy from one register to another:

```
mov  a, x      ; A = X
```

No memory access at all. Fast.

These are not all the modes — there are more on a real CPU — but they cover the main ideas. Each addressing mode is a different *way of computing where the operand is* before the actual operation runs. Most instructions are available in multiple addressing modes, each as a separate opcode.

The cost of an instruction in CPU cycles depends heavily on the addressing mode. Immediate and register-to-register operations are cheap. Direct-page addressing is moderate. Absolute addressing costs a bit more. Indirect modes cost the most, because the CPU has to do extra reads to compute the final address.

This cost difference matters. On a 1-MHz chip running music in real time, the difference between 3 cycles and 6 cycles per instruction adds up. Good programs use the cheap modes when they can.

3.3 A vocabulary check

Let’s make sure the words are all settled:

- **Operation:** what the instruction *does*, abstractly. “Add to `A`,” “jump to address,” “set bit,” etc.

- **Opcode:** the byte that encodes a specific operation in a specific addressing mode. Different addressing modes for the same operation have different opcodes.
- **Operand:** the data the instruction works on, or where to find that data.
- **Addressing mode:** the convention for interpreting the operand bytes — immediate, direct, indexed, etc.
- **Mnemonic:** the human-readable name of an operation, like `MOV`, `ADD`, `JMP`.
- **Instruction:** a complete unit of execution: opcode plus any operand bytes.

In assembly language, we name an instruction by writing its mnemonic followed by its operands in some addressing-mode notation. The assembler picks the right opcode based on the mnemonic and the form of the operands. We will do this constantly in Part II.

3.4 Status flags

We've mentioned **flags** several times. Let's define them properly.

The CPU has a register — called the **PSW** (Processor Status Word), the **flag register**, or sometimes just “the flags” — where each individual *bit* is a separate boolean. Different bits report on different things. The names and meanings vary by CPU; here are the ones the SPC-700 has, with what each one means:

- **N (Negative):** set to bit 7 of the most recent arithmetic or logical result. Equivalent to “the result, interpreted as signed, is negative.”
- **V (Overflow):** set when a signed arithmetic operation produced a result that didn't fit. (Unsigned overflow is reported by `C`; signed overflow is reported by `V`.)
- **Z (Zero):** set when the most recent arithmetic, logical, or load result was zero.

- **C (Carry)**: set when an unsigned addition produced a value that didn't fit (i.e., greater than 255), or used as a "borrow" indicator after subtraction. Also used by shifts and rotates.
- **H (Half-carry)**: set when there was a carry from bit 3 to bit 4 of an arithmetic result. Used for binary-coded-decimal (BCD) operations; we will essentially never need it.
- **I (Interrupt enable)**: allows or disallows the CPU to be interrupted by external events. On the SNES, the SPC-700 has no working interrupt sources, so this flag is essentially decorative.
- **P (direct page)**: selects between two possible direct-page regions. We'll cover this in Part II.
- **B (Break)**: set when a special "break" instruction has been executed. Mostly a debugging artifact.

The four most-used flags in everyday code are **C**, **Z**, **N**, **V**. The others rarely come up.

The crucial property of flags: **they are set as side effects**. Most arithmetic and logical instructions update some flags as a byproduct of computing their primary result. After `ADD A, $30`, the value of `A` is the sum, and the flags `Z`, `C`, `N`, `V`, `H` have all been updated based on that sum. You don't have to ask for this; it's automatic.

This is what makes conditional execution possible. After comparing two values, the flags hold the answer to several questions about the comparison: were they equal (`Z`), was one greater than the other (`C`, `N`), did one overflow (`V`). A subsequent conditional branch can then act on those answers.

The pattern is: do the operation that sets the flags, then do a branch that tests the flag. We'll see this constantly.

3.5 A useful mental model: the instruction as a function

For each instruction, you can think of it as a function with this signature:

instruction : State $\rightarrow$ State

Specifying an instruction means specifying:

1. What parts of the state it reads from. (For example: “the A register, and the byte at the address computed by the addressing mode.”)
2. What parts of the state it writes to. (For example: “the A register, and the C, Z, N, V flags.”)
3. What the new values are, computed as a function of the old.
4. How PC advances. (Usually just by the length of the instruction; for jumps and branches, it’s set explicitly.)

If you can answer those four questions for an instruction, you know exactly what it does. The rest of this book — and the reference material in Appendix A — is organized around answering those questions for every instruction the SPC-700 has.

3.6 A worked example

Here’s a small sequence of instructions, with the state changes spelled out at each step. We’ll start with an empty state — all registers and flags zero, all of memory zero. Then we’ll execute four instructions and trace what happens.

The instructions:

```
mov  a, #$10      ; opcode $E8, operand $10 (2 bytes)
mov  $30, a        ; opcode $C4, operand $30 (2 bytes)
mov  a, #$05      ; opcode $E8, operand $05 (2 bytes)
mov  $31, a        ; opcode $C4, operand $31 (2 bytes)
```

Suppose these instructions live in memory starting at address \$0200. The byte sequence in memory is:

```
$0200: $E8 $10 $C4 $30 $E8 $05 $C4 $31
```

Initial state: PC = \$0200, A = \$00, M(\$30) = \$00, M(\$31) = \$00, all flags = 0.

Step 1: PC is \$0200. The byte there is \$E8 — the opcode for “load A immediate.” The operand byte at \$0201 is \$10. The CPU sets A = \$10, advances PC by 2 to \$0202, and updates the Z and N flags based on the loaded value. (Z = 0 because \$10 ≠ 0; N = 0 because bit 7 of \$10 is 0.)

State: PC = \$0202, A = \$10, memory unchanged, Z = 0, N = 0.

Step 2: PC is \$0202. The byte there is \$C4 — opcode for “store A to direct-page address.” The operand byte at \$0203 is \$30. The CPU writes A’s value (\$10) to memory at \$30, and advances PC by 2 to \$0204. (No flags change on store.)

State: PC = \$0204, A = \$10, M(\$30) = \$10, Z = 0, N = 0.

Step 3: PC is \$0204. Byte is \$E8, operand \$05. CPU sets A = \$05, advances PC to \$0206.

State: PC = \$0206, A = \$05, M(\$30) = \$10.

Step 4: PC is \$0206. Byte is \$C4, operand \$31. CPU writes \$05 to memory at \$31. PC advances to \$0208.

Final state: PC = \$0208, A = \$05, M(\$30) = \$10, M(\$31) = \$05, all flags = 0 except whatever was set by the last load.

What did the program do? It put the 16-bit value \$0510 (in little-endian) at memory addresses \$30-\$31. (Recall: little-endian means low byte first. The low byte \$10 is at \$30; the high byte \$05 is at \$31. The 16-bit value is \$0510.)

Read the steps until they feel mechanical. The CPU is a *very* dumb machine. It does exactly what it is told, byte by byte. There is no magic.

3.7 What you should remember

- An instruction is a function from CPU state to CPU state.

- Operations are categorized: data movement, arithmetic, logic, bit manipulation, control flow, CPU control.
- An **addressing mode** is a way of specifying where an operand lives: immediate, direct, indexed, indirect, etc.
- Cheaper addressing modes (immediate, register, direct) are faster than expensive ones (indirect).
- The CPU has a flag register (PSW) where each bit reports a different boolean about the most recent operation.
- Flags are set automatically as side effects of arithmetic and logical instructions. This is what makes conditional branching possible.
- An instruction's full specification: what it reads, what it writes, what the new values are, how PC advances.

3.8 Exercises

1. The instruction `mov a, [$30]+y` loads A from $M(M(\$30) \mid M(\$31) \ll 8 + Y)$ — that is, from the address formed by treating bytes \$30 and \$31 as a little-endian pointer, plus Y. If $M(\$30) = \00 , $M(\$31) = \20 , $Y = \$05$, what address does the load read from?
 2. The instruction `add a, #$05` adds 5 to A. After executing it, which flags have been updated automatically as side effects?
 3. An opcode is a single byte; there are 256 of them. The SPC-700 has roughly 70 distinct *operations*. Why are there 256 opcodes if there are only 70 operations?
 4. Compute the byte-by-byte trace of the four-instruction example above, but suppose memory at \$30 and \$31 already held \$AA and \$BB before execution. What does the final state look like?
 5. Why is it useful to have multiple addressing modes for the same operation, instead of a single canonical form?
-

Chapter 4

Where the SPC-700 Lives

We have spent three chapters on what a CPU is. We now zoom in on a specific one — the SPC-700 — and the larger system it sits inside.

The SPC-700 is the CPU dedicated to audio inside a 1990s home video game console: the Super Nintendo Entertainment System (SNES, or in Japan, Super Famicom). It is one of two CPUs in the system, and the only one that has anything to do with sound. Understanding why there are two CPUs, and what each does, is essential to understanding what kind of programming we are about to do.

4.1 Two CPUs, one console

A SNES, as far as we care, has two main computational chips: a **main CPU** and an **audio subsystem**.

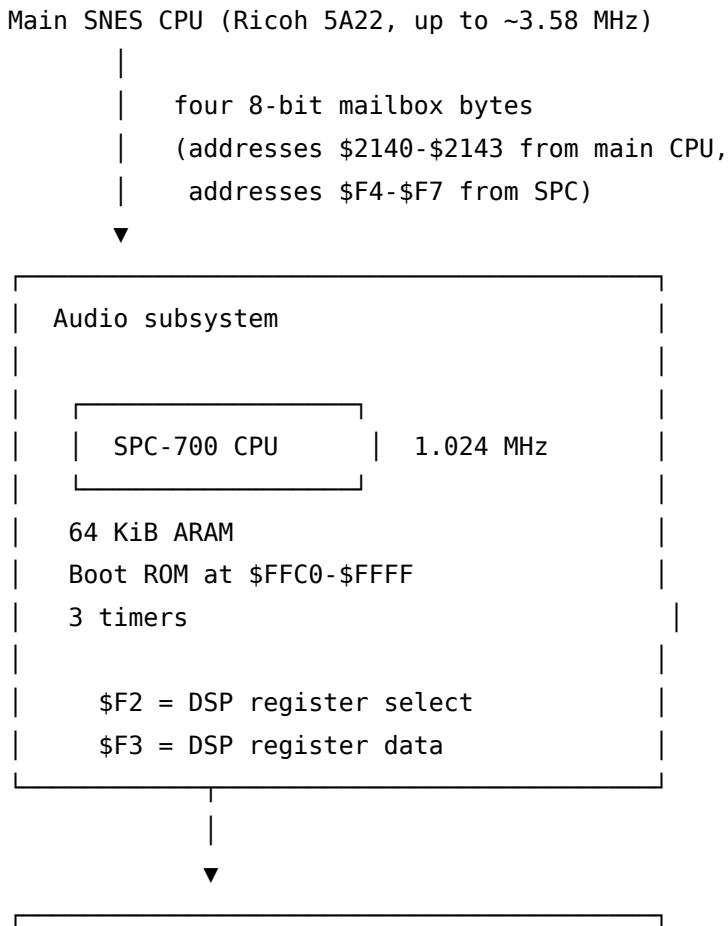
The main CPU is a Ricoh 5A22, a customized 16-bit processor running at up to about 3.58 MHz. (The 5A22 is part of the 65816 architecture family, and SNES homebrewers usually call it “the 65816” or “the main CPU” interchangeably. Either name refers to this chip.) This is the chip that executes the game’s logic — physics, AI, level state, controller input, video timing. When you press a button on the controller, this is the chip that reads it. When the screen scrolls, this is the chip that decides where it scrolls to.

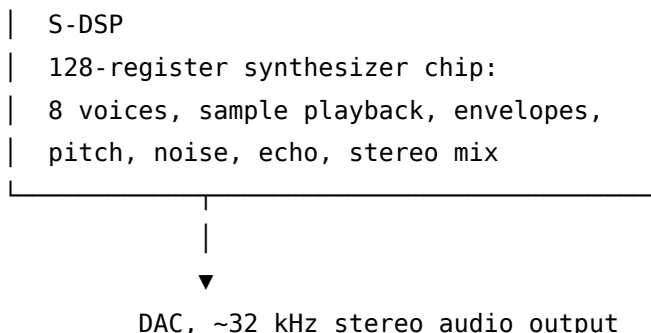
The audio subsystem is, internally, a separate small computer. Its CPU is the **SPC-700**, running at 1.024 MHz. Sitting next to the SPC-700 is a sound

generator chip called the **S-DSP** (Digital Signal Processor), which actually produces the audio waveform. Connected to both is **64 KiB of RAM**, called **ARAM** (Audio RAM), which holds the SPC's program, its data, and the audio samples it plays.

The audio subsystem is physically separate from the main CPU. The two CPUs do not share memory: the main CPU cannot directly read or write ARAM, and the SPC-700 cannot directly read or write the cartridge or the main CPU's RAM. They communicate only through a four-byte mailbox we'll cover in Chapter 16, Inter-CPU Communication.

A picture:





4.2 Why two CPUs?

The reason for the split is partly engineering and partly historical.

The engineering reason: mixing eight audio voices in real time at acceptable quality, while also running a game, was beyond what an early-1990s 16-bit CPU could comfortably do alone. Dedication a coprocessor to audio meant Nintendo could ship richer sound without sacrificing game performance. The main CPU offloads “play this music, play that sound effect” to the audio subsystem and then forgets about it; the audio subsystem handles the moment-to-moment work.

The historical reason: the audio subsystem was designed not by Nintendo but by Sony, under contract. Secondary sources widely identify Sony engineer Ken Kutaragi as the lead. (This narrative appears in many places but is historical context, not hardware documentation.) The relevant fact for us is that the audio subsystem was designed as a self-contained module, by a different company than the rest of the console, and was deliberately walled off from it.

The deliberate wall-off is why our subject of study is so cleanly bounded. The SPC-700 has its own memory, its own clock, its own program. Once it’s running, the rest of the SNES might as well not exist as far as the SPC is concerned.

4.3 What “having” 64 KiB of RAM means

The SPC-700 can address $2^{16} = 65,536$ distinct bytes of memory. We’ll use the abbreviation **64 KiB** for this — that’s “kibibyte,” 1024 bytes per K. (You will sometimes see “64 KB” used loosely to mean the same thing. In this book, KiB is precisely $1024 \times 64 = 65,536$.)

This is a hard ceiling. Everything the audio subsystem does has to fit inside it:

- The SPC’s program (the **sound driver**) — typically 4–8 KiB.
- The audio samples — usually 20–50 KiB.
- Sequence data (the music itself, encoded compactly) — a few KiB per song.
- Variables and stack — under 1 KiB.
- An **echo buffer** for the DSP’s reverb effect — anywhere from 0 to 30 KiB depending on echo length.

Add it up: 64 KiB is not a lot of room. We will spend real time in Part IV thinking about how to allocate it. Audio compositions for the SNES are small partly because of this constraint.

4.4 The S-DSP, in brief

The S-DSP is the audio chip that actually generates sound. It has eight independent **voices**. Each voice plays back a compressed audio sample with its own pitch, volume, panning, and amplitude envelope. The DSP mixes all eight voices, optionally applies a reverb-like **echo** effect, and outputs the result to the stereo audio jacks.

The DSP is *not* a general-purpose CPU. It’s a fixed-function synthesizer with 128 internal registers controlling its behavior. The SPC-700’s job is to write to those registers — to set up voices, change pitches, key notes on and off — at the right moments.

The DSP does not run a “program” in the way the SPC does. It just continuously synthesizes sound based on the values of its registers. Change a register and the sound changes. Don’t change anything and it keeps doing whatever it was doing.

The DSP is reachable from the SPC through two memory addresses:

- Writing to address \$F2 selects which DSP register you want to talk to next.
- Writing to address \$F3 sends a value to the selected register.
- Reading address \$F3 reads back the value of the currently selected register.

So talking to the DSP is a two-step dance: write the register number to \$F2, then write or read the value through \$F3. We will be doing this constantly in Part III. The address pair \$F2/\$F3 is a memory-mapped I/O gateway in exactly the sense Chapter 1 described.

4.5 The mailbox to the main CPU

The SPC has no way to read the cartridge or the main RAM. It cannot ask the main CPU for instructions in any structured way. Their only communication is through four shared bytes:

- From the main CPU’s point of view, these are at addresses \$2140, \$2141, \$2142, \$2143.
- From the SPC’s point of view, the same four bytes appear at addresses \$F4, \$F5, \$F6, \$F7.

Either side can read or write any of the four ports. There is no signal, no interrupt, no FIFO — just four bytes of shared state. The two CPUs invent their own conventions for using them.

Two important uses of this mailbox:

At boot. When the SNES is first powered on, the SPC runs a tiny program from its boot ROM that uses the four ports to receive its main program from

the main CPU. The main CPU sends the SPC's code, byte by byte, over this 4-byte channel. We'll see the protocol in detail in Chapter 16.

At runtime. Once the SPC is running its sound driver, the main CPU uses the ports to send commands — “play song 3,” “play sound effect 12,” “stop the music.” The SPC polls the ports, sees a new command, and responds.

Four bytes is a very small communication channel. SNES games run a remarkable amount of audio scheduling through it.

4.6 The clock asymmetry

The SPC runs at 1.024 MHz — about a million instruction cycles per second. The main CPU runs at up to 3.58 MHz, with the exact effective speed depending on which memory region it's accessing. The two CPUs are *not* synchronized: the audio subsystem has its own 24.576 MHz resonator on the APU board, from which the SPC's 1.024 MHz instruction clock and the DSP's ~32 kHz sample rate are derived. The main CPU has its own clock source. The two clocks drift relative to each other in normal operation, which is part of why all communication between the chips goes through the four-byte mailbox (or, at boot, the IPL handshake) rather than any kind of cycle-precise signaling.

For us, this means three things:

1. The SPC is *slow*, by main-CPU standards. Anything you do on the SPC has to fit in the cycle budget of one millionth of a second. This is plenty for music sequencing but not for general-purpose computation.
2. The two CPUs drift relative to each other. You cannot reliably use the SPC to time things at video-frame precision; the main CPU has to handle that, and it tells the SPC when something needs to happen.
3. The DSP, downstream of the SPC, generates audio at roughly 32 kHz — meaning it produces a stereo audio sample every 32,000th of a second. This is why audio samples are “32 kHz native” — that's the rate the DSP outputs. (The exact rate is closer to 32,000–32,160 Hz depend-

ing on console revision, which matters for studio synchronization but not for normal music.)

4.7 What you'll need to do

Here is, at a high level, what programming the SPC-700 for music looks like:

1. Write a program (the **sound driver**) that runs on the SPC and:
 - Sets up the DSP at boot.
 - Polls the four mailbox ports for commands from the main CPU.
 - Walks through encoded music data, deciding which notes should play when.
 - Writes to the DSP registers to start, modify, and stop notes.
2. Pack the driver, the music data, and the audio samples into a layout that fits in 64 KiB.
3. Have the main CPU upload all of that to ARAM at boot.
4. Have the main CPU send “play song” commands to the SPC at appropriate moments during the game.

The next ten chapters cover how to do all of this. Part II is about writing programs for the SPC. Part III is about controlling the DSP. Part IV is about sound drivers, music, and tools.

4.8 What you should remember

- The SNES has two CPUs: a main CPU running the game, and the SPC-700 running audio.
- The audio subsystem is self-contained: SPC-700 + 64 KiB ARAM + S-DSP.
- The SPC-700 cannot directly access main-CPU memory; the main CPU cannot directly access ARAM.
- The two CPUs communicate through four shared mailbox bytes.

- The S-DSP is reachable from the SPC through addresses \$F2 (register select) and \$F3 (register data).
- The S-DSP has 8 voices, plays compressed samples, and applies echo.
- The SPC runs at 1.024 MHz; the DSP outputs at ~32 kHz.

4.9 Exercises

1. The main CPU runs at up to 3.58 MHz. The SPC runs at 1.024 MHz. Roughly what is the ratio? What does this imply about how much “audio code” the SPC can run between two updates of the main CPU’s screen, if the screen updates at 60 Hz?
 2. The DSP outputs roughly 32,000 stereo sample pairs per second. The SPC runs roughly 1,024,000 cycles per second. How many SPC cycles are there per DSP sample? Is this a lot or a little?
 3. Suppose the main CPU wants to send the SPC a single command: “play song 5.” It only has four mailbox bytes. Sketch a protocol — what does the main CPU write to which ports, in what order, to convey this?
 4. Of the 64 KiB of ARAM, suppose your sound driver code is 6 KiB, your samples are 30 KiB, and your echo buffer is 8 KiB. How much is left for everything else?
 5. The SPC-700 cannot read the cartridge directly. What does this mean for a SNES game with hours of unique music — how does all that music get to the SPC?
-

Part II

Programming the SPC-700

We now have the picture. Time to start writing code.

Part II walks through the SPC-700's specific registers, its memory map, and its instruction set, in roughly the order you need them to write a working program. By the end you will be able to read any SPC-700 disassembly and know, line by line, what the chip is being told to do.

There are seven chapters. Chapter 5 is a short interlude that walks you through the tooling — Mesen2, Asar, and the companion repository — so that the rest of Part II can show you real SPC-700 behavior rather than only describe it. Chapter 6 covers the SPC-700's specific registers and memory layout (filling in the abstract picture of Chapter 2 with concrete details). Chapter 7 covers how code gets into the chip in the first place — boot and upload. Chapters 8 through 11 cover the instruction set, organized by what the instructions do: moving data, arithmetic, logic, control flow.

Read these in order. Each chapter assumes the previous ones.

Chapter 5

Interlude — Setting Up

You have spent four chapters reading. We are about to spend the next fifteen reading and *doing*. This is a short interlude that sets up the doing.

By the end of this chapter you will have:

- A SNES emulator with a working SPC debugger installed.
- An SPC-700 assembler installed.
- The book’s companion repository on your machine.
- Built and run a tiny SPC program of your own.
- Watched a single byte change in front of you, in the emulator, as a result of an instruction you wrote.

That last one is the point. Until now, the SPC-700 has been an idea. By the end of this chapter, it will be a thing you have caused to do something.

This chapter has no exercises at the end because the chapter *is* the exercise.

5.1 What you’ll install

Three things, all free.

Mesen2 is a SNES emulator with a thorough debugger. It will let you load a SNES ROM, watch the SPC-700 execute one instruction at a time, inspect every byte of audio RAM, and see the audio chip’s internal state. Mesen2 is the recommended emulator for this book and for the SNES homebrew scene generally; you can get it at <https://www.mesen.ca>.

Asar is an SPC-700 assembler — the program that takes assembly language source files and turns them into the byte sequences a CPU actually executes. It also handles the 65816 (the SNES main CPU) side, which we will not be writing ourselves but which the companion repository’s “stub ROM” uses behind the scenes. You can get Asar at <https://github.com/RPGHacker/asar/releases>.

The **companion repository** holds the small SPC-700 source files, build scripts, BRR audio samples, and verification notes that go with this book. Clone it from <https://github.com/spencer2718/spc700-book> with a Git client.

The repository’s `assets/docs/INSTALL.md` has detailed platform-specific instructions for Mesen2 and Asar (Windows, Linux, macOS). Follow those for the install. The rest of this chapter assumes you have all three.

5.2 What the stub ROM is, and why you don’t have to write 65816

The SPC-700 lives inside the SNES. To run an SPC-700 program on a real SNES — or on Mesen2, which faithfully emulates one — you have to give the SNES a ROM cartridge. The SNES’s main CPU boots, reads the cartridge, and at some point uses the four-byte mailbox we’ll meet in Chapter 16 to upload your SPC-700 code into ARAM. Only *then* does the SPC-700 start running it.

The companion repository contains a small SNES ROM whose only job is to perform that upload and then idle. It’s called the **stub ROM**, and it’s a few hundred lines of 65816 assembly. The stub ROM is a *cradle*: it boots, hands your SPC-700 code to the SPC-700, and gets out of the way.

You will not be writing or modifying 65816 code in this book. The stub ROM exists so that you can write SPC-700 code, hand it to the stub, and have a real SNES ROM (a `.sfc` file) you can load in Mesen2. From your point of view as a reader, the stub is invisible machinery.

5.3 Your first SPC-700 program

The companion repository ships with the source we will use as your first program. Open `assets/exercises/ch05_setup/start/hello.asm` in a text editor. You should see something like this:

```
arch spc700
norom

org $0000
base $0200

start:
    mov    x, #$ff        ; X = $FF
    mov    sp, x          ; SP = $FF
    clrp                   ; PSW.P = 0 (direct page = $0000-$00FF)

    mov    a, #$42        ; A = $42
    ; TODO: write A to ARAM address $0500

forever:
    bra    forever
```

The lines starting with `mov` and `clrp` and `bra` are SPC-700 assembly instructions. We have not formally introduced any of them yet — the next chapter does that — but they are simple enough that you can read what they intend.

There is one missing line, marked `TODO`. Your job, before going further, is to write that line.

What we want is the byte `$42` (which the previous instruction has placed in register `A`) to be written to ARAM address `$0500`. We need an SPC-700 instruction with the form

```
mov    <destination>, <source>
```

where `<destination>` is an absolute memory address and `<source>` is the A register. The destination address is `$0500`, and the source register is `a`. Stop reading and try to write the line.

(If you are completely new to assembly, the format is `mov $0500, a` — destination first, source second. SPC-700 follows the “destination, source” convention.)

Replace the `T0D0` line with the instruction. Save the file.

5.4 Building it

You now run two short commands in your shell.

First, from inside `assets/exercises/ch05_setup/start/`:

```
./build.sh
```

This runs Asar on `hello.asm` and produces `hello.bin`, a tiny binary file containing the assembled SPC-700 machine code.

Second, from inside `assets/stub-rom/`:

```
./build.sh ../exercises/ch05_setup/start/hello.bin
```

This runs Asar on the stub ROM source, embedding your `hello.bin` into the larger ROM. It produces `stub.sfc`, a complete SNES ROM file.

You now have a real SNES ROM, with your SPC-700 code embedded in it, ready to run.

5.5 Running it

Open Mesen2. Use **File → Open ROM** and select `stub.sfc`.

The screen will be black. This is correct. The stub ROM does not draw anything; its only job is to upload your code to the SPC-700 and idle. The action is happening on the audio CPU, which has no visual presence.

Two debugger windows are relevant. Both open from the *main* Mesen2 window's Debug menu (not from any sub-window's menu):

- **SPC Debugger** (Debug → SPC Debugger, or Ctrl+F): shows the SPC's current execution state. You'll see a panel with PC, A, X, Y, SP, and PSW; a disassembly view; and execution controls (step, run, breakpoint).
- **Memory Tools** (Debug → Memory Tools, or Ctrl+M): shows raw memory as bytes. You'll use this to inspect ARAM.

Open both. Position them side by side if you can.

In the SPC Debugger, set a breakpoint at address \$0200 — that's where your code begins. The simplest way is to right-click on the line in the disassembly view that corresponds to \$0200 (or to type 0200 into a breakpoint dialog if Mesen2 offers one). Then press the run button (or F5).

The emulator runs. The stub ROM boots, sends your SPC-700 code to the SPC, and the SPC starts executing — but the breakpoint catches it at the first instruction. The SPC Debugger now shows PC = \$0200 and your code in the disassembly.

In the Memory Tools window, set **Memory Type = RAM**. This selects the SPC's 64 KiB ARAM. Type 0500 into the address field and press Enter. The view jumps to address \$0500. The byte there is \$00 (memory is zero on a fresh boot).

Press F10 (step) once. PC advances by one byte (from \$0200 to \$0201); the `clrp` instruction has executed, and the P bit in PSW is now 0. Step again. PC advances by two more bytes (to \$0203); the `mov a, #$42` instruction has executed, and A is now \$42. Step *one more time* and look at Memory Tools.

The byte at \$0500 is now \$42.

You just caused the SPC-700 to execute an instruction that wrote to memory, and you watched the result happen.

5.6 What just happened

You wrote SPC-700 source code in a text editor. An assembler translated it to machine code. A second build wrapped it in a SNES ROM. An emulator booted that ROM, the SNES uploaded your code to the SPC-700, and the SPC-700 ran it. You watched the chip's state change one instruction at a time.

This is the loop. **Edit. Build. Run. Step.** Every chapter from here on will return to this loop. The SPC programs will get larger and more interesting. But the mechanics — write some assembly, run two build commands, load `stub.sfc` in Mesen2, look at registers and memory — are the mechanics throughout.

If something didn't work, the companion repository's `assets/docs/TROUBLESHOOTING` is the place to start. Most problems at this stage are tooling-related (a path is wrong, a shell isn't finding `asar`, a Mesen2 setting is off) rather than about the SPC-700 itself.

When you can run the loop smoothly — edit a file, see the result in Mesen2 within a minute or two — you are ready for Chapter 6, which formally introduces the SPC-700's registers and the categories of instructions you will spend Part II learning.

Chapter 6

The Programmer's Model

Chapter 2 introduced the abstract notion of registers, flags, and memory. This chapter says exactly which registers, which flags, and which memory layout the SPC-700 has. It is the foundation everything else in Part II will sit on, so it's worth reading carefully.

6.1 Registers

The SPC-700 has six programmer-visible registers. Six. That is the entire internal state of the CPU.

Name	Width	Role
A	8 bits	Accumulator. The main register for arithmetic and logic.
X	8 bits	Index register.
Y	8 bits	Index register.
YA	16 bits	Y is the high byte, A the low byte.
SP	8 bits	Stack pointer. Always points into memory page \$01.
PSW	8 bits	Processor Status Word. The flag register.
PC	16 bits	Program counter. The address of the next instruction.

Some clarifications:

A is the principal arithmetic register. Most arithmetic instructions either read from A, write to A, or both. If you want to compute “A + B” where A and B are both in memory, you’ll typically load one of them into A first.

X and Y are not interchangeable. They look symmetric on the surface — both are 8-bit index registers — but the instruction set treats them slightly differently. Some addressing modes are available with X but not Y, and vice versa. You will discover the asymmetries one at a time as we go.

YA is a 16-bit pair. When an instruction “operates on YA,” it treats Y and A together as a 16-bit value, with Y as the high byte and A as the low byte. There is no separate hardware register named YA — it’s a logical pairing of the two existing registers. Several instructions, especially 16-bit arithmetic, use this pairing.

SP is 8 bits, but the stack lives in a 16-bit address range. The stack pointer SP holds an 8-bit value, and the actual memory address for the top of the stack is computed as $\$0100 + SP$. So the stack always lives in memory addresses $\$0100$ through $\$01FF$, and SP simply indexes within that 256-byte page. When $SP = \$FF$, the top of stack is at $\$01FF$. When $SP = \$00$, the top is at $\$0100$.

PC is the only 16-bit register without a partner. It holds the full address of the next instruction. Unlike A/X/Y, you don’t write to PC directly with a MOV — you change it indirectly, by executing jumps and branches.

That’s it. Six registers, fewer than a dozen bytes of register state. The rest of the running computer’s state is in the 64 KiB of memory.

6.2 The PSW flags

The PSW byte holds eight flags, one per bit. The convention used by the SNESdev wiki — and the one we’ll use throughout this book — is to write them in the order **N V P B H I Z C** (bit 7 down to bit 0).

Bit	Flag	Set when...
7	N	The most recent result was negative (bit 7 of the result was 1).
6	V	A signed arithmetic operation overflowed.
5	P	The direct page is at \$0100-\$01FF rather than \$0000-\$00FF.
4	B	A BRK instruction was executed.
3	H	Half-carry; nibble carried into bit 4. Used by BCD adjustments.
2	I	Interrupts enabled. (Unused on the SNES — there are no working interrupt sources.)
1	Z	The most recent result was zero.
0	C	Carry. The unsigned-overflow / borrow / shifted-out bit.

Three of these flags are doing most of the work in everyday code: **C**, **Z**, **N**. The others come up but rarely.

C, the carry flag, is essential for any arithmetic wider than a byte. The SPC-700, like most accumulator machines, does not have a “no-carry add” instruction. `ADC A, #5` always adds 5 *plus the current value of C*. If you forget to clear C with `CLRC` before starting a multi-byte addition, you will silently get answers that are off by one. This is the single most common bug in beginner SPC-700 code.

Z, the zero flag, is set automatically when most arithmetic, logical, and load instructions produce a zero result. The conditional branches `BEQ` (“branch if equal”) and `BNE` (“branch if not equal”) test it.

N, the negative flag, is set when bit 7 of the most recent result was 1. In two’s-complement terms, this is “the result is negative.”

P, the direct page flag, is unusual enough that it gets its own section below.

The other four flags — V, B, H, I — exist but won't drive most of your programming. We'll meet them as needed.

6.3 The memory map

Here is the entire 64 KiB address space the SPC-700 sees:

Range	Contents
\$0000-\$00EF	General RAM. Direct page when P = 0. Holds variables.
\$00F0-\$00FF	I/O registers — memory-mapped gateways, not real RAM.
\$0100-\$01FF	Stack page. Also direct page when P = 1.
\$0200-\$FFBF	General RAM. Driver code, samples, sequence data, etc.
\$FFC0-\$FFFF	IPL boot ROM when enabled. Underlying RAM still writable.

A few things to internalize.

The I/O registers are at \$00F0-\$00FF. This is a small block of sixteen addresses where memory-mapped I/O lives. We'll list them in a moment. They are *not* normal memory: reading or writing them does things to hardware. Don't put variables there.

The stack is in page \$01, not page \$00. This is a fixed feature of the chip — the stack pointer SP is 8 bits, and the stack page is wired to \$01. Note that this is also where the direct page ends up if P = 1, which is one reason setting P = 1 is unusual.

The IPL ROM is at the top. When the SPC is first powered on, the top 64 bytes of address space (\$FFC0-\$FFFF) read out the contents of a small built-in ROM that holds the boot loader. The underlying 64 bytes of RAM are still there, but reads see the ROM. Writes go to RAM. Most drivers, after

boot, hide the ROM (by clearing a bit in the CONTROL register at \$F1) so they can use those 64 bytes of RAM.

6.4 The I/O registers

These hardware-facing addresses at \$F0-\$FF are the SPC's window onto the rest of the world. (The "register" terminology here is overloaded — these are not CPU registers, they're memory-mapped hardware control bytes. Both senses of the word "register" are standard usage and the meaning is always clear from context.)

Address	Name	Purpose
\$F0	TEST	Hardware test register. Don't touch.
\$F1	CONTROL	IPL ROM enable, port reset, timer enable.
\$F2	DSPADDR	Selects which S-DSP register \$F3 reads or writes.
\$F3	DSPDATA	Reads or writes the selected S-DSP register.
\$F4	CPUIO0	Mailbox port 0 to/from the main CPU.
\$F5	CPUIO1	Mailbox port 1.
\$F6	CPUIO2	Mailbox port 2.
\$F7	CPUIO3	Mailbox port 3.
\$F8	AUXIO4	General-purpose byte. Read/write RAM, no hardware effect.
\$F9	AUXIO5	General-purpose byte. Read/write RAM, no hardware effect.
\$FA	T0TARGET	Timer 0 target value.
\$FB	T1TARGET	Timer 1 target value.
\$FC	T2TARGET	Timer 2 target value.
\$FD	T0OUT	Timer 0 output. Read clears it.
\$FE	T1OUT	Timer 1 output. Read clears it.
\$FF	T2OUT	Timer 2 output. Read clears it.

Most of Part III will be about \$F2 and \$F3 (the DSP gateway). Most of Part IV will be about \$F4-\$F7 (the mailbox). The timers come up in Chapter 17, *Anatomy of a Sound Driver*, when we talk about how a sound driver paces itself. \$F8 and \$F9 are general-purpose RAM bytes that just happen to live in the I/O page; some drivers use them as scratch storage, but for beginner code there's no reason to single them out.

A few notes:

\$F1 (CONTROL) is write-only. You cannot read it. Any read returns \$00. So you cannot do “read CONTROL, modify a bit, write it back” — you have to write the full byte you want every time. We'll come back to this in Chapter 17.

Writing to \$F1 does things to timers. A 0-to-1 transition on a timer-enable bit resets the timer's internal counter. So writes to \$F1 are not consequence-free; they perturb timer state. Plan your writes deliberately.

\$F2 and \$F3 are the DSP gateway. Writing a register number *r* to \$F2, then writing a value *v* to \$F3, is how you tell the DSP “set register *r* to value *v*.” Reading \$F3 after writing the address to \$F2 reads back the current value of the selected DSP register. We'll cover this in Chapter 12.

\$F4-\$F7 are the mailbox to the main CPU. Either side can read or write these. From the main CPU, the same four bytes are visible at \$2140-\$2143. We'll cover this in Chapter 16.

6.5 Direct page, properly

In Chapter 3 we mentioned “direct” addressing as a short form where a single operand byte specifies a memory address by combining with an implicit high byte. On the SPC-700, that high byte is normally \$00, so direct addressing reaches memory in the range \$0000-\$00FF. This region is called the **direct page** (other CPUs call it “zero page”).

The benefit of direct-page addressing: the instruction is two bytes (opcode + low byte) instead of three (opcode + low byte + high byte), and it exe-

cutes in fewer cycles. Putting your hot variables in direct page makes everything that touches them faster and smaller.

The SPC-700 has an unusual feature: it can move the direct page from \$0000-\$00FF to \$0100-\$01FF. This is what the **P flag** in PSW controls. With $P = 0$ (the default), direct addressing means addresses \$0000 through \$00FF. With $P = 1$, direct addressing means addresses \$0100 through \$01FF.

You can set P with the instruction SETP, and clear it with CLR P .

In practice, almost all sound drivers leave $P = 0$ forever. The reason: page \$01 is where the stack lives. With $P = 1$, your direct-page variables are in the same memory as the stack, which is a recipe for trouble — push something on the stack, it might overwrite a variable. The flexibility exists because Sony was being thorough, not because you're expected to use it.

This means: when you see the assembly notation

```
mov a, $30
```

...with no **!** prefix and no **#** prefix, you should read it as “load A from direct-page address \$30,” which (since P is almost always 0) means “load A from address \$0030.”

6.6 Putting it together

The full state of the running SPC-700 is, mathematically:

$$S = (A, X, Y, SP, PSW, PC, M)$$

where A, X, Y, SP, PSW are 8-bit values; PC is 16-bit; and M is the memory function on \$0000 through \$FFFF. Of those 64 KiB of memory, a small region at \$00F0-\$00FF is memory-mapped I/O rather than RAM, and 64 bytes at the top are normally the boot ROM.

When an instruction executes, it modifies some of these state components. By Part II's end, you'll have a complete picture of what every instruction does to this state.

6.7 What you should remember

- Six registers: A, X, Y, SP, PSW, PC. A and Y can pair as YA for 16-bit operations.
- Eight flags in PSW. The most-used: C, Z, N. Note the order: **N V P B H I Z C** (bit 7 to bit 0).
- The stack is in page \$01. SP indexes within it.
- I/O lives at \$00F0-\$00FF. The DSP gateway is \$F2/\$F3. The mailbox is \$F4-\$F7.
- The direct page is normally \$0000-\$00FF. Putting variables there is fast.
- The IPL ROM occupies \$FFC0-\$FFFF until you hide it.

6.8 Exercises

1. After an arithmetic instruction produces the result \$00, which flags are guaranteed to be set or cleared?
2. The byte \$80 is the most negative signed byte. After a load of \$80 into A, what is the state of the N and Z flags?
3. The variable `tempo` lives at direct-page address \$20. Write the SPC-700 instruction to load A from `tempo`. Write the equivalent instruction using absolute addressing.
4. Suppose SP = \$F0. Where in memory is the next byte that will be pushed? After the push, what's the new SP?
5. Why is it dangerous to set P = 1?

6.9 Try this

The companion repository's `assets/exercises/ch06_programmers_model/` is a hands-on exercise that puts every concept from this chapter under your microscope. There's no code to write; the exercise ships with a

complete ~25-byte payload that exercises the stack pointer, `clrp`, A/X/Y, direct vs. absolute writes, and PUSH/POP in sequence. Your job is to single-step through it in Mesen2's SPC Debugger, with Memory Tools open alongside, and watch each promised state change happen.

The exercise's `README.md` has a step-by-step walkthrough. The emphasis is *observation*: by the end you should have seen, empirically, that the abstract framework from this chapter (state = A, X, Y, SP, PSW, PC, M) corresponds to real bytes you can read in the debugger.

After working through it, the README's "Try this" section asks you to predict what would happen if the stack pointer setup were omitted, then verify your prediction. That's an open exploration worth fifteen minutes — it tests whether you've internalized that "the stack lives at $\$0100 + SP$ " is a *convention* the chip doesn't enforce, not a hardware rule.

Time investment: 30-45 minutes if it's your first hands-on chapter after the Interlude; less once you're fluent with the debugger.

Chapter 7

Boot and Code Loading

How does code get into the SPC-700 in the first place?

This is a real question, not a rhetorical one. The SPC has 64 KiB of RAM but, at power-on, that RAM is uninitialized — it holds whatever happened to be in the cells when the chip woke up. The SPC has no disk, no cartridge port, no way to load code from anywhere on its own. The only inputs it has are the four mailbox bytes shared with the main CPU.

So somehow, the SPC has to get its program through those four bytes, one byte at a time, from the main CPU. This chapter explains how.

7.1 The IPL ROM

The SPC has a small built-in **IPL ROM** (“Initial Program Loader”). It’s exactly 64 bytes long and sits at memory addresses \$FFC0-\$FFFF. It is permanently part of the chip — burned in at the factory, not loaded from anywhere.

When the SPC powers on, its program counter is set to \$FFC0, where the IPL ROM begins. The CPU starts executing the IPL ROM.

The IPL ROM’s job is to cooperate with the main CPU to load a program into ARAM and then run it. Here is what it does, in plain English:

1. Set up the stack pointer.
2. Clear out any stale state in I/O ports.

3. Write the magic byte \$AA to mailbox port 0.
4. Write the magic byte \$BB to mailbox port 1.
5. Wait for the main CPU to write \$CC to mailbox port 0. (Until then, do nothing.)
6. Receive a target address through ports 2 and 3.
7. Receive bytes through port 1, with a counter through port 0, and write them to ARAM at the target address (advancing the address each byte).
8. When the main CPU signals “stop transferring,” receive a final entry-point address through ports 2 and 3, and jump to it.

Step 3 and 4, “write \$AA and \$BB to ports 0 and 1,” is the key: it tells the main CPU *the SPC is alive and ready to receive*. The main CPU spins reading those two ports until it sees \$AA and \$BB come back. Once it does, it knows the audio subsystem is up.

Step 5, “wait for \$CC,” is the inverse: the main CPU writes \$CC to port 0 to say “I’m ready to start sending.” The SPC sees this and starts the upload protocol.

This handshake is universal — every commercial SNES game, every homebrew, every modern driver, all use it. The IPL ROM implements the SPC’s side; the main CPU implements its side as part of its own boot code.

7.2 The upload protocol

We won’t dwell on every byte of the protocol — you can find it in detail on the SNESdev wiki — but the shape is worth understanding.

After the \$AA \$BB ↔ \$CC handshake, both sides are synchronized. The main CPU then sends the destination address (where in ARAM to start writing) through ports 2 and 3, and begins streaming bytes:

For each byte to upload:

Main CPU writes the byte to port 1.

Main CPU writes a counter (incremented from previous) to port 0.

SPC sees port 0 change, reads port 1, stores it in ARAM,
advances the destination address, and echoes the counter back to port 0.
Main CPU sees its counter come back, knows the byte landed,
increments the counter, sends the next byte.

The counter is the synchronization mechanism. It lets the main CPU know exactly when each byte has been received. Without it, the main CPU would have to guess at timing — and the SPC, running at 1 MHz, is slow enough that bad guesses would lose bytes.

When the main CPU is done uploading, it sends a final command saying “stop transferring; jump to entry-point address X.” The SPC reads X from ports 2 and 3 and jumps there. From this moment on, the SPC is running the uploaded code, not the IPL ROM.

7.3 How long does upload take?

A complete upload of, say, a 30 KiB image (driver code plus samples plus sequence data — most of what gets transferred at boot is samples, not driver code) takes roughly 0.78 seconds. SNESdev estimates the protocol at about 520 master clocks per byte; at the main CPU’s effective rate, that works out to about 650 bytes per 60 Hz video frame. A 30 KiB upload is therefore about 47 frames, just under a second.

This is fast enough to do during a logo screen or a fade-in, but it is *not* instantaneous. SNES games typically perform the upload during a scene transition where the player is already looking at a static image, so the delay is masked.

Some games stream samples in chunks instead of one giant upload, to keep the UI more responsive while music data loads. Others upload a small “core” driver at boot and stream music data later. The protocol supports both styles, since the main CPU can keep sending bytes whenever the SPC is in the middle of its upload loop.

7.4 After the upload: hiding the IPL ROM

Once the main CPU has uploaded the driver and signaled “jump,” the SPC executes from the entry-point address. It is now running *your* code. The IPL ROM is still mapped at \$FFC0-\$FFFF, but you don’t need it anymore.

Most drivers, as one of their first acts, hide the IPL ROM. You do this by clearing bit 7 of the CONTROL register at \$F1. With bit 7 clear, reads to \$FFC0-\$FFFF see the underlying RAM rather than the IPL ROM. This frees up 64 bytes of RAM at the top of memory — a meaningful amount on a 64 KiB chip.

A typical first-instruction sequence looks like:

start:

```

mov    x, #$ff                ; set up stack pointer
mov    sp, x

mov    $f1, #$30              ; CONTROL: reset all four mailbox ports,
                                ;             hide IPL ROM, no timers enabled

; ... initialize DSP, install sample directory ...
; ... main loop ...
```

The exact value written to \$F1 depends on what the driver wants to do with the IPL ROM and the timers. We’ll see specifics in Chapter 17, Anatomy of a Sound Driver.

7.5 Where your code lives

By convention, almost every SPC driver is uploaded starting at address \$0200. This is just above the I/O registers (\$00F0-\$00FF) and the stack page (\$0100-\$01FF), and below the rest of RAM. The choice is conventional, not forced — the IPL ROM happily uploads to any address you give it — but \$0200 is what everyone uses, so you should too unless you have a specific reason.

The full memory layout of a typical running driver might look like:

\$0000-\$00EF	Driver variables (direct page)
\$00F0-\$00FF	I/O registers
\$0100-\$01FF	Stack
\$0200-???	Driver code (executable instructions)
???-???	Sample directory (pointers to BRR samples)
???-???	BRR sample data
???-\$FFFF	Echo buffer (allocated from the top, often)

The exact boundaries are decisions the driver author makes. Some drivers pack samples right after code; some leave a fixed gap; some compute layouts at runtime. We'll look at real layouts in Chapter 17.

7.6 Resident drivers vs. one-shot uploads

There are two flavors of SPC code you might write.

A **resident driver** is uploaded once at console power-on and stays in ARAM forever. It has a main loop, polls the mailbox, accepts commands like “play song 5” or “stop SFX channel 3,” and orchestrates audio continuously. This is what every commercial SNES game does.

A **one-shot SPC** — for example, an SPC file you're sharing with friends — is a snapshot of ARAM after a song has already been loaded and started. It captures the driver, its data, and the moment-in-time state. We'll cover SPC files in Chapter 19.

For learning, you'll usually be writing resident drivers — small ones, but resident — and snapshotting them as SPC files for sharing.

7.7 What you should remember

- The SPC has a 64-byte built-in IPL ROM at \$FFC0-\$FFFF that runs at power-on.
- The IPL ROM cooperates with the main CPU to upload your code through the four mailbox ports.

- The boot protocol is a handshake (\$AA \$BB ↔ \$CC) followed by a byte-by-byte stream with a counter.
- A 30 KiB upload takes roughly 0.78 seconds.
- Drivers conventionally start at address \$0200.
- After boot, drivers usually hide the IPL ROM by clearing bit 7 of \$F1, freeing 64 bytes.

7.8 Exercises

1. The IPL ROM is 64 bytes. SPC-700 instructions are 1–3 bytes long. Roughly how many instructions can fit in 64 bytes?
 2. Suppose the main CPU wants to upload 20 KiB of driver and samples. About how long does this take? Express the answer in 60 Hz video frames.
 3. After the upload finishes and the SPC starts running your code, the IPL ROM is still mapped. What happens if your code accidentally jumps to \$FFC0?
 4. Sketch the order of operations the main CPU has to perform to (a) wait for the SPC to be ready, (b) upload 100 bytes starting at address \$0200, (c) tell the SPC to begin executing at \$0200.
 5. Why does the upload protocol use a counter rather than a fixed-rate stream?
-

Chapter 8

Moving Data

If you remember one thing from this chapter, remember this: **the SPC-700 does almost nothing without first moving a byte.** Every arithmetic operation has to have its inputs in the right place. Every DSP write has to put the right value in A and the right register number in Y. Every table lookup has to set up X first.

So we begin with MOV, the data-movement instruction.

8.1 MOV is the core verb

MOV is the SPC-700's "load" and "store" combined. It moves a byte from one place to another. The destination is always written first, the source second:

MOV destination, source

Some of the most common forms:

```

mov  a, #$42           ; A = $42 (immediate)
mov  a, $30            ; A = byte at direct page address $30
mov  a, !source        ; A = byte at absolute address (label "source")

mov  $30, a            ; byte at $30 = A
mov  $30, #$42         ; byte at $30 = $42

mov  a, x              ; A = X (register-to-register)
mov  x, a              ; X = A

```



```

mov  x,  sp          ; X = SP
mov  sp, x           ; SP = X

```

The # prefix means “immediate” — the value following is the literal number, not an address. The ! prefix marks an absolute (16-bit) address. Without either prefix, the operand is a direct-page address (a single byte that combines with an implicit high byte of \$00, since we leave P = 0).

Cost matters. Direct-page moves take 3 cycles; absolute moves take 4. Multiply by a few thousand instructions per video frame and the difference adds up.

8.2 The full move family

Here are all the MOV variants, with their cycle counts and which flags they affect.

Instruction	Cycles	Flags affected
MOV A,#imm	2	N,Z
MOV A,dp	3	N,Z
MOV A,dp+X	4	N,Z
MOV A,!abs	4	N,Z
MOV A,!abs+X	5	N,Z
MOV A,!abs+Y	5	N,Z
MOV A,(X)	3	N,Z
MOV A,(X)+	4	N,Z
MOV A,[dp+X]	6	N,Z
MOV A,[dp]+Y	6	N,Z
MOV X,#imm	2	N,Z
MOV X,dp	3	N,Z
MOV X,dp+Y	4	N,Z
MOV X,!abs	4	N,Z

Instruction	Cycles	Flags affected
MOV Y, #imm	2	N, Z
MOV Y, dp	3	N, Z
MOV Y, dp+X	4	N, Z
MOV Y, !abs	4	N, Z
MOV dp, A	4	none
MOV !abs, A	5	none
MOV (X), A	4	none
MOV (X)+, A	4	none
MOV dp, #imm	5	none
MOV dp, dp	5	none
MOV A, X / A, Y	2	N, Z
MOV X, A / Y, A	2	N, Z
MOV X, SP	2	N, Z
MOV SP, X	2	none

Two patterns to internalize.

First, loading into a register sets N and Z; storing from a register doesn't. This means `MOV A, $30` followed immediately by `BEQ` (“branch if equal to zero”) tests whether the byte at \$30 was zero. You don't need a separate compare — the load already set Z.

Second, the indirect modes are powerful but expensive. `MOV A, [dp]+Y` — meaning “treat the direct-page bytes at dp and dp+1 as a 16-bit pointer; add Y to it; load A from that address” — costs 6 cycles. Three times the cost of an immediate load. Use them when you need them, but don't reach for them by default.

8.3 The addressing modes, named

Each addressing mode appears in the instruction set under a consistent notation. Here it is one more time, all in one place:

Notation	Meaning
#imm	The literal value following the opcode.
dp	The byte at direct-page address dp.
dp+X	The byte at direct-page address dp + X.
dp+Y	The byte at direct-page address dp + Y. (Limited to a few instructions.)
!abs	The byte at 16-bit absolute address.
!abs+X	The byte at abs + X.
!abs+Y	The byte at abs + Y.
(X)	The byte at the address held in X. (Direct-page, so really dp + X.)
(X)+	Same as (X), but X is incremented after the access.
[dp+X]	Treat dp+X and dp+X+1 as a 16-bit pointer. Use that as the address.
[dp]+Y	Treat dp and dp+1 as a 16-bit pointer. Add Y. Use that as the address.

The two indirect forms — [dp+X] and [dp]+Y — are easy to confuse. [dp+X] adds X to the *direct-page address*, then dereferences (so X is selecting which pointer in a table of pointers to use). [dp]+Y dereferences first, then adds Y to the *resulting address* (so Y is an offset into whatever the pointer points to).

The [dp]+Y form is the workhorse for traversing data. Imagine the bytes at \$30/\$31 form a pointer to “the start of the current row of song data,” and Y is a byte index into that row. Then MOV A, [\$30]+Y reads the next byte of song data with one instruction.

8.4 Moving 16-bit values: MOVW

MOVW (“move word”) moves a 16-bit value between YA (the Y/A register pair) and a direct-page word. The word is stored little-endian: the byte at `dp` is the low byte (going into A), and the byte at `dp+1` is the high byte (going into Y).

```
movw ya, $20      ; Y = byte at $21, A = byte at $20
movw $20, ya      ; byte at $20 = A, byte at $21 = Y
```

This is one of the most useful instructions on the chip. We will see it constantly when we get to DSP writes (where we want to write a register address and a value in one shot) and in 16-bit math.

MOVW YA, `dp` sets N and Z based on the full 16-bit value. MOVW `dp`, YA does not affect flags.

8.5 The stack: PUSH and POP

Pushing and popping work as you’d expect from Chapter 2’s discussion.

```
push a           ; *(SP--) = A
push x
push y
push psw
```

```
pop a            ; A = *(++SP)
pop x
pop y
pop psw
```

There is a wrinkle. **POP A, POP X, and POP Y do not update flags.** This is different from MOV A, `$30`, which does. If you POP A and then expect BEQ to branch on whether the popped value was zero, you will be disappointed. Insert an explicit CMP A, `#0` (or equivalent) if you need to test.

POP PSW is special: it loads the flag register itself, which is exactly what you want when restoring state at the end of a routine.

8.6 XCN: a small but useful instruction

XCN A exchanges the high and low nibbles (4-bit halves) of A. So if A is \$3C, after XCN A it's \$C3.

```
xcn a          ; swap nibbles of A
```

This is occasionally useful for packing two 4-bit fields into a byte, or for accessing the high nibble of a value without doing four shifts.

8.7 Worked example: copying a block of memory

Here's a small routine that copies up to 256 bytes from a source to a destination:

```
; Inputs:
; $30/$31 = source address (16-bit, little-endian)
; $32/$33 = destination address (16-bit, little-endian)
; Y       = byte count (1..255; 0 means "do nothing")
; Clobbers: A, Y

memcpy:
    cmp     y, #0
    beq     .done
.loop:
    dec     y
    mov     a, [$30]+y      ; read byte from source[Y]
    mov     [$32]+y, a      ; write byte to dest[Y]
    cmp     y, #0           ; restore Z based on Y (the load above clobbers Z)
    bne     .loop          ; continue until we've just processed offset 0
.done:
    ret
```

Read this carefully — it's a small program but it pulls together several ideas.

The source pointer is in two consecutive direct-page bytes at \$30/\$31. [\$30]+y means “form a 16-bit pointer from \$30 and \$31, add Y, and use

that as the address.” We decrement Y first, then read and write at that offset.

The `cmp y, #0` before the branch is necessary, and a common surprise. After `mov a, [$30]+y`, the Z flag reflects whether the *byte loaded* was zero — not whether Y is zero. Without an explicit compare, the loop would terminate early on any zero byte in the source. This kind of flag bookkeeping is a recurring annoyance on the SPC-700; some patterns (DBNZ, CBNE) sidestep it, but for `[dp]+Y` traversal you usually have to be explicit. When Y has just been decremented from 1 down to 0, the CMP sets $Z = 1$, the BNE doesn’t branch, and we exit. The loop body executes exactly once per byte for offsets count - 1 down to 0.

This is a representative SPC-700 inner loop: small, dense, using one direct-page pointer with an index register, and built around a self-decrementing counter. The same shape appears constantly throughout audio drivers.

8.8 What you should remember

- MOV destination, source moves a byte. The destination comes first.
- Loading into a register sets N and Z. Storing does not.
- MOVW moves a 16-bit value between YA and a direct-page word.
- `[dp]+Y` is the workhorse indirect mode for traversing data.
- POP A/X/Y does not update flags. Compare explicitly if you need to branch on the popped value.

8.9 Exercises

1. Write a single instruction to copy the byte at direct-page address \$10 to direct-page address \$20.
2. The pointer `song_ptr` is a 16-bit value at \$40/\$41. The current row offset is in Y. Write a single instruction that loads A with the byte at `*song_ptr + Y`.
3. Write a routine that fills 256 bytes of memory at \$1000-\$10FF with the value \$AA.

4. After `MOV A, #$80`, what is the state of N, Z, and C?
 5. After `POP A` where the popped byte is `$00`, will `BEQ` branch? Why or why not?
-

Chapter 9

Arithmetic

The SPC-700 can add, subtract, compare, increment, decrement, multiply, and divide. It has special instructions for the YA 16-bit pair. It also has decimal-adjust instructions that nobody uses for music. We'll cover what matters and skim the rest.

9.1 Add and subtract: ADC, SBC, and the carry flag

The two basic arithmetic instructions are **ADC** (“add with carry”) and **SBC** (“subtract with carry/borrow”). Both *include the carry flag* in the calculation. There is no plain ADD or SUB.

This means a single 8-bit addition almost always looks like:

```
clrc                ; clear C; otherwise it adds an extra 1 silently
adc  a, #5
```

If you forget the CLRC, your code will work or not work depending on what C happened to be from a previous instruction. This is a consistent source of bugs.

For subtraction, the convention is that **C represents the inverse of borrow**. To do a fresh subtraction, you set C first:

```
setc                ; clear borrow before subtraction
sbc  a, #5
```


ADC sets C if the result wrapped past 255 (unsigned overflow). SBC clears C if the result wrapped past 0 (a borrow occurred). Both also set N (sign), V (signed overflow), Z (zero), and H (half-carry, for BCD).

9.2 Multi-byte arithmetic

Once you understand carry, multi-byte arithmetic is mechanical. To add the 16-bit value at \$22/\$23 to the 16-bit value at \$20/\$21, storing the sum back at \$20/\$21:

```
clrc                ; fresh add
mov  a, $20         ; load low byte of first
adc  a, $22         ; add low byte of second (no carry-in yet)
mov  $20, a         ; store low byte of result
mov  a, $21         ; load high byte of first
adc  a, $23         ; add high byte of second + carry from low byte
mov  $21, a         ; store high byte of result
```

The carry from the low-byte add propagates automatically into the high-byte add. This is the whole reason ADC includes carry.

The same idea extends to 24-bit, 32-bit, or arbitrary-width arithmetic: clear C once, then chain ADC instructions through successive byte pairs.

9.3 16-bit math the easy way: ADDW, SUBW, CMPW

For some 16-bit operations, the SPC-700 has dedicated instructions that operate on YA and a direct-page word.

Instruction	Effect	Cycles	Flags affected
ADDW YA, dp	YA = YA + word at dp/dp+1	5	N, V, H, Z, C
SUBW YA, dp	YA = YA - word at dp/dp+1	5	N, V, H, Z, C
CMPW YA, dp	Compare YA to word at dp/dp+1	4	N, Z, C

These do not use the carry flag as input — they always do a clean 16-bit operation. You don't need to CLRC first. They set V, N, H, Z, and C based on the full 16-bit result.

A common use:

```
; Add pitch_delta (16-bit at $20/$21) to current_pitch (at $22/$23)
movw ya, $22           ; YA = current_pitch
addw ya, $20           ; YA += pitch_delta
movw $22, ya           ; store back
```

Three instructions, instead of the six-instruction byte-by-byte version. Reach for ADDW/SUBW whenever your operands are 16-bit and live in direct page.

9.4 Compare: CMP

CMP does a subtraction but throws away the numerical result, keeping only the flags. After `CMP A, #5`:

- Z is set if `A == 5`.
- C is set if `A >= 5` (treating both as unsigned).
- N is set based on the sign of the (discarded) result.

The conditional branch instructions are designed around this:

```
cmp    a, #10
beq    equal           ; A == 10
bne    not_equal       ; A != 10
bcs    greater_eq      ; A >= 10 (unsigned)
bcc    less            ; A < 10 (unsigned)
```

BMI and BPL branch on the N flag, which gives you signed comparisons — but be careful. Signed comparison on the SPC-700 is not as clean as unsigned, because handling signed-overflow correctly requires looking at both N and V. For most purposes, work in unsigned arithmetic when you can.

CMP exists for A, X, and Y, with various addressing modes. The full table is in Appendix A; the most-used forms:

Instruction	Cycles
CMP A, #imm	2
CMP A, dp	3
CMP A, !abs	4
CMP X, #imm	2
CMP Y, #imm	2
CMPW YA, dp	4

9.5 Increment and decrement

The simplest arithmetic instructions are also the most-used.

```

inc    a
inc    x
inc    y
inc    $30           ; direct-page byte
inc    $30+x         ; direct-page indexed
inc    !table        ; absolute

dec    a
dec    x
dec    y
dec    $30

```

INC and DEC set N and Z but **do not affect C**. This makes them safe to use inside multi-byte arithmetic loops without worrying about clobbering the carry chain.

For 16-bit values in direct page, INCW and DECW operate on a word in one instruction:

Instruction	Cycles	Flags
INCW dp	6	N, Z (from full 16-bit result)
DECW dp	6	N, Z

9.6 Multiply: MUL YA

MUL YA multiplies Y by A and stores the 16-bit result back into YA. The high byte goes into Y, the low byte into A. It takes 9 cycles, which is fast — there is no software trick that beats it for general 8×8 multiplication.

```
mov    a, #7
mov    y, #3
mul    ya                ; YA = 7 × 3 = 21
                        ; A = 21, Y = 0
```

Flags: N and Z are set based on Y (the high byte of the 16-bit result). This is sometimes counterintuitive. If your product fits in 8 bits, Y will be 0 and Z will be set — even though the value (in A) is nonzero. Don't rely on Z for "is the product zero." Use CMP if you care.

MUL is the fastest way to scale things. For example, "voice number times 16" is "the DSP register base for that voice":

```
mov    a, voice_num
mov    y, #16
mul    ya                ; A = voice_num × 16, Y = 0 (since voice_num ≤ 7)
                        ; A is now the DSP register base for that voice
```

9.7 Divide: DIV YA, X

DIV YA, X divides YA by X. The quotient goes into A, the remainder into Y.

```
mov    a, #100
mov    y, #0
```

```
mov    x, #7
div    ya, x           ; A = 14, Y = 2 (100 / 7 = 14 remainder 2)
```

It takes 12 cycles. It is the only divide instruction the chip has.

DIV has a documented edge: the quotient is only fully defined when it fits in 9 bits — that is, when the result is at most \$1FF. The V flag receives bit 8 of the quotient, so V tells you whether the result didn't fit in A. Beyond a 9-bit quotient, the operation enters a documented-but-fiddly fallback that's not worth relying on.

The practical rule for ordinary 8-bit-quotient use: **keep $Y < X$** . That guarantees the quotient is at most 8 bits (V will be clear, A holds the full quotient), the remainder fits in Y, and you don't have to think about edge cases. If you want a quotient larger than 8 bits, use a software routine or stage the division in pieces.

For music drivers, DIV is occasionally useful for tempo calculations, but for note-frequency math it's usually replaced by precomputed tables — both because tables are faster, and because they sidestep the quotient-range question entirely.

9.8 Decimal adjust: DAA, DAS

Brief mention only. DAA (decimal adjust after addition) and DAS (decimal adjust after subtraction) are used when you want **binary-coded decimal (BCD)** arithmetic — packing two decimal digits per byte. They use the H flag set by ADC/SBC.

For music drivers, you almost never want BCD. Score displays, level numbers, and so on are the domain of the main CPU, not the SPC. Skip DAA/DAS unless you have a specific reason.

9.9 Two-address arithmetic in memory

A handful of instructions let you operate on memory without going through A:

Instruction	Effect	Cycles
ADC dp, dp	$dp = dp + dp + C$	6
ADC dp, #imm	$dp = dp + imm + C$	5
ADC (X), (Y)	$(X) = (X) + (Y) + C$	5
SBC dp, dp	$dp = dp - dp - !C$	6
SBC dp, #imm	$dp = dp - imm - !C$	5
CMP dp, dp	flags only	6
CMP dp, #imm	flags only	5

These are a real cycle savings when you'd otherwise do "load, op, store." They show up in sequencer code when you're doing things like "decrement the channel-N timer in memory" without disturbing A.

9.10 What you should remember

- ADC and SBC always include carry. Use CLRC before fresh adds, SETC before fresh subtracts.
- For 16-bit math, prefer ADDW/SUBW/CMPW over byte-by-byte.
- CMP is subtraction without storing the result. BEQ, BNE, BCS, BCC follow it naturally.
- INC/DEC don't touch C, so they're safe inside add/subtract chains.
- MUL YA is fast and useful. Especially "voice times 16" for DSP register addressing.
- DIV YA, X works but has quirks. Most music math uses precomputed tables.

9.11 Exercises

1. Write a routine that adds two 16-bit values: one at \$20/\$21, one at \$22/\$23, storing the result back at \$20/\$21. Use ADDW.

2. The variable `voice` at direct-page address `$10` holds a voice number 0–7. Write code that puts the voice’s DSP register base ($\text{voice} \times 16$) into `A`.
 3. Write code that compares the 16-bit value at `$20/$21` to `$1000` and branches to a label `too_high` if it’s greater, `too_low` if less, and `just_right` if equal.
 4. Why does `INC dp` not need a `CLRC` before it?
 5. After `MUL YA` where `Y` was 4 and `A` was 100, what is in `A` and `Y`? What flags are set?
-

Chapter 10

Logic, Shifts, and Bits

This chapter covers operations that treat bytes as bit patterns rather than as numbers: AND, OR, EOR (exclusive OR), shifts, rotates, and the SPC-700's surprisingly rich set of single-bit instructions.

10.1 Byte-wise logic: AND, OR, EOR

These work the way they do everywhere. AND clears bits where the second operand has a zero. OR sets bits where the second operand has a one. EOR flips bits where the second operand has a one.

```
and  a, #%11110000    ; clear lower 4 bits of A
or   a, #%00000001    ; set bit 0 of A
eor  a, #$ff          ; invert all bits of A
```

The % prefix introduces a binary literal — handy when you're thinking about which specific bits you want.

All three instructions set N and Z based on the result. None affect C.

Cycle costs follow the same pattern as ADC: 2 cycles for immediate, 3 for direct page, 4 for absolute, more for indirect modes. The full table is in Appendix A.

There are also memory-to-memory forms: AND dp, dp, AND dp, #imm, AND (X), (Y), and similarly for OR and EOR.

10.2 Shifts: ASL and LSR

A **shift** moves all bits one position. The bit shifted out goes into C; a zero is shifted in on the other end.

ASL (arithmetic shift left) shifts toward bit 7. The old bit 7 becomes C. This is equivalent to multiplying by 2 (with C catching the overflow).

LSR (logical shift right) shifts toward bit 0. The old bit 0 becomes C. This is equivalent to unsigned division by 2.

```

mov    a, #$0f           ; A = 00001111
asl    a                 ; A = 00011110, C = 0
asl    a                 ; A = 00111100, C = 0

mov    a, #$80           ; A = 10000000
asl    a                 ; A = 00000000, C = 1, Z = 1

```

Both instructions can operate on A, on direct-page bytes, on direct-page indexed bytes, or on absolute addresses.

10.3 Rotates: ROL and ROR

A **rotate** is like a shift, but the bit shifted out comes back in on the other side — *through* the C flag.

ROL (rotate left through carry): the new bit 0 is the old C, and the new C is the old bit 7.

ROR (rotate right through carry): the new bit 7 is the old C, and the new C is the old bit 0.

This means a rotate “remembers” what came before through the carry flag, which is exactly what you need for multi-byte shifts:

```

; Shift the 16-bit value at $20/$21 left by one bit
asl    $20               ; shift low byte; high bit goes to C
rol    $21               ; rotate high byte through C

```

The first instruction shifts; the second rotates the carry-out from the first into the bottom of the second byte. The 16-bit value has been shifted left as a single 16-bit unit.

10.4 XCN, revisited

We met XCN A in Chapter 8. It exchanges the high and low nibbles. It's worth noting again here because it's effectively four shifts in one instruction — XCN A is equivalent to four ASL A or four LSR A, but in 5 cycles instead of 8.

10.5 Single-bit operations on direct page

The SPC-700 has rich support for individual bits.

Instruction	Effect	Cycles
SET1 dp.bit	set bit bit of byte at dp	4
CLR1 dp.bit	clear bit bit of byte at dp	4

The notation dp.bit means a specific bit (0–7) of a specific direct-page byte. So SET1 \$30.0 sets bit 0 of the byte at \$30, and CLR1 \$30.7 clears bit 7.

This is a single 4-cycle instruction. Doing the same thing via load-OR-store would be $3 + 2 + 4 = 9$ cycles plus an extra register clobbered.

10.6 Test-and-modify on absolute bytes

Instruction	Effect	Cycles
TSET1 !abs	Set bits in memory that are set in A; flags from old memory – A.	6
TCLR1 !abs	Clear bits in memory that are set in A; flags from old memory – A.	6

In other words, TSET1 does $\text{memory} = \text{memory OR } A$ and TCLR1 does $\text{memory} = \text{memory AND NOT } A$. The flag behavior is the trap — N and Z come from a *subtraction* of A from the old memory value, not a bitwise test. If the old memory value equaled A, Z is set.

These instructions are useful when you want to atomically (well, in two cycles' worth of memory access) test-and-modify a flag word. In introductory code you can mostly ignore them.

10.7 Branching on a single bit

Instruction	Effect	Cycles
BBS <i>dp.bit, rel</i>	Branch if bit is set	5 not taken / 7 taken
BBC <i>dp.bit, rel</i>	Branch if bit is clear	5 not taken / 7 taken

These are wonderful. BBS \$30.0, handler branches to handler if bit 0 of \$30 is set. No load, no AND, no compare — just one instruction. Drivers use them constantly for flag bytes that hold many small flags.

10.8 Bit operations on the carry flag

Instruction	Effect	Cycles
MOV1 C, mem.bit	C = bit	4
MOV1 mem.bit, C	bit = C	6
AND1 C, mem.bit	C = C AND bit	4
AND1 C, /mem.bit	C = C AND NOT bit	4
OR1 C, mem.bit	C = C OR bit	5
OR1 C, /mem.bit	C = C OR NOT bit	5

Instruction	Effect	Cycles
EORl C, mem.bit	C = C EOR bit	5
NOTl mem.bit	bit = NOT bit	5

Here `mem.bit` is a 13-bit address followed by a 3-bit bit number, encoded into one operand. This lets you address any single bit in the entire 64 KiB address space.

These are powerful for assembling logic across scattered flag bits. They are also rarely needed in introductory driver code. You can write competent music drivers that never use them — but if you ever read disassemblies of compact, optimized drivers, you’ll see them.

10.9 Worked example: building a voice mask

Throughout the rest of the book we’ll often need to convert a voice number (0–7) into a “voice bit” — a byte with a single bit set in position N. This is the format that the KON, KOFF, EON, NON, and PMON DSP registers want.

A natural first attempt: shift a 1 left N times.

```
; Input: X = voice number, 0..7
; Output: A = byte with bit X set
voice_bit_loop:
    mov    a, #1
.loop:
    cmp    x, #0
    beq    .done
    asl    a
    dec    x
    bra    .loop
.done:
    ret
```

This is correct but slow — up to ~30 cycles for $X = 7$. A much faster version uses an 8-byte lookup table:

`voice_bit_table:`

```
db    %00000001
db    %00000010
db    %00000100
db    %00001000
db    %00010000
db    %00100000
db    %01000000
db    %10000000
```

`voice_bit:`

```
mov    a, !voice_bit_table+x
ret
```

Two instructions. About 6 cycles total, regardless of X . This is how real drivers look: small lookup tables for anything that would otherwise be a loop.

10.10 What you should remember

- AND, OR, EOR work bit-wise; they affect N and Z but not C.
- ASL/LSR shift; the bit shifted out goes to C, a zero comes in on the other side.
- ROL/ROR rotate through C, useful for multi-byte shifts.
- SET1 dp.bit / CLR1 dp.bit modify a single bit in 4 cycles.
- BBS / BBC branch on a single bit.
- TSET1 / TCLR1 test-and-modify, with subtraction-style flag behavior.
- Lookup tables are usually faster than computing things bit-by-bit.

10.11 Exercises

1. Write a routine that shifts the 24-bit value at \$20/\$21/\$22 right by one bit.
 2. The byte at \$30 holds eight independent flags. Write code that branches to `flag5_handler` if bit 5 is set.
 3. Using the 8-byte lookup table from the worked example, write code that, given `X = voice number`, sets exactly that voice's bit in the byte at \$31 without disturbing the other bits.
 4. After `TSET1 !flags` with `A = $05` and the byte at `flags` originally equal to \$01, what does `flags` become? What flags in PSW are set?
 5. Why might a driver author prefer `BBC $30.0, no_command` over the more explicit `MOV A, $30 / AND A, #1 / BEQ no_command`?
-

Chapter 11

Control Flow

A program is data plus a pattern of jumps. We've already used a few branch instructions in passing — BEQ, BNE, BCS, BCC showed up in earlier examples. This chapter pulls all the control-flow instructions into one place.

11.1 Conditional branches

A **conditional branch** tests one PSW flag and either branches or doesn't. Each branch instruction targets a specific flag and condition.

Instruction	Branch when...
BEQ rel	$Z = 1$ (last result was zero, or compare was equal)
BNE rel	$Z = 0$
BCS rel	$C = 1$ (carry set, or unsigned $\geq$ after compare)
BCC rel	$C = 0$
BMI rel	$N = 1$ (result negative)
BPL rel	$N = 0$ (result non-negative)
BVS rel	$V = 1$ (overflow set)
BVC rel	$V = 0$

All of them take 2 cycles when not taken, 4 cycles when taken. They are one of the few SPC-700 instructions whose cost depends on what happens at runtime.

The relative offset is a signed 8-bit byte added to the address of the *next* instruction (the byte after the branch instruction itself). This means a branch can reach approximately ± 128 bytes from where it sits. If you need to jump farther, use `JMP` instead — or invert the condition and branch over a `JMP`:

```
; This won't reach (target > 128 bytes away):
```

```
beq    far_label
```

```
; This will:
```

```
bne    skip
```

```
jmp    far_label
```

```
skip:
```

11.2 The unconditional branch: BRA

```
bra    target
```

BRA is unconditional. Same range as conditional branches, ± 128 bytes. 4 cycles. Useful because it's smaller and faster than `JMP` for short distances.

11.3 Compare-and-branch in one instruction

The SPC-700 has two combined compare-and-branch instructions that are wonderful for tight loops.

CBNE dp, rel: compare A to the byte at dp; branch if they're not equal.

```
mov    a, #0xff
```

```
cbne   $30, not_ff      ; branch if byte at $30 != 0xff
```

CBNE dp+X, rel: same, but the direct-page address is indexed by X. Useful for searching tables.

DBNZ dp, rel: decrement the byte at dp; branch if not zero.

DBNZ Y, rel: decrement Y; branch if not zero.

DBNZ is the canonical loop instruction. Want to do something 100 times?

```
mov    y, #100
.loop:
    ; ... loop body ...
    dbnz y, .loop
```

It's two instructions of overhead per iteration. If count lives in memory and you don't want to clobber Y, the direct-page form **DBNZ dp, rel** works the same way on a memory byte.

11.4 Jumps: JMP

```
jmp    !address           ; absolute, 3 cycles
jmp    [!address+x]       ; absolute indirect indexed, 6 cycles
```

JMP !address is a plain unconditional jump anywhere in the 64 KiB address space.

JMP [!address+X] is a *jump table* in disguise. It treats `address + X` and `address + X + 1` as a 16-bit pointer and jumps there. This is how dispatch tables work — you have an array of routine pointers, you set X to your “command number times 2,” and one instruction routes you to the right routine.

Example dispatch:

```
; X = command number (0..N), already multiplied by 2 by the caller
jmp    [!command_table+x]
```

```
command_table:
    dw    play_song
    dw    stop_song
    dw    set_volume
    dw    play_sfx
    ; ...
```

(dw declares a 16-bit data word — two bytes, little-endian.)

This pattern is everywhere in real sound drivers. Commands from the main CPU come in as a byte; you double it and dispatch.

11.5 Subroutine calls: CALL and RET

A **subroutine** is a block of code you can invoke from multiple places, returning to the caller when done. The instructions for this are CALL and RET.

- CALL !abs pushes the address of the *next* instruction onto the stack, then jumps to !abs.
- RET pops a 16-bit address off the stack and jumps there.

So calling a subroutine looks like:

```
; somewhere in the main code:  
call do_thing  
; ... continues here after do_thing returns ...
```

```
do_thing:  
    ; ... the subroutine body ...  
    ret
```

When CALL do_thing executes, the address right after it is pushed on the stack, and execution jumps into do_thing. When RET executes inside do_thing, that pushed address is popped and used as the new PC. We're back where we left off.

Subroutines can call other subroutines. As long as every CALL is matched by a RET, the stack stays balanced and everything works. If you push extra things on the stack inside a subroutine, you have to pop them before RET, or RET will pop the wrong bytes and jump somewhere unintended.

Instruction	Cycles	Notes
CALL !abs	8	General subroutine call.
PCALL upage	6	Calls a routine in page \$FF. The operand is a single byte (the low byte of the target).
TCALL n	8	Indexed call through a vector table at the top of memory.
RET	5	Return from CALL.
RETI	6	Return from interrupt; pops PSW and PC.

PCALL is a one-byte-operand variant for calling routines whose addresses are in the high page (\$FF00-\$FFFF). It's a code-size optimization rarely worth using unless you're squeezing the last bytes from a tight driver.

TCALL n calls one of 16 specific addresses, looked up in a vector table at the top of memory. TCALL 0 reads its target from \$FFDE/\$FFDF; TCALL 1 from \$FFDC/\$FFDD; the pattern is $\text{vector_address} = \$FFDE - 2 \times n$. Each TCALL is a one-byte instruction (the opcode encodes n directly), making this

the most compact way to dispatch — but only 16 targets, in a region that collides with the IPL ROM unless you’ve hidden it.

TCALL is rarely used in practice. If you see it in a disassembly, look at the vector table at \$FFC0-\$FFDF to see what it’s calling.

11.6 Software interrupt: BRK

BRK pushes PC and PSW, sets the B flag, clears the I flag, and jumps to the address stored at \$FFDE/\$FFDF. This is the same vector as TCALL 0.

In practice, BRK is essentially never used in SNES audio code. There is no debugging system to handle it. If you encounter BRK in a disassembly, it’s almost certainly a bug — typically your CPU has wandered into memory it shouldn’t be executing and is interpreting data as instructions. The opcode \$00 is NOP (do nothing) and \$0F is BRK, so a CPU that runs off into a region of zero-filled memory will simply NOP its way along until it eventually hits whatever non-zero garbage comes next, at which point execution becomes unpredictable.

BRK takes 8 cycles.

11.7 Returning from interrupt: RETI

RETI pops PSW first, then pops PC. This is for actual interrupt handlers, which on the SNES means almost nothing — the SPC-700 has no working interrupt sources. RETI is useful only if you’ve pushed PSW and PC manually for some reason.

11.8 CPU control instructions

A handful of small instructions affect the CPU itself rather than memory.

Instruction	Cycles	Effect
CLRC	2	C = 0
SETC	2	C = 1

Instruction	Cycles	Effect
NOTC	3	C = NOT C
CLRV	2	V = 0, H = 0
CLRP	2	P = 0 (direct page = \$0000-\$00FF)
SETP	2	P = 1 (direct page = \$0100-\$01FF)
EI	3	I = 1 (interrupts enabled)
DI	3	I = 0 (interrupts disabled)
NOP	2	nothing
SLEEP	3	sleep until interrupt — don't use
STOP	2	halt CPU — don't use

The first six are useful. The interrupt-related ones (EI, DI) only change the I bit and are essentially decorative on the SNES.

SLEEP and STOP are dangerous. With no working interrupt source on the SNES, SLEEP waits forever — execution stops until an interrupt that never comes. STOP halts the CPU until reset. Either one will hang your driver. If the CPU somehow ends up executing them (usually because of a bug), the music stops.

NOP is a do-nothing instruction. Sometimes useful for fine-tuning timing in extremely tight loops, but in practice the SPC-700 is not the place to do cycle-accurate work the way you might on the main CPU.

11.9 A simple main loop

We have enough to write a driver shell now. Here is the shape of a driver's main loop:

```
main_loop:
    call poll_command_ports
    call music_tick
```

```
call service_dsp_writes
bra  main_loop
```

Three subroutines, called in sequence forever. Each of them is small. `poll_command_ports` reads the four mailbox bytes and decides whether the main CPU has issued a new command. `music_tick` advances the music sequencer. `service_dsp_writes` writes any pending DSP register changes.

We will fill these in over the next few chapters.

11.10 What you should remember

- Conditional branches reach ± 128 bytes. For longer jumps, use `JMP`.
- `DBNZ Y, rel` is the canonical loop instruction.
- `JMP [!table+X]` is the dispatch-table instruction.
- `CALL` pushes the actual return address; `RET` pops it.
- `SLEEP` and `STOP` will hang your driver. Don't use them.
- Most drivers run a forever loop calling a small set of subroutines.

11.11 Exercises

1. Write a routine that searches the 16-byte table at `samples_table` for a byte equal to A. Return `X` = the index where it was found, or `X = $FF` if not found.
2. Write a 256-iteration loop using `DBNZ Y, rel`. (Hint: 0 in Y means 256 iterations after the first decrement.)
3. Sketch a dispatch table that handles four commands: 0 = play song, 1 = stop song, 2 = set master volume, 3 = play SFX. The command byte arrives in A. Use `JMP [!table+X]`.
4. Why is `BRK` essentially never used in SPC code? What would you do if you wanted a “trap on bug” feature?
5. Suppose your driver's main loop calls three subroutines, and they take a total of 200 cycles per pass. About how many times per second

does the main loop run, given the 1.024 MHz clock? Is this enough for sample-rate work? For tempo-tick work?

Part III

The S-DSP

You have been programming a CPU. It has registers and memory; it executes instructions; it has nothing inherent to do with sound.

Sound happens in a different chip — the **S-DSP** — which sits next to the SPC-700 and listens to it through a tiny window of two memory addresses. Everything you've learned in Part II is in service of pushing the right bytes through that window at the right time.

This part covers what's on the other side of the window: voices, pitch, samples, envelopes, noise, modulation, echo. It is here that the book stops being about a CPU and starts being about a synthesizer.

Chapter 12

The DSP Register Window

The SPC-700 talks to the S-DSP through exactly two memory-mapped addresses: \$F2 and \$F3. This chapter is about how that window works, the etiquette of writing through it, and what the DSP's register file looks like.

12.1 The window

\$F2 DSPADDR Selects which DSP register to access
\$F3 DSPDATA Reads or writes the selected register

To write a value V to DSP register R:

```
mov    $f2, #R           ; select register R
mov    $f3, #V           ; write V to it
```

To read DSP register R:

```
mov    $f2, #R
mov    a, $f3             ; A = current value of register R
```

The DSP's register file has 128 registers, addressed from \$00 to \$7F. The high bit of \$F2 (\$80) is special: when set, it disables writes through \$F3. This is a hardware safety feature; in practice, most code masks the address to 7 bits and ignores the high bit. If you find your DSP writes mysteriously aren't taking effect, check that you didn't accidentally set bit 7 of \$F2.

12.2 The MOVW idiom

Writing two bytes to \$F2 and \$F3 in sequence is so common that the SPC-700 essentially has a one-instruction shortcut:

```
mov    a, #R           ; register address in A
mov    y, #V           ; value in Y
movw   $f2, ya         ; writes A to $F2, Y to $F3
```

MOVW \$f2, ya writes A to \$F2 and then Y to \$F3 back-to-back, in five cycles total. This is the canonical DSP-write idiom, and you'll see it dozens of times in any driver.

Be aware: “back-to-back from the SPC’s point of view” is not the same as “atomic with respect to the DSP’s polling cycle.” The DSP samples its registers at specific phases of its own internal cycle, and a MOVW can land anywhere relative to those phases. For most registers this doesn’t matter. For KON/KOFF (covered below) it occasionally does.

A very common helper subroutine looks like:

```
; Write Y to DSP register A
; Inputs:
;   A = register number (0-127)
;   Y = value
dsp_write:
    movw   $f2, ya
    ret
```

You’ll call this constantly. Some drivers inline it everywhere instead of using a CALL.

12.3 When the DSP actually reads its registers

The S-DSP doesn’t react instantly to register writes. It runs an internal cycle that samples different registers at different phases. For most registers — volumes, ADSR settings, sample sources — the polling happens often enough (per-sample or every few samples) that you can write whenever

you want and the change takes effect within a small fraction of a millisecond.

For **KON** and **KOFF** (which we'll meet in detail in Chapter 13), the polling is timing-sensitive. Community-documented timing puts the KON/KOFF poll at every second sample, on a fixed phase of the DSP's processing cadence. The detail you need to know as a driver author is that **a single voice should not be both keyed-on and keyed-off in the same poll window**, or the DSP may key it on and immediately enter release, producing a missed or stuttering note.

The practical consequences:

- Don't write KON twice in quick succession; only the latest write may take effect.
- Don't key-off and immediately key-on the same voice within a few CPU instructions; stage them across ticks instead.
- Use shadow bytes in direct page to accumulate "voices to key on" across a tick, then write KON exactly once at the end of the tick.

12.4 The DSP register map at a glance

Here is the structure of the 128-register file. We'll explain what each register *does* over the next few chapters.

Per-voice registers (8 voices × 10 bytes each):

For voice number V (0 through 7), the register base is $V \times 16$.

Offset	Name	Description
\$x0	VOLL	Left volume (signed, -128 to +127)
\$x1	VOLR	Right volume
\$x2	PITCHL	Pitch low byte

Offset	Name	Description
\$x3	PITCHH	Pitch high byte (lower 6 bits used)
\$x4	SRCN	BRR sample source number
\$x5	ADSR1	ADSR enable + attack/decay rates
\$x6	ADSR2	Sustain level + sustain rate
\$x7	GAIN	GAIN-mode envelope control
\$x8	ENVX	Current envelope value (status; hardware-updated, normally read only)
\$x9	OUTX	Current output sample (status; hardware-updated, normally read only)
\$xA-\$xF	(mixed)	Not part of the per-voice slot. The columns at \$xC, \$xD, \$xF hold global registers (see below); other addresses are undocumented.

So voice 0's pitch low byte is at register \$02, voice 1's is at register \$12, voice 2's is at register \$22, and so on.

Global registers:

Address	Name	Description
\$0C	MVOLL	Main volume left
\$1C	MVOLR	Main volume right

Address	Name	Description
\$2C	EVOLL	Echo volume left
\$3C	EVOLR	Echo volume right
\$4C	KON	Key-on bits (one per voice)
\$5C	KOFF	Key-off bits
\$6C	FLG	Flags: reset, mute, echo write disable, noise rate
\$7C	ENDX	End-of-sample flags (hardware-set status)
\$0D	EFB	Echo feedback
\$2D	PMON	Pitch modulation enable
\$3D	NON	Noise enable
\$4D	EON	Echo enable
\$5D	DIR	Sample directory page
\$6D	ESA	Echo start address (page)
\$7D	EDL	Echo delay length
\$0F-\$7F (every \$10)	FIR0-FIR7	Echo filter coefficients

If you look at the per-voice registers and the globals together, you'll see the pattern: each voice *X* occupies the row $\$X0$ – $\$X9$ (10 bytes), with the remaining columns at $\$XC$, $\$XD$, and $\$XF$ holding global registers — MVOL/EVOL/KON/KOFF/FLG/ENDX in the $\$xC$ column, EFB/PMON/NON/EON/DIR/ESA/EDL in the $\$xD$ column, and FIR0-FIR7 in the $\$xF$ column. A few addresses ($\$1D$, the $\$xA$ / $\$xB$ / $\$xE$ columns) have no documented function and should be left alone.

The full table is in Appendix B.

12.5 A first interaction: silencing the DSP

When you take over the DSP, the first thing you usually do is silence it. The FLG register has bits for this.

FLG (\$6C):

Bit	Meaning
7	RESET — soft reset; mutes output and resets all envelopes
6	MUTE — silences DAC output
5	ECHO WRITE DISABLE — prevents writes to echo buffer
4-0	Noise frequency (5-bit value, indexes a fixed rate table)

To put the DSP into a clean state:

```
mov  a, #$6c          ; FLG
mov  y, #%01100000    ; MUTE + ECHO WRITE DISABLE
movw $f2, ya
```

This silences output and prevents any in-progress echo writes from corrupting RAM. Then you set up your sample directory, your voices, and so on. Finally, when you're ready to make sound:

```
mov  a, #$6c          ; FLG
mov  y, #%00000000    ; clear MUTE; allow echo writes
movw $f2, ya
```

Bring up MUTE first, then deal with echo separately. Echo can't make noise until you've set EVOLL, EVOLR, and the FIR coefficients to sane values; we'll cover the safe sequence in Chapter 15.

12.6 Don't write what you don't have to

A general piece of style advice. The DSP register window is fast — 5 cycles per write — but if your driver is doing 60 ticks per second and 8 voices' worth of writes per tick, that adds up. The trick used by every real driver is

shadow registers: keep a copy of the DSP register state in direct page, and only write to the DSP when something has actually changed.

This is sometimes called a “DSP write queue” or a “DSP shadow.” A simple version:

```
; In direct page:
voice0_voll        = $40
dirty_flags        = $50    ; one bit per dirty register
; bit 0: voice 0 VOLL needs a write
; bit 1: voice 0 VOLR needs a write
; ...

; To set voice 0 left volume to $50:
mov    $40, #$50
setl   $50.0          ; mark dirty

; In service_dsp_writes:
bbc    $50.0, .no_voll_change
mov    a, #$00          ; voice 0 VOLL register address
mov    y, $40
movw   $f2, ya
clr1   $50.0
.no_voll_change:
; ... etc for every register ...
```

This pattern scales. We’ll see it in real driver layouts in Chapter 17.

12.7 What you should remember

- The DSP is accessed through a two-register window at \$F2 (address) and \$F3 (data).
- MOVW \$f2, ya is the canonical write idiom: register number in A, value in Y.
- KON and KOFF are timing-sensitive. Write them once per tick from a shadow byte.

- The DSP register file has 128 registers: 8 voices $\times$ 10 per-voice + globals scattered across the $\$xC/\$xD/\$xF$ columns.
- FLG controls reset/mute/echo-write-disable and the noise rate.
- Most drivers use shadow registers to avoid redundant DSP writes.

12.8 Exercises

1. Write a routine that takes a voice number 0–7 in X and a register offset 0–15 in A, and computes the absolute DSP register address for that voice. (Hint: $\text{voice} \times 16 + \text{offset}$.)
 2. Write a routine that silences voice 3 by setting both its VOLL and VOLR to zero.
 3. Sketch a “dirty bit” scheme for shadow registers covering one voice’s eight writable per-voice registers (VOLL, VOLR, PITCHL, PITCHH, SRCN, ADSR1, ADSR2, GAIN). How many bits do you need? Where would you store them?
 4. The high bit of $\$F2$ disables writes through $\$F3$. Why might this exist as a hardware feature?
 5. What happens, qualitatively, if you write to KON twice within a few SPC instructions?
-

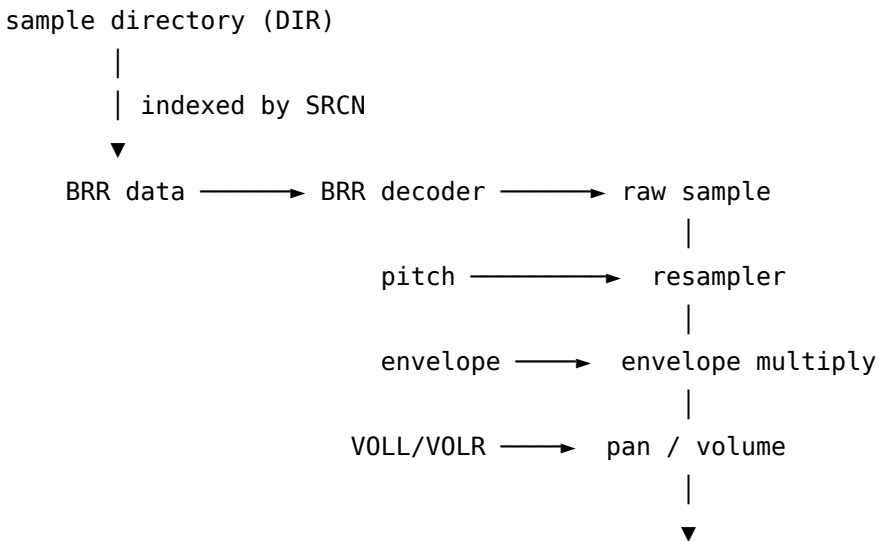
Chapter 13

Voices, Pitch, and Envelopes

The S-DSP has eight voices. Each one is a little sample player: it reads compressed sample data from ARAM, decodes it on the fly, multiplies by its envelope, multiplies by its volume, and adds the result to the stereo mix. We covered the register layout in Chapter 12. This chapter explains what each per-voice register actually controls, so you can play notes that sound like notes.

13.1 A voice, conceptually

The signal flow inside one voice is:



to main mix and (optionally) echo

Five things control this for each voice:

- **SRCN** picks which sample to play.
- **PITCHL/PITCHH** control how fast that sample plays back.
- **ADSR1/ADSR2** or **GAIN** control how the envelope evolves over time.
- **VOLL/VOLR** control the final per-voice stereo amplitude.
- **KON/KOFF** start and end notes.

We'll cover each in turn.

13.2 SRCN: picking the sample

SRCN (Source Number) is a single byte: 0–255. The DSP combines it with the directory pointer DIR to find the BRR data:

$$\text{directory entry address} = \text{DIR} \times 256 + \text{SRCN} \times 4$$

If DIR is \$20, the directory lives at \$2000-\$23FF. SRCN = 0 means look at \$2000, SRCN = 1 means look at \$2004, and so on, up to SRCN = 255 at \$23FC.

Each directory entry is four bytes:

bytes 0-1: BRR start address (little-endian)

bytes 2-3: BRR loop address (little-endian)

We'll cover BRR format itself in Chapter 14. For now: **SRCN picks an entry in the directory, and the directory tells the DSP where the actual sample data is.** You can have up to 256 samples loaded at once, in principle. In practice, ARAM is small and games typically load fewer than 32 samples at a time.

SRCN is sampled on key-on. Changing SRCN while a voice is playing has no effect until the voice is keyed on again or until the sample loops.

13.3 Pitch: PITCHL and PITCHH

The pitch register is a 14-bit value spread across two bytes. Bits 0–7 go in PITCHL; bits 8–13 go in the low 6 bits of PITCHH (the top 2 bits of PITCHH are ignored).

The relationship is:

$$\text{playback rate} = \frac{\text{pitch}}{\$1000} \times 32000 \text{ Hz}$$

So:

- pitch = \$1000 plays the sample at its native 32 kHz rate.
- pitch = \$0800 plays it an octave below (\$1000 / 2).
- pitch = \$2000 plays it an octave above.
- pitch = \$0000 doesn't play.
- The maximum pitch is \$3FFF, just under two octaves above native rate.

This means **the sample's recorded pitch matters too**. If you record middle C at the sample rate that gives it \$1000 playback, then to play middle C you set pitch = \$1000. To play C an octave higher, set pitch = \$2000. To play C an octave lower, set pitch = \$0800. The DSP doesn't know what note your sample "is" — it only knows the rate.

In practice, samples are recorded at a known reference pitch, and the driver maintains a **pitch table** indexed by note number that maps "musical note" to "DSP pitch value."

A reasonable pitch table covers some range of semitones — say, two octaves — with each entry being the 16-bit pitch value for that note. To play note N at the sample's native pitch, you write the table entry. To detune or slide, you do 16-bit arithmetic on the pitch using ADDW/SUBW.

Pitch calculations follow the equal-temperament formula:

$$\text{pitch}(\text{note}) = \text{pitch}(\text{reference}) \times 2^{(\text{note} - \text{reference})/12}$$

So if $\text{pitch}(C4)$ is \$1000, then $\text{pitch}(C5)$ is \$2000, $\text{pitch}(D4)$ is $\$1000 \times 2^{(2/12)} \approx \1107 , and so on. Most drivers ship a precomputed table.

13.4 ADSR mode

ADSR1 and ADSR2 together implement a four-stage envelope: attack, decay, sustain, release.

ADSR1 (\$x5):

bit 7: ADSR enable (1 = ADSR mode, 0 = GAIN mode)

bits 6-4: decay rate (3 bits, 0-7)

bits 3-0: attack rate (4 bits, 0-15)

ADSR2 (\$x6):

bits 7-5: sustain level (3 bits, 0-7; level = $(n+1)/8$ of full)

bits 4-0: sustain rate (5 bits, 0-31; release/sustain decrement)

When a voice is keyed on with ADSR1 bit 7 set, the envelope starts at zero and:

1. **Attack** rises to full level (\$7FF) at the rate set by attack rate.
2. **Decay** falls from full level to the sustain level at the rate set by decay rate.
3. **Sustain** stays at the sustain level, optionally decaying further at the sustain rate.
4. **Release** (when KOFF is set for the voice) falls toward zero at a fixed rate.

The exact timing for each rate value comes from a hardware lookup table. Higher rate values mean faster envelope changes. Attack rate \$F is essentially instant; rate \$0 is so slow it's almost stationary. Drivers usually pick from a small palette of envelope shapes per instrument.

A typical ADSR setup for a "piano-ish" sound:

```

; Voice 0 ADSR: attack fast, decay medium, sustain medium-low, sustain r
mov  a, #$05          ; voice 0 ADSR1
mov  y, #$8F          ; ADSR enable, decay $00, attack $F (fast)
movw $f2, ya
mov  a, #$06          ; voice 0 ADSR2
mov  y, #$60          ; sustain level $3 (4/8), sustain rate 0
movw $f2, ya

```

Don't worry about getting these values “right” yet — finding good ADSR settings is part of the craft of SNES music. The point is to know what each bit field does.

13.5 GAIN mode

If ADSR1 bit 7 is **clear**, the voice uses GAIN (\$x7) instead of ADSR. GAIN can do one of two things depending on its top bit:

If GAIN bit 7 = 0 (direct mode): the lower 7 bits set the envelope level directly, from 0 to \$7F. The envelope is held at that level forever.

If GAIN bit 7 = 1 (parameterized mode): the next two bits choose one of four envelope behaviors, and the lower five bits are a rate.

Bits 6-5	Mode	Behavior
00	Linear decrease	Subtract a constant per step
01	Exponential decrease	Multiply by $1 - 1/256$ per step
10	Linear increase	Add a constant per step
11	Bent-line increase	Add larger steps below \$5FF, smaller above

The 5-bit rate field selects how often a step happens, again via a hardware table.

GAIN is more flexible than ADSR for special purposes. Drivers commonly:

- Use ADSR for normal instrument envelopes.
- Switch a voice to GAIN mode mid-note for special fades.
- Use direct GAIN to set a fixed level for percussion.
- Use linear-decrease GAIN for a clean “fade out” effect that ADSR can’t quite do.

A useful idiom: set ADSR1 to \$00 (clearing bit 7), then write GAIN directly. The voice will play at whatever GAIN level you set, with no ADSR at all.

13.6 VOLL and VOLR

VOLL and VOLR are signed 8-bit values: -128 to $+127$. They scale the voice’s output for the left and right channels independently.

Negative volume is real — it inverts the polarity of the signal. This is rarely useful musically, but it’s how the hardware handles “subtractive” mixing in some echo configurations.

For normal use, treat VOLL/VOLR as 0 – 127 . To pan a voice to the right, set VOLL low and VOLR high. To pan to the left, the reverse.

Per-voice volume interacts with master volume (MVOLL/MVOLR), the envelope (ENVX), and the sample value itself. The full mix equation, simplified:

$$\text{voice_left} = \text{sample} \times \text{ENVX} \times \text{VOLL}$$
$$\text{voice_right} = \text{sample} \times \text{ENVX} \times \text{VOLR}$$
$$\text{mix_left} = \text{sum of voice_left for all voices, then scaled by MVOLL}$$
$$\text{mix_right} = \text{sum of voice_right for all voices, then scaled by MVOLR}$$

This is why a voice can have full volume and still be silent: if ENVX is zero (no key-on, or post-release), the envelope multiplier zeroes everything out. And it's why master volume affects everything uniformly.

13.7 KON: starting a note

KON (\$4C) is one byte. Bit N corresponds to voice N. Setting bit N keys-on voice N: it resets the envelope, restarts the BRR decoder at the start of the sample, and begins playback.

Critically, **KON is sampled by the DSP on its internal cycle**, not instantly. If you write to KON twice in quick succession, the DSP may process only the latest write.

The discipline:

```
; In your tick handler:  
; - Build up a "voices to start this tick" byte in a shadow.  
; - Just before exiting the tick, write the shadow to KON in one go.
```

```
mov    a, kon_shadow  
mov    y, a  
mov    a, #$4c          ; KON  
movw   $f2, ya
```

```
mov    kon_shadow, #0    ; clear for next tick
```

You only ever write KON once per tick. If you key-on voice 0 and voice 3 in the same tick, you set bits 0 and 3 in the shadow and write %00001001 to KON.

13.8 KOFF: ending a note

KOFF (\$5C) is similar: setting bit N causes voice N to enter its release phase. Unlike KON, KOFF is sticky — as long as the bit is set, the voice stays in release. To restart a voice cleanly, you have to clear its KOFF bit before keying it on again.

The pattern:

```
; To release voice 2:
mov    $f2, #$5c
mov    $f3, #%00000100    ; bit 2 set

; Some time later, before keying-on voice 2 again:
mov    $f2, #$5c
mov    $f3, #%00000000    ; clear all KOFF bits
; (Or selectively clear bit 2 from a shadow byte.)
```

In a real driver, KOFF is also handled through a shadow byte: each tick you compute the desired KOFF state and write it once.

13.9 ENDX: knowing when a sample finishes

ENDX (\$7C) is a status register. Bit N is set by the hardware when voice N processes a BRR block whose END flag is set — including end-with-loop blocks, not only end-and-stop blocks. So a voice that’s looping will *also* see its ENDX bit set every time it reaches the end of its loop body.

Most drivers use ENDX as a hint: “this voice has hit at least one end block since I last looked.” A simple read:

```
mov    a, #$7c            ; ENDX
mov    $f2, a
mov    a, $f3            ; A = current ENDX bits
; bit N set means voice N has reached an end block at some point;
; combine with your driver's own bookkeeping to decide what that means.
```

Treat ENDX as **hardware-updated status**, not as a read-to-clear event queue. ENDX bits are set by the hardware on end-flag blocks and updated on the next key-on for that voice. If you need precise per-note completion events, track your own state in the driver and corroborate against ENDX rather than relying on ENDX alone.

13.10 What you should remember

- Each voice occupies 10 register addresses in a 16-byte stride. `$x0–$x7` are the ordinary control registers; `$x8 (ENVX)` and `$x9 (OUTX)` are status registers that the hardware updates and that you normally only read.
- `SRCN` picks the sample. Pitch controls playback rate; `$1000` is native (32 kHz).
- `ADSR1/ADSR2` give you a four-stage envelope. `GAIN` gives you direct or parameterized control.
- `VOLL/VOLR` are signed; treat them as 0-127 for normal panning.
- `KON` starts notes; `KOFF` starts releases. Both are written from shadow bytes once per tick.
- `ENDX` bits are set when a voice processes a `BRR` end-flag block (including looped end blocks). Use it as a hint, not a precise event queue.

13.11 Exercises

1. The pitch value to play middle C (`C4`) is `$1000`. What pitch value plays `G4` (7 semitones above)? What about `C5`? Use the formula and compute a couple by hand; you don't need exact precision.
 2. Write a routine that sets voice 4's pitch from a 16-bit value at direct-page address `$30/$31`.
 3. A voice has `ADSR1 = $8F` and `ADSR2 = $E0`. Decode the four envelope parameters: attack, decay, sustain level, sustain rate.
 4. Write a routine that, given a voice number in `X`, releases that voice (sets the appropriate bit in a `koff_shadow` byte at direct-page `$50`).
 5. Why does the hardware require `ENVX` in addition to `VOLL/VOLR`? What would music sound like if you only had `VOLL/VOLR` and had to fake the envelope by changing them every tick?
-

Chapter 14

BRR Samples

We have been waving our hands about “the sample” for a chapter and a half. In this chapter we open up the format. BRR (Bit Rate Reduction) is the SNES’s compressed sample format. Every sound the SNES makes — every note, every drum hit, every speech sample — is BRR-encoded in ARAM somewhere. Understanding it is non-negotiable.

14.1 The block layout

A BRR sample is a sequence of **9-byte blocks**. Each block encodes 16 sample points using 4 bits each, plus one header byte:

byte 0: header (HSSS FFLE)

bytes 1-8: 16 nibbles, each one signed 4-bit sample

Block size: 9 bytes. Samples per block: 16. Compression: 16 samples $\times$ 16 bits = 32 bytes uncompressed, vs. 9 bytes BRR. Ratio 32:9 $\approx$ 3.5:1, with a fixed bit rate.

14.2 The header byte

The header byte’s bits are:

bits 7-4: SHIFT — how much to shift the decoded nibbles left

bits 3-2: FILTER — predictor filter selection (0-3)

bit 1: LOOP — set means “loop on end” (only meaningful if E is set)

bit 0: END – set means this is the last block

The **shift** is a 4-bit value (0–15) that determines the dynamic range of this block. If shift is 0, the nibbles are interpreted directly: samples are in the range -8 to $+7$. If shift is 12, the nibbles are shifted left 12 bits: samples are in the range -32768 to $+28672$ (very loud). Encoders normally avoid shifts 13–15, since fullsnes documents that those values behave like shift 12 with the nibble’s sign treated specially. If you’re hand-encoding BRR, stay in the 0–12 range unless you have a specific reason.

The shift gives BRR its dynamic adjustment: quiet sections of a sample use small shifts, loud sections use larger shifts. The encoder picks per-block.

The **filter** is a number 0–3 that selects one of four predictor filters. We’ll cover those next.

The **loop** and **end** flags work together. $END = 1$ means “this is the last block.” If $LOOP$ is also 1, the decoder jumps to the loop point (specified in the sample directory) instead of stopping. If $LOOP$ is 0, the voice’s envelope effectively dies and the voice goes silent.

14.3 The four predictor filters

BRR is a *predictor* codec. Each decoded sample is the sum of the encoded nibble (after shift) and a prediction based on the previous one or two decoded samples. The four filters give different prediction strategies.

Let: s = the shifted nibble for this sample
 old = the previously decoded sample
 $older$ = the sample two positions ago

Then the four filters compute:

Filter	Equation	Typical use
0	$new = s$	No prediction
1	$new = s + old \times 15/16$	Bass, slow envelopes
2	$new = s + old \times 61/32 - older \times 15/16$	Sustained tones

Filter	Equation	Typical use
3	$\text{new} = \text{s} + \text{old} \times 115/64 - \text{older} \times 13/16$	Highly correlated

Filter 0 has no dependency on previous samples — it’s pure data. The other three depend on *old* (and sometimes *older*), which the decoder maintains across blocks.

This dependency is the source of most BRR pitfalls. **The very first block of a sample, and the first block at the loop point**, both have the issue that *old* and *older* are stale or undefined. The conventional solution — and what most encoders do by default — is to use **filter 0** for the first block, and often for the loop block too. This is a practical safety measure rather than a hardware requirement: a careful encoder that accounts for filter history and loop continuity can use other filters at these boundaries, but doing so cleanly is hard. Beginners should default to filter 0 at boundaries and not fight it.

14.4 The sample directory

The sample directory is a contiguous block of memory pointed to by the DSP register DIR. Each entry is 4 bytes:

bytes 0-1: BRR start address (little-endian)

bytes 2-3: BRR loop address (little-endian)

So if DIR is \$20, the directory starts at \$2000. Entry 0 is at \$2000-\$2003, entry 1 is at \$2004-\$2007, and so on. There’s no length field — the BRR data itself ends when a block has its END flag set.

The **loop address** is where the decoder jumps when it hits an end-with-loop block. It must point to a block boundary (a 9-byte BRR block alignment) within the sample. Many encoders point loop to the start of the sample for “loop the whole sample” behavior; others let you set a specific loop point for a “one-shot intro then sustained loop” structure.

14.5 What a sample looks like in memory

Suppose you have a sample that's 80 sample points long with a loop point at sample 16. The BRR encoding would be:

80 samples / 16 samples per block = 5 blocks

Block 0: filter 0 (safe initial), header bits 0xxx0000 (no loop, no end)

Block 1: filter N, header bits Hxxx0000 (intermediate)

Block 2: filter N, header bits Hxxx0000 (intermediate)

Block 3: filter N, header bits Hxxx0000 (intermediate)

Block 4: filter 0 or N, header bits Hxxx0011 (LOOP and END set)

If the sample is loaded at \$3000, the directory entry might look like:

\$3000: start address = \$3000

\$3009: loop address = \$3009 (block 1 – start of the "loop" portion)

The first block of the sample (the "intro") plays once on key-on. After block 4, the decoder loops to \$3009 and continues from there indefinitely.

14.6 Encoding samples

You almost certainly will not write your own BRR encoder. There are several good ones available — `brr_encoder` by Bregalad, `BRRtools`, and the encoders built into `AddMusicK` and `SNESMod` all do a competent job. The encoder's job is:

1. Split the sample into 16-sample blocks.
2. For each block, try each filter, compute the resulting reconstruction error, and pick the filter that minimizes error.
3. Pick the smallest shift that doesn't clip the reconstructed values.
4. Write the 9-byte block.
5. Set END/LOOP flags on the appropriate blocks.

For looped samples, the encoder needs to know the loop point and (often) needs to use filter 0 for the loop block to avoid history mismatch.

The musician's job is mostly to:

- Provide a clean source sample (16-bit PCM, ideally at a known sample rate).
- Choose a loop point that's audibly seamless.
- Provide enough sample length that the encoder has good prediction history.
- Listen to the result and adjust.

14.7 Pitfalls

There are five pitfalls every BRR sample writer eventually meets.

1. Loop-point clicks. If the sample value at the loop point doesn't match the value just before the end of the sample, you'll hear a click on every loop. Fix: edit the source so the loop's start and end are at the same value, ideally both at zero crossings.

2. Filter history mismatch at loop. Even if the sample values match, the *decoded* values at the loop point depend on the filter history at the end of the sample. If the encoder didn't use filter 0 for the loop block, the decoder reconstructs using stale history and you'll get a glitch. Fix: encode the loop block with filter 0, or use an encoder that handles this.

3. Filter 3 overflow. The aggressive filter 3 can overflow the 16-bit decoder range under some sample patterns, producing a loud pop. Fix: use a careful encoder; many encoders offer a "no filter 3" option.

4. Initial block click. If the first block uses a non-zero filter, the very start of the sample sounds wrong because there's no valid history. Fix: always use filter 0 for the first block.

5. The Gaussian interpolation bug. This is a real, documented S-DSP hardware bug. The DSP uses a Gaussian window for sample-rate interpolation. Under specific conditions — three consecutive maximum-negative samples in the interpolation window — the Gaussian sum overflows and produces a very loud spike. Avoid sample patterns that have three -32768 values in a row. In practice, this is rare in normal samples but can happen in test signals or aggressively-encoded BRR.

14.8 A small but realistic plan

Suppose you want to load three samples for your driver: a piano, a snare, and a bass. You decide to put the directory at \$2000 (so DIR = \$20) and the BRR data starting at \$2100.

Layout:

```
$2000-$200B    directory (3 entries × 4 bytes = 12 bytes)
$200C-$20FF    unused / reserved
$2100-...      piano BRR data
...           snare BRR data
...           bass  BRR data
```

The directory at \$2000–\$200B looks like this (each entry is two 16-bit, little-endian addresses):

Address	Bytes	Meaning
\$2000-\$2001:	\$00 \$21	; entry 0: piano start = \$2100
\$2002-\$2003:	\$A0 \$21	; entry 0: piano loop = \$21A0
\$2004-\$2005:	\$E5 \$21	; entry 1: snare start = \$21E5
\$2006-\$2007:	\$E5 \$21	; entry 1: snare loop = \$21E5 (loops the whole sample)
\$2008-\$2009:	\$A0 \$22	; entry 2: bass start = \$22A0
\$200A-\$200B:	\$A0 \$22	; entry 2: bass loop = \$22A0

The actual BRR data lives separately, at the addresses the directory entries point to:

```
$2100-$21A0:  piano intro (one-shot portion)
$21A0-$21E4:  piano sustain (looped)
$21E5-$229F:  snare (looped on itself)
$22A0-....:   bass
```

The directory and the BRR data are different regions of memory. The directory just stores pointers; the audio bytes live elsewhere.

To select the piano on voice 0, write $\text{SRCN} = 0$ to register \$04. To select the snare on voice 1, write $\text{SRCN} = 1$ to register \$14. The voice will load the corresponding directory entry on its next key-on.

14.9 What you should remember

- BRR is 9-byte blocks: 1 header + 8 data, encoding 16 samples.
- The header is HSSS FFLE: shift, filter, loop, end.
- Four filters: 0 = no prediction, 1-3 = increasing prediction strength.
- The sample directory has 4-byte entries: start address, loop address.
- Use filter 0 for the first block and (usually) the loop block.
- Loop points must align to BRR block boundaries.

14.10 Exercises

1. A BRR header byte is \$B2. Decode it: what is the shift? Filter? LOOP and END flags?
 2. Suppose your sample directory is at \$2000, you want sample $\text{SRCN} = 5$ to start at \$3500 and loop at \$3580. Write the four bytes that go into the directory at the right offset.
 3. Why must the loop address point to a block boundary?
 4. A musician complains that their drum sample “clicks every time it ends.” They’ve set END but not LOOP. What’s actually happening, and how would you fix it?
 5. Why is filter 0 always safe but filter 3 sometimes problematic?
-

Chapter 15

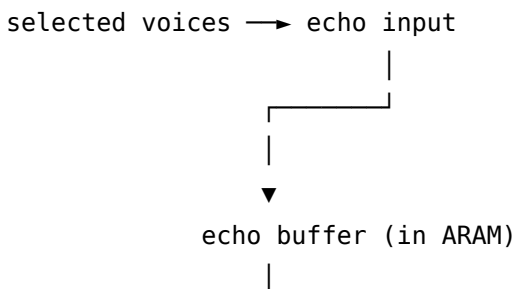
Echo, Noise, and Pitch Modulation

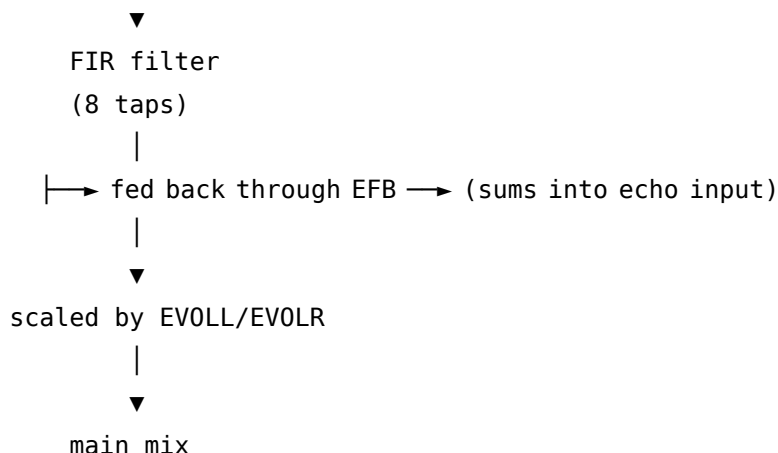
The S-DSP has three special features that go beyond “play eight samples and mix them”: **echo**, **noise**, and **pitch modulation**. None of them are necessary for music to exist. All three are part of what makes SNES music sound *like* SNES music — the echo on Final Fantasy VI, the noise-driven hi-hats in Donkey Kong Country, the modulated bass in Star Fox.

15.1 Echo: the famous SNES reverb

The S-DSP’s echo is, technically, a configurable feedback delay with an 8-tap FIR filter on the output. In musical terms, it’s a flexible reverb-like effect that ranges from subtle ambience to long swelling tails to wild metallic resonances. It’s the most distinctive sonic feature of the SNES.

The echo signal flow:





Six registers and one block of memory to manage:

EON (\$4D) — Echo Enable. One bit per voice. If voice N’s bit is set, that voice’s output is sent to the echo input. Voices not in EON play “dry.”

EVOLL (\$2C), EVOLR (\$3C) — Echo Volume Left and Right. Signed 8-bit. These scale the echo output before it’s mixed into the main signal.

EFB (\$0D) — Echo Feedback. Signed 8-bit. Controls how much of the echo output is fed back into the echo input, creating the recurring delay. Higher feedback means longer tails. Negative feedback can produce interesting “comb filter” effects.

ESA (\$6D) — Echo Start Address. Eight bits. The high byte of the echo buffer’s address in ARAM. So if ESA = \$E0, the echo buffer starts at \$E000.

EDL (\$7D) — Echo Delay Length. Lower 4 bits. Determines the buffer size:

$$\text{echo buffer size (bytes)} = \text{EDL} \times 2048$$

EDL = 0 is a special case (more on that below). EDL = 1 means 2 KiB, EDL = 15 means 30 KiB. The corresponding delay time is $\text{EDL} \times 16$ ms.

FIR0-FIR7 — eight 8-bit signed filter coefficients at registers \$0F, \$1F, \$2F, \$3F, \$4F, \$5F, \$6F, \$7F. These define an FIR filter applied to the echo output.

A simple “all-pass” setup is $\text{FIR7} = \$7\text{F}$ and the rest = $\$00$. Tuning these is part of the art of SNES sound design.

FLG bit 5 — Echo Write Disable. When set, the DSP doesn’t write to the echo buffer. This is essential during echo setup, because if the buffer overlaps your code or sample data and the DSP starts writing into it, you’ll corrupt RAM.

15.2 A safe echo setup

Setting up echo is a delicate dance:

```
; 1. Mute the DSP and disable echo writes.
mov    a, #$6c                ; FLG
mov     y, #%01100000         ; MUTE + ECHO WRITE DISABLE
movw    $f2, ya

; 2. Configure echo parameters while writes are disabled.
mov     a, #$4d                ; EON: which voices feed echo
mov     y, #%00001111         ; voices 0-3 send to echo
movw    $f2, ya

mov     a, #$6d                ; ESA = page $E0
mov     y, #$e0
movw    $f2, ya

mov     a, #$7d                ; EDL = 4 (8 KiB, 64 ms delay)
mov     y, #$04
movw    $f2, ya

mov     a, #$0d                ; EFB
mov     y, #$30                ; moderate feedback
movw    $f2, ya

mov     a, #$2c                ; EVOLL
```

```

mov    y, #$20
movw   $f2, ya

mov    a, #$3c                ; EVOLR
mov    y, #$20
movw   $f2, ya

; 3. Set FIR coefficients (this example: pass-through).
mov    a, #$0f                ; FIR0
mov    y, #$00
movw   $f2, ya
; ... zero out FIR1 through FIR6 ...
mov    a, #$7f                ; FIR7
mov    y, #$7f
movw   $f2, ya

; 4. Wait long enough for the DSP to flush any in-flight echo writes.
;     SNESdev errata recommend up to 7680 samples (~240 ms) for safety.
;     During that time, FLG bit 5 stays set.

; 5. Re-enable echo writes and unmute.
mov    a, #$6c                ; FLG
mov    y, #%00000000
movw   $f2, ya

```

The waiting step in (4) is real and worth taking seriously. The DSP can have echo writes already in flight when you change ESA or EDL; if you re-enable writes too soon, those in-flight writes can land at the *new* address with the *old* data, corrupting whatever's there. The conservative bound documented in SNESdev's errata is on the order of 7680 samples — about 240 ms at 32 kHz. A driver doing this once at startup can simply busy-wait that long.

Once writes are re-enabled, the DSP starts overwriting the echo buffer position by position as it cycles. It doesn't clear the buffer first, so any stale data in the buffer area will be audible for one echo period until it gets over-

written. The clean way to handle this is to manually zero the echo buffer area before enabling echo writes, or to keep the initial echo volume low and ramp it up as the buffer fills.

15.3 EDL = 0: the special case

EDL = 0 is not “echo off.” The DSP still maintains an echo buffer pointer and **continuously overwrites four bytes at ESA**, even with EDL = 0. So even “no echo” still touches RAM at ESA. If you set EDL = 0, you must either keep echo writes disabled (FLG bit 5 set) or make sure ESA points at memory you don’t care about.

To completely disable echo for a block of code: set FLG bit 5 (echo write disable) before doing anything that depends on echo not running.

15.4 The FIR filter

The 8-tap FIR coefficients are signed 8-bit values that act as a finite impulse response filter on the echo output. The output sample at time t is:

$$\text{filtered}_t = \frac{\text{FIR0} \cdot \text{echo}_{t-7} + \text{FIR1} \cdot \text{echo}_{t-6} + \dots + \text{FIR7} \cdot \text{echo}_t}{128}$$

Each FIR coefficient is a signed 8-bit value, but \$80 (−128) is best avoided in the actual coefficients — the asymmetry of two’s complement makes it the one value that cannot be cleanly negated, and it can interact badly with the filter’s internal arithmetic. Stay in \$81 to \$7F.

For overall gain, keep the *signed* sum of all eight coefficients somewhere around \$80 (≈ 128) for unity gain, less for attenuation. A signed sum well above 128 will amplify; one well below zero will invert and amplify, and either case can produce instability when feedback is also enabled. When in doubt, start conservative: zero out FIR0 through FIR6 and put \$7F in FIR7. That’s a clean pass-through and is impossible to make unstable. Then add complexity from there if you want to shape the echo’s tone.

Common configurations:

- **Pass-through:** $\text{FIR0}..6 = 0$, $\text{FIR7} = \$7\text{F}$. The output is the echo signal essentially unchanged.
- **Low-pass:** All eight coefficients positive and roughly equal, summing to about $\$7\text{F}$. Smooths the echo.
- **High-pass:** Coefficients alternating positive and negative, summing to zero. Produces a metallic, sibilant echo.
- **Band-pass / resonant:** More complex patterns; requires actual filter design knowledge.

If you don't know what you're doing, use pass-through. It sounds fine.

15.5 Echo buffer placement

The echo buffer eats ARAM at $\text{EDL} \times 2 \text{ KiB}$. Where you put it matters:

- **It must not overlap your code, your sample directory, or your samples.** ESA points to a buffer the DSP writes to continuously; overlap means corruption.
- **It must not overlap the direct page or the stack.** $\text{ESA} = \$00$ would overlap variables and stack and is a disaster.
- **The buffer must fit:** $\text{ESA} + \text{EDL} \times 8 \text{ pages}$ must not exceed $\$10000$.

Typical placement is at the top of ARAM:

$$\text{ESA} = \text{top of ARAM} - \text{EDL} \times 8 \text{ pages}$$

So for $\text{EDL} = 4$ (8 KiB), $\text{ESA} = \$\text{E0}$ puts the buffer at $\$ \text{E000} - \$ \text{FFFF}$. This requires hiding the IPL ROM (clearing bit 7 of $\$ \text{F1}$) so the buffer can extend through $\$ \text{FFFF}$.

For larger EDL values, the buffer eats deeper into your sample/code space. $\text{EDL} = 15$ (30 KiB) leaves you only 34 KiB for everything else. Most drivers stay at $\text{EDL} = 4$ or $\text{EDL} = 5$ unless they're doing something specifically reverberant.

15.6 Noise

The S-DSP can replace any voice's output with a pseudo-random noise signal. This is how you get hi-hats, snares, wind effects, and explosions on the SNES.

NON (\$3D) — Noise Enable. One bit per voice. Setting bit N replaces voice N's BRR output with noise.

FLG bits 4-0 — Noise Frequency. A 5-bit value indexing a fixed table of noise rates, from \$00 (slowest, 0 Hz — effectively no noise) up to \$1F (fastest, 32 kHz).

Important quirk: a noise voice still uses its BRR decoder, even though the output is replaced by noise. This means **a noise voice still needs a valid sample directory entry**. The trick: point the noise voice at a tiny dummy sample — a single block that loops on itself, with mostly silent data. The BRR decoder happily loops the dummy forever and the actual audio is the noise.

A typical setup:

```
; Voice 7 plays noise.
```

```
mov    a, #$3d          ; NON
```

```
mov    y, #%10000000    ; voice 7 noise enabled
```

```
movw   $f2, ya
```

```
; Set noise rate to mid-range.
```

```
mov    a, #$6c          ; FLG
```

```
mov    y, #%00010000    ; noise rate $10, no mute, no reset, no echo disab
```

```
movw   $f2, ya
```

```
; Voice 7's pitch and envelope still apply normally.
```

```
; Voice 7 must be keyed on with a valid (dummy) sample to start.
```


Pitch *does* affect noise — it controls how the noise is filtered/sampled. Lower pitch values produce more low-frequency-weighted noise; higher values produce brighter noise.

The envelope works as normal, so you can shape your hi-hats with ADSR or GAIN. This is how Donkey Kong Country gets its characteristic percussive sounds.

15.7 Pitch modulation

PMON (\$2D) lets you use one voice's output to modulate another voice's pitch. Setting bit N in PMON means "voice N's pitch is modulated by voice N-1's output."

Voice 0 cannot be pitch-modulated (there's no voice -1). Bit 0 of PMON has no effect.

This produces FM-like effects — a low-frequency LFO voice can wobble the pitch of a melody voice, creating vibrato, growl, or wobble bass. It's most effective when the modulator is a slow sine-like waveform and the carrier is a sustained note. The exact sound depends heavily on relative volumes and pitches.

PMON is one of the more obscure DSP features, but composers who learn it can produce sounds you won't get any other way. The Yoshi's Island soundtrack uses PMON extensively for its woozy, organic feel.

15.8 What you should remember

- Echo costs $\text{EDL} \times 2$ KiB of ARAM. $\text{EDL} = 4$ (8 KiB) is a reasonable default.
- Disable echo writes (FLG bit 5) during setup, then wait ~240 ms before re-enabling.
- FIR coefficients shape the echo's tone; pass-through (\$7F in FIR7, others zero) is the safe default.
- $\text{EDL} = 0$ still writes 4 bytes at ESA continuously — disable echo writes if you want true silence.

- Noise voices still use the BRR decoder; point them at a tiny dummy sample.
- PMON modulates voice $N+1$'s pitch with voice N 's output. Voice 0 can't be modulated.
- These three features turn "eight sample players" into a real synth.

15.9 Exercises

1. You want a 32-ms echo delay. What value should EDL be?
 2. With $EDL = 6$, where should ESA be to put the echo buffer at the top of ARAM (assuming the IPL ROM is hidden)?
 3. Voices 4 and 5 should be sent to echo; voices 0-3 and 6-7 should be dry. What value goes in EON?
 4. Why does a noise voice still need a sample directory entry pointing to a (dummy) BRR sample?
 5. You set up voice 1 to be pitch-modulated by voice 0. Voice 0 plays a slow sine wave. Voice 1 plays a sustained note. What musical effect do you expect to hear, and how would you intensify it?
-

Part IV

Music

You now have the pieces. You know the SPC-700's instructions, you know the DSP's registers, you know how samples and envelopes and echo work. What you don't yet know is how to put it all together into something a player would call "music."

The remaining four chapters cover the architecture of a sound driver, the conversation between the main CPU and the SPC, the modern composing workflow, and the tools you'll actually use.

Chapter 16

Inter-CPU Communication

The main CPU and the SPC-700 talk through four 8-bit ports. From the main CPU's side, they're memory-mapped at \$2140-\$2143. From the SPC's side, they're at \$00F4-\$00F7. The same four bytes; both CPUs can read and write them.

This is a remarkably small communication channel. A modern game engine might send the audio system a hundred messages per frame. The SNES gets four bytes, and even those have to be carefully sequenced.

16.1 The mapping

Main CPU	SPC	Name
\$2140	\$F4	Port 0
\$2141	\$F5	Port 1
\$2142	\$F6	Port 2
\$2143	\$F7	Port 3

Either side can read or write any port at any time. There is no built-in synchronization, no FIFO, no interrupt — just four shared registers.

16.2 The IPL upload protocol, in detail

We sketched the boot upload in Chapter 7. Now let's spell it out precisely.

Phase 1: handshake.

SPC writes \$AA to port 0, \$BB to port 1.

Main CPU spins reading port 0 and port 1 until it sees \$AA, \$BB.

Main CPU writes destination address to ports 2 and 3 (low, high).

Main CPU writes a nonzero value to port 1 (e.g., the byte count or a flag).

Main CPU writes \$CC to port 0. This signals "start transferring."

SPC sees \$CC and echoes it back to port 0.

Main CPU sees its own \$CC come back. Both sides are now synchronized.

Phase 2: byte transfer.

For each byte:

 Main CPU writes byte to port 1.

 Main CPU writes a counter to port 0 (incremented from previous).

 SPC sees the new counter, reads the byte from port 1, stores it.

 SPC echoes the counter to port 0.

 Main CPU sees its counter come back. Continue.

Phase 3: jump.

Main CPU writes the entry address to ports 2 and 3.

Main CPU writes \$00 to port 1 (signaling "jump, don't transfer more").

Main CPU writes a counter to port 0.

SPC reads ports 2 and 3, jumps there.

This is the protocol every commercial SNES game uses, every homebrew, every modern driver. The IPL ROM at \$FFC0-\$FFFF implements the SPC side; the main CPU has its own implementation in its boot code.

How long does upload take? SNESdev estimates roughly 520 master clocks per byte for a tight transfer loop, which works out to about 650 bytes per 60 Hz frame if the main CPU does nothing else. A 30 KiB upload (typical driver-plus-samples-plus-data image — samples are usually the

bulk) is therefore about 47 frames or roughly 0.78 seconds. Fast enough to do during a logo-screen fade-in, but not “instantaneous.” Plan for it.

16.3 Runtime communication

After the driver is uploaded and running, the four ports are used for runtime commands. There’s no standard protocol — every driver invents its own. Common patterns:

Pattern 1: command + arguments.

Main CPU writes:

- Port 0 = command number (e.g., 1 = play song)
- Port 1 = argument 1 (e.g., song number)
- Port 2 = argument 2
- Port 3 = argument 3

SPC's main loop polls port 0:

- If port 0 != last_seen_command:
 - Read ports 1-3 as arguments.
 - Dispatch on command number.
 - Echo command number back to (some agreed) port to acknowledge.
 - last_seen_command = port 0

Pattern 2: command counter.

To avoid the case where the main CPU sends the same command twice and the SPC can’t tell, drivers often use a *counter* in port 0 instead of the command number itself:

Main CPU:

- Port 1 = command number
- Port 2 = argument 1
- Port 3 = argument 2
- Port 0 = command counter (incremented every command)

SPC:

```
Loop: read port 0. If it differs from last_counter, process.  
    last_counter = port 0  
Echo counter back so main CPU knows the command was received.
```

This way, the same command number can be sent multiple times — what changes is the counter.

Pattern 3: ring buffer over multiple writes.

Some drivers use the four ports as a tiny window into a larger logical message stream. The main CPU writes one byte at a time and a sequence number; the SPC reads them out into a buffer. Higher bandwidth than naive command/args, at the cost of complexity.

16.4 The collision problem

There is a subtle issue. The main CPU and the SPC can both read and write the same port at the same time. If they both write at the same instant, only one write “wins.” If the main CPU writes while the SPC is reading, the SPC might see a partially-updated value (depending on hardware timing).

In practice this manifests as: **if you don’t have a clear protocol for who writes which port and when, you’ll occasionally see garbage values.**

The standard discipline is to give each port a clear “owner” at any given moment. For example: “Main CPU owns port 0 (writes commands) and port 1 (writes args). SPC owns ports 2 and 3 (writes acks).” When main CPU wants to send a command, it writes the args first, then the command counter; the SPC echoes through ports 2/3 only after fully consuming the command.

A common defensive trick is **double-read polling**: read the same port twice in a row, and only believe the value if you got the same thing both times. This catches the rare half-written values.

16.5 A polling skeleton

Here’s a minimal SPC-side command poller. It uses port 0 as a counter:


```

; Direct page:
last_cmd      = $10

poll_command:
    mov    a, $f4                ; read port 0 (command counter)
    cmp    a, last_cmd
    beq    .none                ; same as last time, no new command
    mov    last_cmd, a

    ; New command. Read fields, then dispatch.
    mov    a, $f5                ; port 1 = command type (0..127)
    asl    a                    ; double for 16-bit table indexing
    mov    x, a                  ; X = table offset (used by JMP)
    mov    a, $f6                ; A = arg 1 (preserved across JMP)
    mov    y, $f7                ; Y = arg 2 (preserved across JMP)
    jmp    [!command_table+x]    ; tail-call into the handler

command_table:
    dw     cmd_play_song         ; type 0
    dw     cmd_stop_song         ; type 1
    dw     cmd_set_volume        ; type 2
    dw     cmd_play_sfx          ; type 3
    ; ...

.none:
    ret

```

A few things worth noticing about this skeleton.

The `JMP [!command_table+x]` is a *tail call*: the dispatched handler doesn't return to `poll_command`, it returns to whoever called `poll_command` in the first place. That's fine, and it's why we don't need a `CALL/RET` pair — the handler's own `RET` does the work.

The handler receives arg 1 in A and arg 2 in Y. We deliberately put the command type into X (where the JMP needs it for indexing) and the args into the registers that survive the JMP. If a handler needs more than two bytes of arguments, it would have to read further from the ports itself, or stage args through direct-page bytes before the dispatch.

The ASL A doubles the command type for 16-bit table indexing. This means the command numbering must stay below 128 — at command 128, the doubled value wraps and we'd index into the wrong slot. Real drivers either keep the command set small or expand the dispatch to handle wider indices.

16.6 Latency

The main CPU sees the SPC respond on the order of a few microseconds to a few milliseconds, depending on how fast the SPC's main loop polls. A typical driver polls the command port at the start of every “tick” — every 60th of a second or so. That's the *worst-case* latency: about 16 ms. For sound effects, this is fine. For tight rhythmic synchronization (like cuing music to a video frame exactly), you have to send the command a tick or two early.

16.7 What you should remember

- Four bytes total: \$2140-\$2143 from the main CPU, \$00F4-\$00F7 from the SPC.
- Boot uses a fixed handshake (\$AA \$BB ↔ \$CC) followed by a byte-by-byte transfer.
- A 30 KiB upload takes roughly 0.78 seconds.
- Runtime communication is per-driver. Common pattern: command counter + args.
- Collisions can produce garbage. Define ownership and use counter/ack patterns.
- Latency is tens of milliseconds at worst, usually less.

16.8 Exercises

1. Write SPC-side code that detects “the main CPU has written a new value to port 0” using a `last_cmd` shadow byte.
 2. The main CPU wants to send a 6-byte “load song” command. How might you sequence this over four ports? Sketch the protocol.
 3. Why does the boot protocol echo the main CPU’s counter back? What goes wrong without that?
 4. If the SPC’s main loop takes 3 ms per pass, what’s the worst-case latency between “main CPU writes command” and “SPC sees it”?
 5. Suppose you write port 1 immediately after port 0 from the main CPU’s side, and the SPC happens to be in the middle of reading port 1 right then. What might the SPC see, and how does the “double-read” trick catch it?
-

Chapter 17

Anatomy of a Sound Driver

A **sound driver** is the SPC-side program that turns the S-DSP from “eight sample players” into “a music engine.” Every SNES game has one. They differ wildly in detail but share a structure. This chapter explains that structure.

17.1 What a driver does

Given the hardware we’ve described, a driver has roughly this list of responsibilities:

1. **Boot setup.** Initialize the DSP, install the sample directory, configure echo, set master volume.
2. **Command handling.** Poll the four ports for commands from the main CPU.
3. **Sequencing.** Walk through music data, advancing notes and parameters at the right times.
4. **Voice allocation.** Decide which of the eight voices plays which note. Handle priority (music vs. SFX, important vs. background).
5. **Per-voice state management.** Track each voice’s note, instrument, envelope phase, pitch slide, vibrato, panning.
6. **DSP register writes.** Translate per-voice state into actual DSP register writes.
7. **Effects processing.** Pitch slides, vibrato, tremolo, arpeggios, panning sweeps — all the things that make tracker music expressive.

8. **Sample loading.** Either load all samples once at boot, or stream samples in/out as songs change.

For an introductory driver, you can skip many of these. A minimum-viable driver only needs (1), (2), (3), (5), and (6). The rest are quality-of-life features.

17.2 The driver's main loop

Almost every driver looks like this at the top level:

```
main_loop:
    call poll_command_ports      ; (2)
    call wait_for_tick           ; tempo timing
    call music_tick              ; (3), (4), (5), (7)
    call service_dsp_writes      ; (6)
    bra  main_loop
```

Three things to notice.

First, the main loop is **infinite**. The SPC has no operating system; the driver runs forever from boot until console reset.

Second, the loop is structured around a **tick** — a fixed unit of musical time. A typical standalone SPC driver derives its tick from one of the SPC's three hardware timers and runs at somewhere between 60 Hz and 250 Hz. Some game engines instead drive tempo from the main CPU (sending a “tick” command per video frame) or use a hybrid; this book focuses on the standalone-timer case because it's how an SPC file plays back without a host.

Tempo is implemented by counting ticks. At 250 Hz, 24 ticks per 16th note gives a 16th-note duration of 96 ms; four 16ths per beat is 384 ms; that's about 156 BPM. Slowing or speeding the song means changing the ticks-per-row count, not changing the tick rate itself.

Third, **command polling and music ticking are separate**. You poll commands every pass through the loop, but only advance music on tick boundaries. Otherwise, music speed would depend on how busy the loop is.

17.3 Tick timing

Drivers usually tick by reading one of the SPC's three timers. Timers 0 and 1 use an 8 kHz base; timer 2 uses a 64 kHz base. Each timer has an 8-bit target and a separate **4-bit output counter** at `$FD/$FE/$FF`. The internal counter increments at the timer's base rate; whenever it reaches the target, the output counter increments and the internal counter resets. Reading the output register at `$FD/$FE/$FF` returns its current 4-bit value and resets it to zero.

Two things to be careful of:

- **The output counter is only 4 bits.** It saturates / wraps past 15 back to 0. If you read it less often than 16 ticks pass, you lose timing information silently — the counter has no way to tell you “the actual elapsed count was 18, not 2.” For a tick rate of 250 Hz, you must service the counter at least every $16/250 \approx 64$ ms; in practice you'll service it every loop pass, well above that.
- **CONTROL (`$F1`) is write-only and writes affect timers globally.** Reading `$F1` returns `$00`, not what you wrote. Writing to `$F1` always touches timer state — at minimum, a 0→1 transition on a timer's enable bit resets that timer's internal counter and output register. You cannot safely do read-modify-write on `$F1`. Always know exactly what bit pattern you're writing.

Here's the full CONTROL byte layout:

Bit	Name	Effect when set
7	IPLN	IPL ROM mapped at \$FFC0-\$FFFF (clear to expose underlying RAM)
6	—	unused
5	PC23	One-shot reset of mailbox ports \$F6/\$F7 (auto-clears)
4	PC01	One-shot reset of mailbox ports \$F4/\$F5 (auto-clears)
3	—	unused
2	T2EN	Timer 2 enabled
1	T1EN	Timer 1 enabled
0	T0EN	Timer 0 enabled

So the boot value \$30 (= %00110000) resets all four mailbox ports and hides the IPL ROM, with all timers disabled. The runtime value \$01 (= %00000001) enables Timer 0, leaves the IPL ROM hidden, and stops resetting the ports. The 0→1 transition on bit 0 between the two writes resets Timer 0's internal counter and output, giving us a clean tick stream from that moment forward.

To set up Timer 0 to fire at 250 Hz (4 ms ticks):

```

mov    $fa, #$20           ; T0TARGET = 32; 8000/32 = 250
mov    $f1, #$01           ; CONTROL: enable Timer 0
                                ; (Don't read $f1 first – it always reads as
```

To wait for the next tick:

```

wait_for_tick:
    mov    a, $fd           ; read T0OUT (resets to 0 after read)
    beq    wait_for_tick    ; loop until output is nonzero
    ret
```

Each pass through `wait_for_tick` reads `T0OUT`. If `T0OUT` is zero (no ticks have elapsed), we loop. If it's nonzero (one or more ticks have elapsed), we proceed. The read clears `T0OUT`, so on the next call we wait fresh.

This is a *passive* polling loop, which means the SPC is busy-waiting between ticks. That's fine — there's nothing else for the SPC to do, and 1 MHz is plenty for music processing.

17.4 Per-channel state

A driver maintains state for each of its “channels.” A *channel* is a software concept: a stream of musical events. The mapping between channels and DSP voices is usually 1:1 (channel 0 plays on voice 0), but for SFX priority, channels and voices can swap dynamically.

Per-channel state typically includes:

- Current note number
- Current instrument (sample + envelope settings)
- Pointer into the song data
- Tick countdown to next event
- Pitch slide / vibrato state
- Volume / panning / tremolo state
- Detune offset
- “Note off pending” flag

A reasonable layout in direct page, for an 8-channel driver:

```
$00    channel 0 song pointer low
$01    channel 0 song pointer high
$02    channel 0 ticks-to-next-event
$03    channel 0 current note
$04    channel 0 instrument
$05    channel 0 volume
$06    channel 0 vibrato phase
$07    channel 0 flags (note-off-pending, etc.)
```



```
$08      channel 1 song pointer low
$09      channel 1 song pointer high
... etc ...
```

```
$40      KON shadow
$41      KOFF shadow
$42      dirty bits 0 (which voices need register updates)
$43      dirty bits 1
```

Eight channels $\times$ eight bytes each = 64 bytes, fits nicely. Plus shadows for KON, KOFF, and dirty bits.

17.5 A single channel's tick

Here's what `music_tick` does, channel by channel:

```
for each channel:
```

```
    decrement ticks-to-next-event
    if ticks-to-next-event > 0:
        update slow effects (vibrato, slide, etc.)
        continue
```

```
    # Time for the next event. Read from song data.
```

```
    event = read byte from *channel.song_pointer; advance pointer
```

```
    if event is "set instrument":
```

```
        channel.instrument = read next byte
```

```
    elif event is "set volume":
```

```
        channel.volume = read next byte
```

```
    elif event is "play note":
```

```
        note = read next byte
```

```
        duration = read next byte
```

```
        channel.current_note = note
```

```
        channel.ticks-to-next-event = duration
```

```
    # Mark this voice for key-on.
```

```

        kon_shadow |= (1 << channel)
        # Mark this voice's registers as dirty.
        dirty_bits |= (1 << channel)
    elif event is "note off":
        koff_shadow |= (1 << channel)
    elif event is "end of pattern":
        ... handle looping ...
    elif event is "tempo":
        tempo = read next byte
    # ... more event types ...

```

This is the heart of any tracker-style or MML-style driver: a small interpreter for music data. The data format is a sequence of bytes, with each byte (or small group of bytes) representing a musical event.

17.6 service_dsp_writes

After ticking all channels, the driver translates state into DSP writes. The dirty-bit pattern is convenient:

`service_dsp_writes:`

```

        mov     x, #0                ; voice index
.loop:
    ; Compute bit mask for voice X. (Use the voice-mask lookup table from
    ; If dirty bit is clear, skip this voice.
    ; Otherwise, write all of this voice's registers from shadow data:
    ;   VOLL, VOLR, PITCHL, PITCHH, SRCN, ADSR1, ADSR2, GAIN
    ; Clear the dirty bit.

    inc     x
    cmp     x, #8
    bne     .loop

    ; Finally, write KON and KOFF shadows.
    mov     a, #$4c                ; KON

```

```
mov    y, kon_shadow
movw   $f2, ya
mov    kon_shadow, #0           ; clear for next tick

mov    a, #$5c                 ; KOFF
mov    y, koff_shadow
movw   $f2, ya
; Note: don't clear KOFF; it's sticky. Driver must explicitly clear

ret
```

This pattern — “build up shadow state during the tick, write it to the DSP at the end” — is universal among drivers. It avoids redundant writes, ensures KON gets exactly one write per tick, and keeps the timing predictable.

17.7 Voice allocation and SFX priority

When a sound effect needs to play, the driver has to give it a voice. If all eight voices are playing music notes, the driver has to decide what to do.

Common strategies:

- **Reserved SFX voices.** The driver reserves, say, voice 6 and voice 7 for sound effects. Music never touches them. SFX always goes there. Simple but wasteful — your music is limited to six voices.
- **Priority-based stealing.** Each note has a priority. SFX has higher priority than music. When SFX needs a voice, the driver “steals” the lowest-priority music voice and plays SFX there. The music voice is silenced and resumes when SFX finishes.
- **Adaptive allocation.** The driver tracks which voices are currently playing music and which are idle. SFX preferentially uses idle voices; if none are idle, it steals based on priority. Most modern drivers do something like this.

Voice allocation is one of the messier parts of a driver. It’s a real engineering problem disguised as a small detail.

17.8 A worst-case ARAM budget

Let's plan a serious driver's memory layout. Suppose:

- 5 KiB of driver code
- 16 instruments
- ~1 KiB of sequence data per minute of music; let's plan for two minutes resident, so 2 KiB
- 8 KiB echo buffer (EDL = 4)
- A directory of 32 sample slots (128 bytes)

The samples themselves take whatever's left:

Total ARAM:	65536 bytes
- Driver code:	5120
- Sequence data:	2048
- Sample directory:	128
- Echo buffer:	8192
- Variables/stack:	512

Available for samples: 49536 bytes $\approx$ 48 KiB

48 KiB of BRR, divided among, say, 12 unique instruments, gives each instrument an average of 4 KiB of BRR data. At ~3.5:1 compression, that's ~14 KiB of source PCM, or about 0.4 seconds at 32 kHz, or about 2 seconds at 8 kHz (downsampled).

This is the SNES audio reality: short samples, looped. Most instruments are 1–4 KiB of BRR. A piano sample might be 8 KiB. A long drum hit might be 6 KiB. You don't have room for orchestra-quality sample libraries — you have room for a small palette of instruments and the cleverness of the composer.

17.9 What you should remember

- A driver is a perpetual loop: poll commands, wait for tick, advance music, write DSP.
- Use a hardware timer to define ticks; commonly 60 Hz to 250 Hz.

- Maintain per-channel state in direct page. Use shadow registers for the DSP.
- `service_dsp_writes` applies dirty state to the DSP at the end of each tick.
- Voice allocation between music and SFX is a real engineering problem.
- ARAM budgets are tight. Plan for it from the start.

17.10 Exercises

1. Sketch the per-channel state structure for a 4-channel driver. How many direct-page bytes does it use?
 2. Suppose you want a tempo of 120 BPM with 24 ticks per beat. What rate does the driver need to tick at? What target value should you give Timer 0 (which runs at 8 kHz) to achieve that?
 3. Why is it important to write KON exactly once per tick, after all per-channel processing?
 4. Design a simple priority scheme for SFX-vs-music voice allocation. What happens when an SFX takes a music voice and finishes? How do you resume the music?
 5. Compute the ARAM budget for a driver that wants 16 KiB of echo buffer ($EDL = 8$). How does this constrain the music?
-

Chapter 18

A Composer's Workflow

You now know enough to read a driver. The next question is: how do you actually make music? In 1992, the answer was “work for a Japanese game studio that has its own driver and tools.” In 2026, the answer is much friendlier, and this chapter walks through the modern landscape.

18.1 The four stages

Every workflow, regardless of which tool you choose, has four stages.

- 1. Compose.** Write the actual notes. This happens in some kind of sequencer — a tracker, an MML compiler, or a DAW that can export to a SNES-friendly format.
- 2. Prepare samples.** Take whatever sounds you want (real instruments, synthesized tones, drums) and convert them to BRR. This includes choosing loop points, encoding, and listening for artifacts.
- 3. Assemble.** Combine the driver code, your sequence data, and your samples into an ARAM image — typically an SPC file or a ROM patch.
- 4. Test.** Play the result in an emulator, on real hardware via flashcart, or in a standalone SPC player. Iterate.

The tools you choose affect each of these stages, but the structure is universal.

18.2 Choosing a tool

Here's a survey of the options available, with what each is good for and what it costs you in setup.

AddMusicK. A toolchain and inserter that wraps a modified SMW/N-SPC-style sound engine, used to add custom music to Super Mario World ROM hacks. You write in an MML-like (Music Macro Language) text format. For an SMW hacker, AddMusicK *is* the music-driver workflow — it's a complete pipeline rather than just a converter. Mature and well-documented within the SMW hacking community.

- *Good for:* SMW hacks; getting started quickly with MML; learning by reading a real driver-and-toolchain.
- *Caveats:* Tightly coupled to SMW. If you're doing original homebrew, you'll want one of the homebrew-oriented tools below.

SNESMod. A homebrew tool that takes Impulse Tracker (.it) module files and converts them to SNES-playable form, with its own driver. Familiar for anyone who's used trackers like OpenMPT or ModPlug.

- *Good for:* Composers coming from the tracker world; homebrew projects that want a "drop-in" sound system.
- *Caveats:* IT subset only — some IT features don't translate. Limited to the driver's specific design choices. Channel and sample limits.

SNESGSS. An integrated tracker + driver that lets you compose directly in a tracker UI and produce a SNES-ready output. Has its own song format and driver.

- *Good for:* Composers who want a one-stop tool with a friendly UI; people who don't want to manage external tools.
- *Caveats:* Less flexible than IT; smaller community; specific to its driver.

Terrific Audio Driver (TAD). A modern homebrew driver designed from scratch. Uses an MML-like language. Well-documented, actively maintained, designed for general use.

- *Good for:* Modern homebrew projects; composers who want a clean driver with good docs.
- *Caveats:* MML-style input may not suit tracker-trained composers.

Roll your own. You can write your own driver. After this book, you have the foundation. People do.

- *Good for:* Learning, full control, or a unique sound design that no existing driver supports.
- *Caveats:* Significant engineering effort. Plan to spend weeks before you have something musically useful.

18.3 Where the existing N-SPC drivers fit

You may have heard of “N-SPC” or seen Star Fox sound driver disassemblies online. N-SPC is the family of sound drivers Nintendo and licensees used in many first-party games. It’s an excellent reference for studying how a real production driver works — but **the code is proprietary**, and using it (or a close derivative) for your own released project would be a copyright issue. Treat N-SPC disassemblies as study material, not as a starting point for your own work.

18.4 The composing experience

What does day-to-day SNES composing actually feel like? It depends on the tool, but the rhythms are similar.

You start with a sample bank. Maybe you pull eight or twelve samples from a freesound library, or you record your own, or you synthesize them in a DAW. You convert each one to BRR with your encoder, listening for clicks at the loop point and adjusting until it sounds clean.

You write notes — in MML, in a tracker, however your tool wants them. You assign instruments (which means assigning samples + envelope settings). You set tempo, you arrange parts.

You preview. The first preview always sounds wrong. The samples are too loud, or too quiet, or out of tune relative to each other. The envelope is too punchy on this instrument and too slow on that one. The echo is too washy or too dry.

You iterate. You re-encode samples with different settings. You fiddle with ADSR. You adjust per-instrument volume. You discover that the bass instrument needs a different sample than you started with.

And eventually you have something that sounds the way you wanted. The whole process feels like a constraint puzzle — what can you fit in 64 KiB that will sound good through this hardware? — and the constraints are part of the appeal.

18.5 Sample preparation, in detail

This is where most beginning SNES composers struggle. The samples you can find online are usually 44.1 kHz 16-bit; the SPC plays at 32 kHz; BRR encoding is lossy. A few practical tips:

Pre-process your sample. Trim silence, normalize amplitude, fade in and out if needed.

Resample to a sensible rate. If you record at 32 kHz natively, your sample plays at native pitch when the DSP pitch is \$1000. If you record at 16 kHz and double-pitch on playback, you save half the ARAM but add aliasing. If you record at 8 kHz and quadruple-pitch, you save 75% of the ARAM but the sample sounds noticeably gritty. Pick based on your priorities.

Choose loop points carefully. For sustained instruments (organ, strings, voice), pick a loop point where the waveform is at a zero crossing and the cycle is reasonably stable. For short percussive sounds, you usually don't loop — just set the END flag without LOOP.

Encode with awareness. Most encoders have a “use filter 0 only” option. This produces slightly worse compression but eliminates filter-history glitches. Use it when in doubt.

Listen on real hardware (or accurate emulation). Inaccurate emulation can make samples sound better or worse than reality. Test on hardware for important projects, on Mesen2 or similar for normal development.

18.6 A first project plan

If you've made it through this book and want to actually make music, here's a recommended first project.

1. **Pick a tool.** Terrific Audio Driver if you like text-based input, or SNES-Mod if you have prior tracker experience.
2. **Get the tool working.** Build the example song from the tool's documentation. Listen to it in an emulator. Confirm you can iterate from “edit text/tracker file” → “rebuild” → “hear result” in under a minute.
3. **Make a four-channel piece.** Bass, drums, melody, harmony. Use the example samples. Don't worry about quality yet; worry about flow.
4. **Replace one sample with your own.** Pick the easiest one — probably the bass — and substitute a sample you've prepared yourself. This forces you to do the BRR encoding loop.
5. **Replace the rest of the samples one at a time.** Each one is a new lesson in BRR.
6. **Add echo.** Tune EDL, EFB, and the FIR coefficients until you like the space.
7. **Make a second piece** with the same sample bank but a different mood. Notice what's easy and what's hard.
8. **Make a third piece** with completely different samples. Now you're building a sample library.

By the time you've done all eight, you'll know more about SNES audio than 99% of the people who play games on it. You'll also have a portfolio.

18.7 What you should remember

- Workflow has four stages: compose, prepare samples, assemble, test.
- Pick a tool that matches your composing style — MML, tracker, or DAW export.
- Sample preparation is where most beginners struggle. Practice it.
- Iteration is everything. Set up your toolchain so the loop is fast.
- The constraints are the medium. Embrace them.

18.8 Exercises

1. Look up Terrific Audio Driver and SNESMod. Read each one's "getting started" page. Which sounds more like your composing style?
 2. Find a free 16-bit WAV sample online (a piano note, say). Convert it to BRR using whatever encoder is available with your chosen tool. Listen to the encoded version. What changed?
 3. Pick a SNES game whose music you like. Find an SPC of one of its songs. Open it in an SPC player that shows DSP state (Mesen2 works) and observe: how many voices are active? What are their pitches? How do they change over time?
 4. Make a 30-second test piece: just one instrument, one melody, no harmony. Get it sounding clean before you add anything else.
 5. The composer's puzzle: you have 32 KiB available for samples. You want a piano (12 KiB), a bass (4 KiB), a drum kit (10 KiB total across kick, snare, hat), and pads (6 KiB). It fits! But the piece feels thin. What's the cheapest sample you could add to give it more texture?
-

Chapter 19

Tooling, Testing, and SPC Files

The last chapter of the main text covers the practical infrastructure: which assembler to use, which emulator to debug in, and what's actually inside an SPC file.

19.1 Assemblers

You will need an assembler to translate your SPC-700 source code into machine code. Several have direct SPC-700 support; the choice mostly comes down to ecosystem.

Asar is the most common assembler in the SNES ROM-hacking world. Its syntax is friendly, it has good macro support, and many existing SNES audio projects (including the Star Fox sound driver disassembly and AddMusicK) use it. Asar's `arch spc700-inline` directive lets you mix SPC-700 and 65816 code in the same source file, which matches how the upload protocol actually works.

```
; Asar example
arch spc700-inline      ; tell Asar this is SPC code
org $0200

start:
    mov     x, #$ff
    mov     sp, x
```

; ...

WLA-DX has direct SPC-700 support and is widely used in homebrew projects with C/ASM build pipelines. **bass** and **xkas-plus** are also listed by SNESdev's tools page as supporting SPC-700.

ca65 is part of the cc65 toolchain and is a great fit for the 65816 side of a SNES homebrew project, but its native instruction set support is for 6502/65816, not SPC-700. Projects that want to use ca65 across the whole codebase typically emit SPC-700 code through macro packs or preprocessing — workable, but more work than picking an assembler with native SPC-700 support.

For learning, **Asar** is probably the easiest start because the most documentation and example code in the SNES audio scene uses it. Pick **WLA-DX** if you're already using it for the rest of your project.

19.2 Emulators with SPC debuggers

You will spend many hours staring at debugger windows. Pick a good one.

Mesen2 (or the older Mesen-S) is the modern recommendation. It has separate debuggers for the main CPU and the SPC, register-write logging, ARAM hex view, DSP register view, breakpoint and watchpoint support, and event tracing. Free and open source.

bsnes-plus is a debugging fork of bsnes. Less actively maintained than Mesen2 but still useful — different UI, similar capabilities. Some experienced users prefer it.

no\$sns is Martin Korth's emulator. Strong on hardware accuracy and tightly tied to his fullsnes documentation. If you're investigating obscure DSP behavior, no\$sns is often where you'll find an emulator that matches the hardware most precisely.

For day-to-day development, Mesen2 is the most ergonomic. Use no\$sns when you're chasing a bug that might be at the hardware-accuracy level.

19.3 What to debug for

A few specific things to watch for as a beginning SPC programmer.

Step through your boot code. The first thing the SPC does after upload is execute your start label. Watch the registers. Is the stack pointer set correctly? Is \$F1 getting the value you expect?

Inspect ARAM after upload. Confirm your samples are at the addresses you expected. Confirm your sample directory entries match. A driver that “doesn’t make sound” is often a driver where the directory points at empty memory.

Watch DSP register writes. Mesen2 lets you log every write to \$F2/\$F3. If your driver “stops working after a few seconds,” check whether you’re writing nonsense to a DSP register at some point.

Check ENDX and ENVX. If a voice “isn’t playing,” check ENVX — if it’s zero, the envelope is silent. Check ENDX — if it’s set, the sample finished and the voice may be effectively muted.

Listen. Always listen. Even with the best debugger, your ears are your ground truth. If something sounds off, try to describe it precisely (clicks at loop points? pitch wobbling? envelope too snappy?) — that description will guide where to look.

19.4 The SPC file format

An .spc file is a snapshot of most of the audio subsystem at a moment in time. It captures:

- The full 64 KiB of ARAM
- All 128 DSP registers
- The SPC-700’s CPU registers (PC, A, X, Y, PSW, SP)
- Optional ID666 metadata (game title, song title, composer, length, etc.)

What it **does not** capture is the full live state of the APU. Internal items not in the file include the timer counters, the echo buffer pointer, the BRR decoder/interpolator state for each voice, the active envelope phases, and the current values latched on the four mailbox ports. Players reconstruct as much as they can from the captured state and let the SPC and DSP “settle” — which usually works well enough that you don’t notice, but can produce subtly different playback for the first few hundred milliseconds compared to the original session.

The Alpha-II v0.30 file layout, which is what most modern SPC files use:

Offset	Size	Contents
\$00000	33	Header magic: "SNES-SPC700 Sound File Data v0.30"
\$00021	2	\$1A \$1A
\$00023	1	Tag-present flag (\$1A = ID666 present, \$1B = no tag)
\$00024	1	Minor version
\$00025	2	PC (little-endian)
\$00027	1	A
\$00028	1	X
\$00029	1	Y
\$0002A	1	PSW
\$0002B	1	SP
\$0002E	210	ID666 text/binary tag area
\$00100	65536	64 KiB ARAM dump
\$10100	128	DSP register dump
\$10180	64	Unused
\$101C0	64	Extra RAM (the IPL ROM region's underlying RAM)

(Earlier versions of the format had slightly different layouts; the v0.30 layout above is the one to read against.)

When an SPC player starts a song, it loads this snapshot into a virtual SPC and lets it run. Whatever the SPC was doing at the moment of capture is what the player hears. This is why .spc files are great for sharing music: they’re self-contained.

19.5 Creating SPC files

Most drivers have a way to dump an SPC at runtime. Some emulators have a “save SPC” feature that captures the current state. AddMusicK and SNES-Mod both produce SPC files as part of their build process. Once you have a driver running, capturing an SPC is trivial.

This is also how SPC archives like SNESmusic.org work: someone played the game, captured an SPC of each song, and tagged it. The driver and samples are all in the snapshot.

19.6 ID666 metadata

ID666 metadata holds the human-readable information about the song. The standard ID666 tag — the one all players understand — sits in the SPC file’s **header**, in the 210-byte block at offset \$0002E (just after the captured CPU registers and before the ARAM dump). Common fields stored there:

- Song title
- Game title
- Dumper’s name
- Comments
- Date dumped
- Number of seconds before fade
- Length of fade
- Composer
- Default channel disables
- Emulator used

A later extension called **xid6** adds richer metadata — longer titles, multiple composer fields, OST track numbers, and so on. xid6 lives *after* the body of the file (after the ARAM and DSP register dumps, starting around \$10200), as an optional appended block. Older players read only the header tag and ignore xid6; modern players read both and prefer xid6 fields where they conflict with the header tag.

Players read this and display it. When you publish your music, fill in at least the standard tag — the title, composer, and game/album name. `xid6` is nice-to-have.

19.7 Running on real hardware

For most learning, an emulator is fine. When you want to put your music on a real SNES, you have two options:

1. **Flash cartridge.** A device like the SD2SNES / FXPak Pro lets you load ROMs from an SD card. You assemble your code into a ROM, copy it to the SD card, plug the cartridge into the SNES, and play. This is how most homebrew is tested.
2. **Burn a real ROM.** Possible but more involved — you need a programmer and a flashable EPROM cartridge. Mostly only relevant if you're producing physical copies.

For SPC files specifically, you can also use audio-only playback: there are dedicated SPC players that work standalone (no game), and some flash carts have an SPC player mode. Capture an SPC, play it on hardware, listen to it through actual SNES analog output. The result will sound subtly different from the emulator — some emulators get the noise, the high-frequency rolloff, or the analog character not quite right. For released work, listen on hardware at least once.

19.8 A small testing checklist

Before you call a song “done”:

- ☐ Listen on at least two different setups (e.g., headphones and speakers).
- ☐ Listen on real hardware or a known-accurate emulator.
- ☐ Check for clicks at sample loops.
- ☐ Check for clipping in loud sections.
- ☐ Check that echo doesn't produce sustained drone or feedback.

- ☐ Check that all voices key off cleanly at the end.
- ☐ Check that pause/resume (if your driver supports it) works.
- ☐ Listen one more time, the next morning, with fresh ears.

19.9 What you should remember

- Asar is the most common assembler in the SNES audio scene. WLA-DX is a solid alternative; bass and xkas-plus also support SPC-700 directly. ca65 is for the 65816 side and emits SPC-700 only via macro packs.
- Mesen2 is the recommended emulator for SPC debugging.
- An SPC file is a snapshot of the entire audio state, but missing some internal hardware items (timers, envelope phases, decoder state).
- ID666 metadata is what players display; fill it in.
- For released work, listen on real hardware.

19.10 Exercises

1. Set up Mesen2 and load any commercial SNES ROM. Find the SPC debugger. Step through the SPC code for a few thousand instructions while music plays. What patterns do you see?
 2. Find a publicly-available SPC file (e.g., an SPC of a game whose music you like). Open it in a hex editor. Find the ARAM dump (offset \$0100). Can you identify the BRR sample data? The driver code?
 3. Look up Asar's documentation. Write a "hello world" SPC program that uploads, plays a single tone using the first-voice example from Chapter 13, Voices, Pitch, and Envelopes, and runs forever.
 4. List three differences between hearing a song through an emulator and hearing it on real hardware. Why does each one matter?
 5. Why is the SPC file format the way it is? What design pressures led to "snapshot the whole state"?
-

Part V

Appendices

Appendix A

Full Instruction Reference

This appendix lists every SPC-700 instruction by group, with cycle counts and flag effects. Flags are written in the order **N V P B H I Z C**; a dot means unchanged.

A.1 Move and stack

Instruction	Cycles	Flags
MOV A, #imm	2	N.....Z.
MOV A, (X)	3	N.....Z.
MOV A, (X)+	4	N.....Z.
MOV A, dp	3	N.....Z.
MOV A, dp+X	4	N.....Z.
MOV A, !abs	4	N.....Z.
MOV A, !abs+X	5	N.....Z.
MOV A, !abs+Y	5	N.....Z.
MOV A, [dp+X]	6	N.....Z.
MOV A, [dp]+Y	6	N.....Z.
MOV X, #imm	2	N.....Z.
MOV X, dp	3	N.....Z.
MOV X, dp+Y	4	N.....Z.

Instruction	Cycles	Flags
MOV X, !abs	4	N.....Z.
MOV Y, #imm	2	N.....Z.
MOV Y, dp	3	N.....Z.
MOV Y, dp+X	4	N.....Z.
MOV Y, !abs	4	N.....Z.
MOV (X), A	4	unchanged
MOV (X)+, A	4	unchanged
MOV dp, A	4	unchanged
MOV dp+X, A	5	unchanged
MOV !abs, A	5	unchanged
MOV !abs+X, A	6	unchanged
MOV !abs+Y, A	6	unchanged
MOV [dp+X], A	7	unchanged
MOV [dp]+Y, A	7	unchanged
MOV dp, X	4	unchanged
MOV dp+Y, X	5	unchanged
MOV !abs, X	5	unchanged
MOV dp, Y	4	unchanged
MOV dp+X, Y	5	unchanged
MOV !abs, Y	5	unchanged
MOV A, X	2	N.....Z.
MOV A, Y	2	N.....Z.
MOV X, A	2	N.....Z.
MOV Y, A	2	N.....Z.
MOV X, SP	2	N.....Z.
MOV SP, X	2	unchanged
MOV dp, dp	5	unchanged

Instruction	Cycles	Flags
MOV dp,#imm	5	unchanged
MOVW YA,dp	5	N.....Z.
MOVW dp,YA	4	unchanged
PUSH A/X/Y/PSW	4	unchanged
POP A/X/Y	4	unchanged
POP PSW	4	(restored)
XCN A	5	N.....Z.

A.2 Arithmetic

Instruction	Cycles	Flags
ADC A,#imm	2	NV..H.ZC
ADC A,(X)	3	NV..H.ZC
ADC A,dp	3	NV..H.ZC
ADC A,dp+X	4	NV..H.ZC
ADC A,!abs	4	NV..H.ZC
ADC A,!abs+X	5	NV..H.ZC
ADC A,!abs+Y	5	NV..H.ZC
ADC A,[dp+X]	6	NV..H.ZC
ADC A,[dp]+Y	6	NV..H.ZC
ADC (X),(Y)	5	NV..H.ZC
ADC dp,dp	6	NV..H.ZC
ADC dp,#imm	5	NV..H.ZC
SBC (same modes as ADC)	(same)	NV..H.ZC
CMP A,#imm	2	N.....ZC
CMP A,(X)	3	N.....ZC
CMP A,dp	3	N.....ZC

Instruction	Cycles	Flags
CMP A,dp+X	4	N.....ZC
CMP A,!abs	4	N.....ZC
CMP A,!abs+X	5	N.....ZC
CMP A,!abs+Y	5	N.....ZC
CMP A,[dp+X]	6	N.....ZC
CMP A,[dp]+Y	6	N.....ZC
CMP X,#imm	2	N.....ZC
CMP X,dp	3	N.....ZC
CMP X,!abs	4	N.....ZC
CMP Y,#imm	2	N.....ZC
CMP Y,dp	3	N.....ZC
CMP Y,!abs	4	N.....ZC
CMP (X),(Y)	5	N.....ZC
CMP dp,dp	6	N.....ZC
CMP dp,#imm	5	N.....ZC
ADDW YA,dp	5	NV..H.ZC
SUBW YA,dp	5	NV..H.ZC
CMPW YA,dp	4	N.....ZC
INC A/X/Y	2	N.....Z.
INC dp	4	N.....Z.
INC dp+X	5	N.....Z.
INC !abs	5	N.....Z.
DEC (same as INC)	(same)	N.....Z.
INCW dp	6	N.....Z.
DECW dp	6	N.....Z.
MUL YA	9	N.....Z.
DIV YA,X	12	NV..H.Z.

Instruction	Cycles	Flags
DAA A	3	N.....ZC
DAS A	3	N.....ZC

A.3 Logic

Instruction	Cycles	Flags
AND A,#imm	2	N.....Z.
AND A,(X)	3	N.....Z.
AND A,dp	3	N.....Z.
AND A,dp+X	4	N.....Z.
AND A,!abs	4	N.....Z.
AND A,!abs+X	5	N.....Z.
AND A,!abs+Y	5	N.....Z.
AND A,[dp+X]	6	N.....Z.
AND A,[dp]+Y	6	N.....Z.
AND (X),(Y)	5	N.....Z.
AND dp,dp	6	N.....Z.
AND dp,#imm	5	N.....Z.
OR (same as AND)	(same)	N.....Z.
EOR (same as AND)	(same)	N.....Z.
AND1 C,mem.bit	4	C
AND1 C,/mem.bit	4	C
OR1 C,mem.bit	5	C
OR1 C,/mem.bit	5	C
EOR1 C,mem.bit	5	C
NOT1 mem.bit	5	unchanged
MOV1 C,mem.bit	4	C

Instruction	Cycles	Flags
MOV1 mem.bit,C	6	unchanged

A.4 Shifts and rotates

Instruction	Cycles	Flags
ASL A	2	N.....ZC
ASL dp	4	N.....ZC
ASL dp+X	5	N.....ZC
ASL !abs	5	N.....ZC
LSR (same as ASL)	(same)	N.....ZC
R0L (same as ASL)	(same)	N.....ZC
R0R (same as ASL)	(same)	N.....ZC

A.5 Bits

Instruction	Cycles	Flags
SET1 dp.bit	4	unchanged
CLR1 dp.bit	4	unchanged
TSET1 !abs	6	N.....Z.
TCLR1 !abs	6	N.....Z.
BBS dp.bit,rel	5/7	unchanged
BBC dp.bit,rel	5/7	unchanged

A.6 Branches and jumps

Instruction	Cycles	Flags
BRA rel	4	unchanged
BEQ rel	2/4	unchanged
BNE rel	2/4	unchanged
BCS rel	2/4	unchanged
BCC rel	2/4	unchanged
BMI rel	2/4	unchanged
BPL rel	2/4	unchanged
BVS rel	2/4	unchanged
BVC rel	2/4	unchanged
CBNE dp,rel	5/7	unchanged
CBNE dp+X,rel	6/8	unchanged
DBNZ dp,rel	5/7	unchanged
DBNZ Y,rel	4/6	unchanged
JMP !abs	3	unchanged
JMP [!abs+X]	6	unchanged
CALL !abs	8	unchanged
PCALL upage	6	unchanged
TCALL n	8	unchanged
RET	5	unchanged
RETI	6	(restored)
BRK	8	...1.0..

A.7 Flag and CPU control

Instruction	Cycles	Flags / Effect
CLRC	2	0
SETC	2	1

Instruction	Cycles	Flags / Effect
NOTC	3	~
CLRV	2	.0..0...
CLRP	2	..0.....
SETP	2	..1.....
EI	3	1..
DI	3	0..
NOP	2	unchanged
SLEEP	3	hangs SPC on SNES
STOP	2	halts CPU; reset only

Branch cycle notation 5/7 means 5 cycles when not taken, 7 when taken.

Appendix B

DSP Register Map

The S-DSP has 128 registers. They are addressed through \$F2 (address) and \$F3 (data) on the SPC side. The high bit of \$F2 disables writes when set; clear it for normal use.

B.1 Per-voice registers (for voice V , base = $V \times 16$)

The eight voices are accessed at register bases \$00, \$10, \$20, \$30, \$40, \$50, \$60, \$70. Each voice occupies ten register addresses with the layout below; the remaining addresses in each voice's row (\$xA–\$xF) are reserved for global registers and are documented in the next section.

Offset	Name	Access	Description
\$x0	VOLL	R/W	Left volume, signed 8-bit
\$x1	VOLR	R/W	Right volume, signed 8-bit
\$x2	PITCHL	R/W	Pitch low byte
\$x3	PITCHH	R/W	Pitch high byte (low 6 bits used; top 2 ignored)
\$x4	SRCN	R/W	BRR sample source number
\$x5	ADSR1	R/W	bit 7 = ADSR enable, 6–4 = decay, 3–0 = attack
\$x6	ADSR2	R/W	bits 7–5 = sustain level, 4–0 = sustain rate
\$x7	GAIN	R/W	direct level, or bit 7 = 1 for parameterized

Offset	Name	Access	Description
\$x8	ENVX	status	Current envelope value (0–\$7F); hardware-set
\$x9	OUTX	status	Current output sample, signed; hardware-set

ENVX and OUTX are status registers updated by the DSP itself. You can read them to inspect the current envelope value and current output sample for a voice — useful for debugging and for envelope-following effects — but writing to them isn't a normal operating-mode thing to do; the hardware will overwrite the value on the next sample.

B.2 Global registers

Address	Name	Description
\$0C	MVOLL	Main volume left (signed 8-bit)
\$1C	MVOLR	Main volume right (signed 8-bit)
\$2C	EVOLL	Echo output volume left
\$3C	EVOLR	Echo output volume right
\$4C	KON	Key-on bits; bit N keys voice N
\$5C	KOFF	Key-off bits; bit N starts release for voice N
\$6C	FLG	bit 7=RESET, 6=MUTE, 5=ECHO WRITE DISABLE, 4-0=NOISE RATE

Address	Name	Description
\$7C	ENDX	Hardware-set when a voice processes a BRR end-flag block (including looped end blocks). Treat as status, not a read-to-clear event queue.
\$0D	EFB	Echo feedback (signed 8-bit)
\$1D	—	No documented function.
\$2D	PMON	Pitch modulation enable; bit N modulates voice N by voice N-1
\$3D	NON	Noise enable; bit N replaces voice N output with noise
\$4D	EON	Echo enable; bit N sends voice N to echo input
\$5D	DIR	Sample directory page (high byte of directory address)
\$6D	ESA	Echo start address (high byte)
\$7D	EDL	Echo delay length, low 4 bits; $\text{buffer} = \text{EDL} \times 2048 \text{ bytes}$
\$0F	FIR0	FIR coefficient 0 (signed 8-bit)
\$1F	FIR1	FIR coefficient 1
\$2F	FIR2	FIR coefficient 2
\$3F	FIR3	FIR coefficient 3
\$4F	FIR4	FIR coefficient 4
\$5F	FIR5	FIR coefficient 5
\$6F	FIR6	FIR coefficient 6
\$7F	FIR7	FIR coefficient 7

B.3 FLG (\$6C) decoded

Bit	Name	Effect when set
7	RESET	Soft reset; mutes output, resets envelopes
6	MUTE	Mutes DAC output
5	ECHO WRITE DISABLE	Prevents writes to echo buffer
4-0	NOISE RATE	5-bit index into hardware noise rate table

A safe initial value during setup is %01100000 (MUTE + ECHO WRITE DISABLE, no noise). After setup, %00000000 clears all of these.

B.4 Echo configuration constants

EDL	Buffer size	Delay time
0	4 bytes (continuously overwritten at ESA)	~0 ms; disable echo writes or place ESA in scratch RAM
1	2048 bytes	~16 ms
2	4096 bytes	~32 ms
3	6144 bytes	~48 ms
4	8192 bytes	~64 ms
5	10240 bytes	~80 ms
6	12288 bytes	~96 ms
7	14336 bytes	~112 ms
8	16384 bytes	~128 ms
9-15	up to 30720 bytes	up to ~240 ms

Appendix C

Common Pitfalls and Errata

A consolidated list of pitfalls organized by category. Each one is something a real driver author has run into.

C.1 Instruction-level pitfalls

Forgetting CLRC before ADC. The carry flag is whatever the last operation left it as. If you don't clear it, your add picks up an extra 1 nondeterministically. Always CLRC before fresh adds; always SETC before fresh subtracts.

Assuming POP A/POP X/POP Y updates flags. It does not. If you pop a value and need to test it, use a CMP afterward.

SETP overlapping the stack. With P=1, direct page is at \$0100-\$01FF, the same place as the stack. If you SETP and also push, your direct-page variables get corrupted. Most drivers leave P=0 always.

Using SLEEP or STOP. With no working interrupt source on the SNES, SLEEP waits forever and STOP halts the CPU until reset. Either one stops your music. If your driver hits one, it's a bug.

Misusing DIV YA, X. The instruction is fully defined only when the quotient fits in 9 bits. The V flag receives bit 8 of the quotient. The simplest rule of thumb: **keep Y < X** to guarantee an 8-bit quotient and avoid the edge cases entirely. If you need wider quotients, use a software routine.

Treating DAA/DAS as a “decimal mode.” They are decimal-adjust-after-arithmetic instructions, not a “decimal mode” you turn on. They use the H flag from the previous ADC/SBC. For music drivers, you almost never need them.

C.2 DSP-level pitfalls

Writing to KON more than once per tick. The DSP polls KON internally on its own schedule; multiple writes between polls may collapse. Use a shadow byte and write KON once per tick.

Writing KOFF then immediately KON for the same voice. You may inadvertently restart the voice while it’s still in release, or the KOFF may cancel the KON. The discipline: clear KOFF first (let release proceed), then on the next tick or after a delay, write KON.

Forgetting to clear MUTE and RESET in FLG. No sound. Always check FLG. The default FLG state after reset has both bits set.

Echo buffer overlapping code or sample data. ESA must point to a region you’ve reserved for echo. Otherwise the DSP overwrites your code. **EDL=0 is not “echo off” — it still continuously overwrites four bytes at ESA.** If you set EDL=0, either keep echo writes disabled or aim ESA at scratch RAM you don’t care about.

Changing EDL/ESA while echo is active. Delayed writes already in flight may write to wherever the *old* configuration pointed. Disable echo writes (FLG bit 5), wait long enough for in-flight writes to complete (SNESdev errata recommend on the order of 7680 samples — about 240 ms — for safety), change config, then re-enable.

FIR coefficients producing runaway gain. If the sum of FIR coefficients (counting signs) exceeds 128, the filter amplifies. With feedback enabled, this can produce a self-sustaining drone. Keep the absolute coefficient sum reasonable.

Setting bit 7 of \$F2. Disables writes through \$F3. If your DSP writes mysteriously aren't taking effect, check whether you set bit 7 of \$F2.

Treating ENDX as a read-to-clear event queue. ENDX bits are hardware-set when a voice processes a BRR end-flag block, including looped end blocks. The standard documentation does not describe reading or writing \$7C as clearing ENDX. Use it as a hint and track per-note completion in your own driver state.

Read-modify-write on \$F1 (CONTROL). \$F1 is write-only and reads back as \$00. A pattern like `mov a,$f1 / or a,#1 / mov $f1,a` actually writes \$01, which works only by coincidence. Always write a known full byte to \$F1. Note also that any write to \$F1 affects timer state — a 0→1 transition on a timer's enable bit resets that timer's internal counter and output register.

C.3 BRR pitfalls

Loop point not on a 16-sample block boundary. The DSP loops to a BRR block address. If your loop point isn't aligned, the decoder misinterprets the data and you get noise.

Filter $\neq 0$ for the first block. The decoder's history is undefined at the start of a sample. Filter 0 at the first block is the safest default; most encoders do this automatically. If you see a click at note start, this is the first thing to check.

Filter $\neq 0$ at the loop point. When the decoder loops back, its history is from the end of the sample, which usually doesn't match what filter 1/2/3 expects. Filter 0 at the loop block is the standard safety practice — not a hardware requirement, but the easy thing to get right.

Loop waveform discontinuity. Even with filter 0, if the value at the loop point doesn't match the value just before the end, you'll click on every loop. Edit your source sample so the loop is at a zero crossing on both sides.

Filter 3 overflow. Filter 3's coefficients are aggressive. Some sample patterns can overflow the 16-bit decoder. Audit loud or transient samples; many encoders offer a "no filter 3" option.

Gaussian interpolation bug. Three consecutive maximum-negative sample values in the interpolation window can overflow the Gaussian sum and produce a loud pop. Rare in normal samples, more common in synthetic test signals.

C.4 IPL and port pitfalls

Forgetting to hide the IPL ROM after boot. Until you clear bit 7 of \$F1, the IPL ROM is mapped at \$FFC0-\$FFFF, hiding the underlying RAM. If you want those 64 bytes back, hide the ROM.

Port read/write collisions. The main CPU and SPC can write the same port at the same time. If your protocol doesn't define ownership and acknowledgment, you'll occasionally lose commands. Use counters and acks.

Assuming the main CPU can read ARAM. It cannot. Every byte that needs to cross has to go through the four-port mailbox, one byte at a time.

Reusing \$F4-\$F7 without protocol. If your driver expects a command in port 0 but the main CPU has already written a different command and your driver was busy, the first command is gone. Use a counter or queue.

C.5 Undocumented opcodes

The SPC-700 has no significant "illegal" or undocumented opcode culture. The 256-opcode space is fully defined. If a disassembler shows you a BRK at an unexpected location, it's almost certainly a bug — typically your CPU has wandered into uninitialized memory and is executing data.

The right thing to focus on is exact semantics, timing, I/O side effects, DSP polling timing, echo hazards, BRR edge cases, and port synchronization — not "secret instructions."

Appendix D

Suggested Learning Path

A 10-step progression for someone working through this material from scratch. Each step has a clear, audible result. If you can't hear something at the end of a step, don't move on.

1. SPC registers, direct page, stack, MOV. Set up a development environment (assembler + emulator with debugger). Write a program that just sets up the stack pointer and runs forever. Step through it. Confirm SP, PSW, and the registers behave as you expect. *No sound yet — but you can read the chip.*

2. Branches, loops, and timers. Add a timer-driven loop. Use Timer 0 to tick at a fixed rate. Toggle a byte in ARAM each tick. Watch it in the debugger. *Now you have a heartbeat. Music can ride on this.*

3. DSP writes. Write a simple dsp_write helper. Use it to set master volume, clear echo, and enable output. *Still silent, but the DSP is configured.*

4. Play one BRR sample. Hand-encode a single short BRR sample (or use an encoder), put it at a known address, set up the directory, configure voice 0, and key it on. *You hear sound.* This is the moment everything before this becomes worth it.

5. Pitch tables and note durations. Build a 12-note pitch table covering one octave. On each tick, decide whether it's time to change the note. Cycle through the scale. *You hear a sequence.* It will sound rough, but it will sound like something.

6. Envelopes and key-off. Add ADSR. Key-off voices when notes end. Listen for the difference: notes now have shape — they attack, sustain, and release instead of cutting off abruptly. *Your sequence sounds like instruments.*

7. Command protocol from the main CPU. Add port-polling. Write a tiny main-CPU stub (or use an emulator’s “fake main CPU” mode) that sends commands to start/stop your sequence. *You can now control music externally.*

8. Sample directory management. Add a second sample. Allow voices to choose between them. *Different instruments now exist.*

9. Multi-channel sequencing. Run multiple voices in parallel, with their own per-channel state. *You have polyphony.*

10. Echo, SFX priority, and ARAM budgeting. Add echo. Reserve voices for SFX or implement priority-based stealing. Think hard about ARAM. *Now you have a real driver.*

This path takes weeks if you’re working through it as a hobby, longer if you’re learning the prerequisites along the way. That’s normal. Each step has a sharp boundary: can you hear what you intended? If yes, move on. If no, debug.

By the end, you’re not the world’s best SPC-700 programmer. But you can read any disassembly, you can pick up any modern composing tool, and you understand what the SNES is doing every time it makes sound. That is the goal.

Appendix E

A One-Octave Pitch Table

Chapter 13, Voices, Pitch, and Envelopes, referred to “a pitch table indexed by note number” without showing one. This appendix gives a small, complete pitch table covering one octave, with the surrounding code to extend it across the full musical range.

E.1 The math

The S-DSP plays a sample at native rate (32 kHz, the rate it samples at) when the pitch register is \$1000. To play the same sample at a different musical pitch, scale the pitch register by the frequency ratio you want.

For equal temperament, the ratio between adjacent semitones is $2^{(1/12)}$. The 12 ratios for one octave, computed to four decimal places:

Semitone	Note	Ratio ($\times 2^{(n/12)}$)
0	C	1.0000
1	C#	1.0595
2	D	1.1225
3	D#	1.1892
4	E	1.2599
5	F	1.3348
6	F#	1.4142

Semitone	Note	Ratio ($\times 2^{(n/12)}$)
7	G	1.4983
8	G#	1.5874
9	A	1.6818
10	A#	1.7818
11	B	1.8877

Multiplying these by \$1000 (= 4096) and rounding to the nearest integer gives the table.

E.2 The table

*; 12 pitch values, one per semitone, for one octave.
; Each entry is a 16-bit pitch value, little-endian.
; Index 0 plays the sample at native rate (one octave below the
; entries with their high bit set).*

pitch_table:

dw \$1000 ; C
dw \$10F4 ; C#
dw \$11F6 ; D
dw \$1307 ; D#
dw \$1429 ; E
dw \$155C ; F
dw \$16A1 ; F#
dw \$17F9 ; G
dw \$1966 ; G#
dw \$1AE9 ; A
dw \$1C82 ; A#
dw \$1E34 ; B

A 4-decimal-place computation, rounded; a more precise table will differ by one or two units in the low byte of some entries. For musical purposes this precision is well within the ear's resolution.

E.3 Extending to multiple octaves

The DSP's pitch register is **14 bits wide** — values from \$0000 to \$3FFF. The native rate \$1000 is therefore not in the middle of the range; it's a quarter of the way up. From \$1000 you can go down arbitrarily (each shift right halves the rate, an octave at a time) but only *just under two octaves* upward ($\$3FFF / \$1000 \approx 3.999$). Anything that would compute a pitch $\geq \$4000$ cannot be played at that pitch — the high bits silently truncate, and you get a different note than you intended.

There are two ways to extend a base table across more octaves:

Stay above the base. Keep the base table at \$1000 and play only from there upward. You get two clean octaves: notes 0–11 from the table directly, notes 12–23 by shifting each table entry left by one. Going higher overflows the 14-bit register.

Drop the base. If you want, say, four octaves of usable range, place the base table an octave lower. The lowest octave entries become \$0800-\$0F1A (each value is half the corresponding entry in the table above), and shifting up gives you up to \$3C68 for the highest note before hitting the ceiling. This buys you more range at the cost of a slightly less-resolved low octave (fewer pitch ticks per semitone).

Going *down* is always a 16-bit shift right (divide by 2). Going *up* is a shift left (multiply by 2), provided the result fits in 14 bits.

E.4 Looking up a note

Here's a routine that takes a "note number" in A — where note 0 is the C in our base octave, note 12 is one octave up — and writes the resulting pitch to voice 0's PITCHL/PITCHH (registers \$02/\$03). To stay within the 14-bit pitch range with a base of \$1000, the note input must be in 0..23.


```
; Input: A = note number (0..23 covers two octaves with base $1000)
; Clobbers: A, X, Y, $30/$31
```

```
set_voice0_pitch:
```

```
; Split A into octave (A / 12) and semitone-in-octave (A mod 12).
mov    y, #0           ; Y = high byte of dividend = 0
mov    x, #12          ; divisor
div     ya, x           ; A = note / 12 (octave),
                        ; Y = note mod 12 (semitone in octave)
```

```
; Now A = octave (0 or 1), Y = semitone (0..11).
; Look up the base pitch from the table using Y.
```

```
push    a              ; save octave for later
mov     a, y
asl     a              ; semitone × 2 for 16-bit table indexing
mov     x, a
mov     a, !pitch_table+x
mov     $30, a         ; $30 = pitch low
mov     a, !pitch_table+1+x
mov     $31, a         ; $31 = pitch high
```

```
; Apply the octave shift: multiply the 16-bit pitch by 2 once per octave
```

```
pop     a              ; A = octave
cmp     a, #0
beq     .write
```

```
.shift_loop:
```

```
asl     $30
rol     $31
dec     a
bne     .shift_loop
```

```
.write:
```

```
; Write the 16-bit pitch to voice 0's PITCHL/PITCHH (registers $02/$03)
```

```
mov    a, #$02          ; PITCHL
mov     y, $30
movw    $f2, ya
mov     a, #$03          ; PITCHH
mov     y, $31
movw    $f2, ya
ret
```

A small subtlety in the DIV setup: DIV YA, X divides the 16-bit dividend YA by the 8-bit divisor X. Since our note number is 0..23 and arrives in A, we just zero Y (the high byte) and divide. After the DIV, A holds the quotient (octave) and Y holds the remainder (semitone).

The routine is fine for two octaves of range, which is enough to learn with. For wider ranges, real drivers typically:

- Lower the base table (e.g. \$0400 for an extra two octaves below) and accept the trade-off in low-octave precision.
- Skip the octave-shift loop in favor of a flat table (one entry per usable note across the entire range), at the cost of 2 bytes per note of ROM. A 48-note flat table is 96 bytes; a 60-note one is 120 bytes. Cheap.
- Apply detune, vibrato, and slide as 16-bit additions to the cached pitch each tick — but always clamp the result to \$0000-\$3FFF before writing, to avoid the silent-truncation bug above.

E.5 A note on tuning

Equal temperament is one tuning system among many. SNES music almost universally uses it, but if you want just intonation, a meantone temperament, or a microtonal tuning, you can — just compute the ratios for your tuning of choice and substitute them in. The DSP doesn't know what musical system you're working in; it only knows how fast to play your sample.

If your samples are *not* tuned to the standard A=440 reference, the pitch values above will produce notes that are in tune *with each other* but trans-

posed relative to other instruments. For mixed ensembles you'll want to retune samples or apply a per-instrument detune offset.

Appendix F

Embedding the SPC in a Host Application

This appendix is reserved for content covering how to embed an SPC-700 emulator inside a host application — for example, a VST or standalone audio application that uses real SPC-700 emulation as its sound source. Topics planned: the architectural distinction between SPC playback (loading and rendering .spc files), live SPC core operation (running the SPC and injecting commands via the mailbox), and full SNES stub emulation (driving the SPC the way a SNES game would). Real-time audio threading, sample-rate conversion, host-side sample/instrument management, and integration with libraries like blargg's `snescsp` are also planned.

This appendix is not yet written. Readers who want to begin a host- integration project before this content lands can study the companion C700 fork (a CLAP/AU/VST plugin built around `snescsp`) and the SNES homebrew community's tooling.

The SPC-700 is a 30-year-old, 1 MHz, 8-bit chip with 64 KiB of RAM, and people are still making new music with it because the constraints turn out to be a pleasure to work inside. Welcome to the club.